

SWITCHING THEORY AND LOGIC DESIGN

LECTURE NOTES

**B.TECH
(II YEAR – II SEM)**

Prepared by:

Sankalp Batr



(R15A0407) SWITCHING THEORY AND LOGIC DESIGN**OBJECTIVES**

This course provides in-depth knowledge of switching theory and the logic design techniques of digital circuits, which is the basis for design of any digital circuit. The course objectives are:

- To learn basic techniques for the design of digital circuits and fundamental concepts used in the design of digital systems.
- To understand common forms of number representation in digital electronic circuits and to be able to convert between different representations.
- To implement simple logical operations using combinational logic circuits
- To design combinational logic circuits, sequential logic circuits.
- To impart to student the concepts of sequential circuits, enabling them to analyze sequential systems in terms of state machines.
- To implement synchronous state machines using flip-flops.

UNIT -I:**Number System and Boolean Algebra And Switching Functions:**

Number Systems, Base Conversion Methods, Complements of Numbers, Codes- Binary Codes, Binary Coded Decimal Code and its Properties, Unit Distance Codes, Error Detecting and Correcting Codes, Hamming Code.

Boolean Algebra:

Basic Theorems and Properties, Switching Functions, Canonical and Standard Forms, Algebraic Simplification of Digital Logic Gates, Properties of XOR Gates, Universal Logic Gates. Multilevel NAND/NOR realizations.

UNIT -II:**Minimization and Design of Combinational Circuits:**

K- Map Method, up to Five variable K- Maps, Don't Care Map Entries, Prime and Essential prime Implications, Quine Mc Cluskey Tabular Method, Combinational Design, Arithmetic Circuits, Comparator, decoder, Encoder, Multiplexers, DeMultiplexers, Code Converters.

UNIT -III:**Sequential Machines Fundamentals:**

Introduction, Basic Architectural Distinctions between Combinational and Sequential circuits, classification of sequential circuits, The binary cell, The S-R-Latch Flip-Flop The D-Latch Flip-Flop, The "Clocked T" Flip-Flop, The "Clocked J-K" Flip-Flop, Design of a Clocked Flip-Flop, Conversion from one type of Flip-Flop to another, Timing and Triggering Consideration.

UNIT -IV:**Sequential Circuit Design and Analysis:**

Introduction, State Diagram, Analysis of Synchronous Sequential Circuits, Approaches to the Design of Synchronous Sequential Finite State Machines, Design Aspects, State Reduction, Design Steps, Realization using Flip-Flops Counters - Design of Ripple Counter, Synchronous counter, Ring Counter, Registers, Shift Register.

UNIT -V:**Sequential Circuits:**

Finite state machine- capabilities and limitations, Mealy and Moore models, minimization of completely specified and incompletely specified sequential machines, Partition techniques and Merger chart methods-concept of minimal cover table.

Algorithmic State Machines:

Salient features of the ASM chart-Simple examples- Weighing machine and Binary multiplier.

TEXT BOOKS:

1. Digital Design- Morris Mano, PHI, 3rd Edition.
2. Switching Theory and Logic Design-A. Anand Kumar, PHI, 2nd Edition.
3. Switching and Finite Automata Theory- Zvi Kohavi & Niraj K. Jha, 3rd Edition, Cambridge.

REFERENCE BOOKS:

1. Introduction to Switching Theory and Logic Design – Fredriac J. Hill, Gerald R. Peterson, 3rd Ed, John Wiley & Sons Inc.
2. Digital Fundamentals – A Systems Approach – Thomas L. Floyd, Pearson, 2013.
3. Switching Theory and Logic Design – Bhanu Bhaskara –Tata McGraw Hill Publication, 2012
4. Fundamentals of Logic Design- Charles H. Roth, Cengage Learning, 5th, Edition, 2004.
5. Digital Logic Applications and Design- John M. Yarbrough, Thomson Publications, 2006. 6. Digital Logic and State Machine Design – Comer, 3rd, Oxford, 2013.

OUTCOMES

Upon completion of the course, student should possess the following skills:

- Be able to manipulate numeric information in different forms
- Be able to manipulate simple Boolean expressions using the theorems and postulates of Boolean algebra and to minimize combinational functions.
- Be able to design and analyze small combinational circuits and to use standard combinational functions to build larger more complex circuits.
- Be able to design and analyze small sequential circuits and to use standard sequential functions to build larger more complex circuits.

UNIT - 1

NUMBER SYSTEMS & CODES

- Introduction about digital system
- Philosophy of number systems
- Complement representation of negative numbers
- Binary arithmetic
- Binary codes
- Error detecting & error correcting codes
- Hamming codes

INTRODUCTION ABOUT DIGITAL SYSTEM

A Digital system is an interconnection of digital modules and it is a system that manipulates discrete elements of information that is represented internally in the binary form.

Now a day's digital systems are used in wide variety of industrial and consumer products such as automated industrial machinery, pocket calculators, microprocessors, digital computers, digital watches, TV games and signal processing and so on.

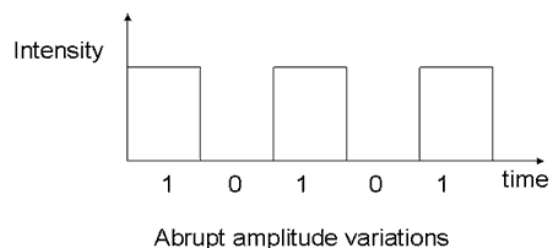
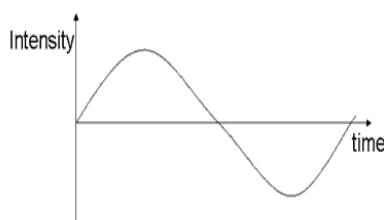
Characteristics of Digital systems

- Digital systems manipulate discrete elements of information.
- Discrete elements are nothing but the digits such as 10 decimal digits or 26 letters of alphabets and so on.
- Digital systems use physical quantities called signals to represent discrete elements.
- In digital systems, the signals have two discrete values and are therefore said to be binary.
- A signal in digital system represents one binary digit called a bit. The bit has a value either 0 or 1.

Analog systems vs Digital systems

Analog system process information that varies continuously i.e; they process time varying signals that can take on any values across a continuous range of voltage, current or any physical parameter.

Digital systems use digital circuits that can process digital signals which can take either 0 or 1 for binary system.



Advantages of Digital system over Analog system

1. Ease of programmability

The digital systems can be used for different applications by simply changing the program without additional changes in hardware.

2. Reduction in cost of hardware

The cost of hardware gets reduced by use of digital components and this has been possible due to advances in IC technology. With ICs the number of components that can be placed in a given area of Silicon are increased which helps in cost reduction.

3. High speed

Digital processing of data ensures high speed of operation which is possible due to advances in Digital Signal Processing.

4. High Reliability

Digital systems are highly reliable one of the reasons for that is use of error correction codes.

5. Design is easy

The design of digital systems which require use of Boolean algebra and other digital techniques is easier compared to analog designing.

6. Result can be reproduced easily

Since the output of digital systems unlike analog systems is independent of temperature, noise, humidity and other characteristics of components the reproducibility of results is higher in digital systems than in analog systems.

Disadvantages of Digital Systems

- Use more energy than analog circuits to accomplish the same tasks, thus producing more heat as well.
- Digital circuits are often fragile, in that if a single piece of digital data is lost or misinterpreted the meaning of large blocks of related data can completely change.
- Digital computer manipulates discrete elements of information by means of a binary code.
- Quantization error during analog signal sampling.

NUMBER SYSTEM

Number system is a basis for counting various items. Modern computers communicate and operate with binary numbers which use only the digits 0 & 1. Basic number system used by humans is Decimal number system.

For Ex: Let us consider decimal number 18. This number is represented in binary as 10010.

We observe that binary number system takes more digits to represent the decimal number. For large numbers we have to deal with very large binary strings. So this fact gave rise to three new number systems.

- i) Octal number systems
- ii) Hexa Decimal number system
- iii) Binary Coded Decimal number (BCD) system

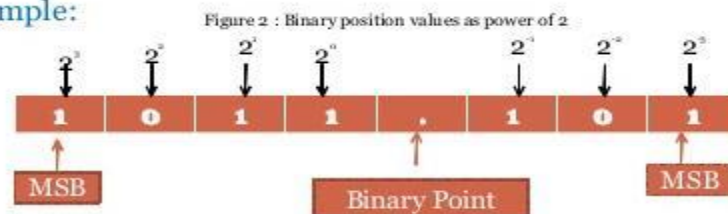
To define any number system we have to specify

- Base of the number system such as 2, 8, 10 or 16.
- The base decides the total number of digits available in that number system.
- First digit in the number system is always zero and last digit in the number system is always base-1.

Binary number system:

The binary number has a radix of 2. As $r = 2$, only two digits are needed, and these are 0 and 1. In binary system weight is expressed as power of 2.

• Example:



The left most bit, which has the greatest weight is called the Most Significant Bit (MSB). And the right most bit which has the least weight is called Least Significant Bit (LSB).

For Ex: $1001.01_2 = [(1) \times 2^3] + [(0) \times 2^2] + [(0) \times 2^1] + [(1) \times 2^0] + [(0) \times 2^{-1}] + [(1) \times 2^{-2}]$

$$1001.01_2 = [1 \times 8] + [0 \times 4] + [0 \times 2] + [1 \times 1] + [0 \times 0.5] + [1 \times 0.25]$$

$$1001.01_2 = 9.25_{10}$$

Decimal Number system

The decimal system has ten symbols: 0,1,2,3,4,5,6,7,8,9. In other words, it has a base of 10.

Octal Number System

Digital systems operate only on binary numbers. Since binary numbers are often very long, two shorthand notations, octal and hexadecimal, are used for representing large binary numbers. Octal systems use a base or radix of 8. It uses first eight digits of decimal number system. Thus it has digits from 0 to 7.

Hexa Decimal Number System

The hexadecimal numbering system has a base of 16. There are 16 symbols. The decimal digits 0 to 9 are used as the first ten digits as in the decimal system, followed by the letters A, B, C, D, E and F, which represent the values 10, 11,12,13,14 and 15 respectively.

Decimal	Binary	Octal	Hexadecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Number Base conversions

The human beings use decimal number system while computer uses binary number system. Therefore it is necessary to convert decimal number system into its equivalent binary.

- i) Binary to octal number conversion
- ii) Binary to hexa decimal number conversion

The binary number: 001 010 011 000 100 101 110 111
The octal number: 1 2 3 0 4 5 6 7

The binary number: 0001 0010 0100 1000 1001 1010 1101 1111
The hexadecimal number: 1 2 5 8 9 A D F

- iii) Octal to binary Conversion

Each octal number converts to 3 binary digits

Code
0 - 000
1 - 001
2 - 010
3 - 011
4 - 100
5 - 101
6 - 110
7 - 111

To convert 653_8 to binary, just substitute code:

6 5 3
↓ ↓ ↓
110 101 011

- iv) Hexa to binary conversion

4 F D 7
↓ ↓ ↓ ↓
0100 1111 1101 0111

www.electronics-tutorials.com

- v) Octal to Decimal conversion

Ex: convert 4057.06_8 to decimal

$$=4 \times 8^3 + 0 \times 8^2 + 5 \times 8^1 + 7 \times 8^0 + 0 \times 8^{-1} + 6 \times 8^{-2}$$

$$=2048 + 0 + 40 + 7 + 0 + 0.0937$$

$$=2095.0937_{10}$$

vi) Decimal to Octal Conversion

Ex: convert 378.93_{10} to octal

378_{10} to octal: Successive division:

$$\begin{array}{r} 8 \overline{) 378} \\ \underline{8 \mid 47 \text{ --- } 2} \\ 8 \overline{) 5 \text{ --- } 7} \quad \uparrow \\ \underline{0 \text{ --- } 5} \end{array}$$

$$=572_8$$

0.93_{10} to octal :

$$0.93 \times 8 = 7.44$$

$$0.44 \times 8 = 3.52$$

$$0.53 \times 8 = 4.16$$

$$0.16 \times 8 = 1.28$$

$$=0.7341_8$$

$$378.93_{10} = 572.7341_8$$

vii) Hexadecimal to Decimal Conversion

Ex: $5C7_{16}$ to decimal

$$= (5 \times 16^2) + (C \times 16^1) + (7 \times 16^0)$$

$$= 1280 + 192 + 7$$

$$= 147_{10}$$

viii) Decimal to Hexadecimal Conversion

Ex: 2598.6751_{10}

$$\begin{array}{r} 16 \overline{) 2598} \\ \underline{16 \mid 162 \text{ --- } -6} \\ 10 \text{ --- } -2 \end{array}$$

$$= A26_{(16)}$$

$$0.675_{10} = 0.675 \times 16 \rightarrow 10.8$$

$$= 0.800 \times 16 \rightarrow 12.8 \quad \downarrow$$

$$= 0.800 \times 16 \rightarrow 12.8$$

$$= 0.800 \times 16 \rightarrow 12.8$$

$$= 0.ACCC_{16}$$

$$2598.675_{10} = A26.ACCC_{16}$$

ix) Octal to hexadecimal conversion:

The simplest way is to first convert the given octal no. to binary & then the binary no. to hexadecimal.

Ex: 756.603_8

7	5	6	.	6	0	3
111	101	110	.	110	000	011
0001	1110	1110	.	1100	0001	1000
1	E	E	.	C	1	8

x) Hexadecimal to octal conversion:

First convert the given hexadecimal no. to binary & then the binary no. to octal.

Ex: $B9F.AE_{16}$

B	9	F	.	A	E		
1011	1001	1111	.	1010	1110		
101	110	011	111	.	101	011	100
5	6	3	7	.	5	3	4

$$= 5637.534$$

Complements:

In digital computers to simplify the subtraction operation & for logical manipulation complements are used. There are two types of complements used in each radix system.

- The radix complement or r 's complement
- The diminished radix complement or $(r-1)$'s complement

Representation of signed no.s binary arithmetic in computers:

- Two ways of rep signed no.s
 1. Sign Magnitude form
 2. Complement form
- Two complimented forms
 1. 1's complement form
 2. 2's complement form

Advantage of performing subtraction by the compliment method is reduction in the hardware.(instead of addition & subtraction only adding ckt's are needed.)

i.e, subtraction is also performed by adders only.

Instead of subtracting one no. from other the compliment of the subtrahend is added to minuend. In sign magnitude form, an additional bit called the sign bit is placed in front of the no. If the sign bit is 0, the no. is +ve, If it is a 1, the no is -ve.

Ex:

0	1	0	1	0	0	1
---	---	---	---	---	---	---



Sign bit

=+41 magnitude

1	1	0	1	0	0	1
---	---	---	---	---	---	---

$$= -41$$

Note: manipulation is necessary to add a +ve no to a -ve no

Representation of signed no.s using 2's or 1's complement method:

If the no. is +ve, the magnitude is rep in its true binary form & a sign bit 0 is placed in front of the MSB. If the no is _ve , the magnitude is rep in its 2's or 1's compliment form & a sign bit 1 is placed in front of the MSB.

Ex:

Given no.	Sign mag form	2's comp form	1's comp form
01101	+13	+13	+13
010111	+23	+23	+23
10111	-7	-7	-8
1101010	-42	-22	-21

Special case in 2's comp representation:

Whenever a signed no. has a 1 in the sign bit & all 0's for the magnitude bits, the decimal equivalent is -2^n , where n is the no of bits in the magnitude .

Ex: 1000= -8 & 10000=-16

Characteristics of 2's compliment no.s:

Properties:

1. There is one unique zero
2. 2's comp of 0 is 0
3. The leftmost bit can't be used to express a quantity . it is a 0 no. is +ve.
4. For an n-bit word which includes the sign bit there are $(2^{n-1}-1)$ +ve integers, 2^{n-1} -ve integers & one 0 , for a total of 2^n unique states.
5. Significant information is contained in the 1's of the +ve no.s & 0's of the -ve no.s
6. A -ve no. may be converted into a +ve no. by finding its 2's comp.

Signed binary numbers:

Decimal	Sign 2's comp form	Sign 1's comp form	Sign mag form
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0011	0011	0011
+0	0000	0000	0000

-0	--	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111
8	1000	--	--

Methods of obtaining 2's comp of a no:

- In 3 ways
 1. By obtaining the 1's comp of the given no. (by changing all 0's to 1's & 1's to 0's) & then adding 1.
 2. By subtracting the given n bit no N from 2^n
 3. Starting at the LSB , copying down each bit upto & including the first 1 bit encountered , and complimenting the remaining bits.
- Ex: Express -45 in 8 bit 2's comp form

+45 in 8 bit form is 00101101

I method:

1's comp of 00101101 & the add 1

00101101

11010010

+1

— — — — — — — — — —

11010011

is 2's comp form

II method:

Subtract the given no. N from 2^n

$2^n = 100000000$

Subtract 45 = -00101101

+1

— — — —

11010011

is 2's comp

III method:

Original no: 00101101

Copy up to First 1 bit 1

Compliment remaining : 1101001

bits

11010011

Ex:

-73.75 in 12 bit 2's comp form

I method

$$\begin{array}{r} 01001001.1100 \\ 10110110.0011 \\ +1 \\ \hline \end{array}$$

10110110.0100 is 2's

II method:

$$2^8 = 100000000.0000$$

$$\text{Sub } 73.75 = -01001001.1100$$

$$\begin{array}{r} \hline 10110110.0100 \text{ is 2's comp} \end{array}$$

III method :

Originalno : 01001001.1100

Copy up to 1'st bit : 100

Comp the remaining bits: 10110110.0

$$\begin{array}{r} \hline 10110110.0100 \end{array}$$

2's compliment Arithmetic:

- The 2's comp system is used to rep -ve no.s using modulus arithmetic . The word length of a computer is fixed. i.e, if a 4 bit no. is added to another 4 bit no . the result will be only of 4 bits. Carry if any , from the fourth bit will overflow called the Modulus arithmetic.

$$\text{Ex: } 1100 + 1111 = 1011$$

- In the 2's compl subtraction, add the 2's comp of the subtrahend to the minuend . If there is a carry out , ignore it , look at the sign bit I.e, MSB of the sum term .If the MSB is a 0, the result is positive.& it is in true binary form. If the MSB is a 1 (carry in or no carry at all) the result is negative.& is in its 2's comp form. Take its 2's comp to find its magnitude in binary.

Ex: Subtract 14 from 46 using 8 bit 2's comp arithmetic:

$$\begin{array}{rcl} +14 & = & 00001110 \\ -14 & = & 11110010 \quad \text{2's comp} \\ +46 & = & 00101110 \\ -14 & = & +11110010 \quad \text{2's comp form of -14} \\ \hline \end{array}$$

$$\begin{array}{r} \text{---} \\ -32 \quad (1)00100000 \end{array}$$
 ignore carry
 Ignore carry, The MSB is 0. so the result is +ve. & is in normal binary form. So the result is +00100000=+32.

EX: Add -75 to +26 using 8 bit 2's comp arithmetic

$$\begin{array}{r} +75 = 01001011 \\ -75 = 10110101 \quad \text{2's comp} \\ +26 = 00011010 \\ -75 = +10110101 \quad \text{2's comp form of -75} \\ \hline -49 \quad 11001111 \end{array}$$
 No carry

No carry, MSB is a 1, result is -ve & is in 2's comp. The magnitude is 2's comp of 11001111. i.e, 00110001 = 49. so result is -49

Ex: add -45.75 to +87.5 using 12 bit arithmetic

$$\begin{array}{r} +87.5 = 01010111.1000 \\ -45.75 = +11010010.0100 \end{array}$$

$$\begin{array}{r} \text{---} \\ -41.75 \quad (1)00101001.1100 \end{array}$$
 ignore carry
 MSB is 0, result is +ve. =+41.75

1's compliment of n number:

- It is obtained by simply complimenting each bit of the no., & also, 1's comp of a no, is subtracting each bit of the no. from 1. This complemented value rep the -ve of the original no. One of the difficulties of using 1's comp is its rep of zero. Both 00000000 & its 1's comp 11111111 rep zero.
- The 00000000 called +ve zero & 11111111 called -ve zero.

Ex: -99 & -77.25 in 8 bit 1's comp

$$\begin{array}{r} +99 = 01100011 \\ -99 = 10011100 \\ \\ +77.25 = 01001101.0100 \\ -77.25 = 10110010.1011 \end{array}$$

1's compliment arithmetic:

In 1's comp subtraction, add the 1's comp of the subtrahend to the minuend. If there is a carryout, bring the carry around & add it to the LSB called the **end around carry**. Look at the sign bit (MSB). If this is a 0, the result is +ve & is in true binary. If the MSB is a 1 (carry or no carry), the result is -ve & is in its comp form. Take its 1's comp to get the magnitude in binary.

Ex: Subtract 14 from 25 using 8 bit 1's

EX: ADD -25 to +14

$$\begin{array}{r} 25 = 00011001 \\ -45 = 11110001 \\ \hline +11 \quad \quad (1)00001010 \end{array}$$

+1

$$\begin{array}{r} \hline 00001011 \end{array}$$

MSB is a 0 so result is +ve (binary)

$$=+11_{10}$$

$$\begin{array}{r} +14 = 00001110 \\ -25 = +11100110 \\ \hline -11 \quad \quad 11110100 \end{array}$$

No carry MSB =1
result=-ve=-11₁₀

Binary codes

Binary codes are codes which are represented in binary system with modification from the original ones.

- ☐ Weighted Binary codes
- ☐ Non Weighted Codes

Weighted binary codes are those which obey the positional weighting principles, each position of the number represents a specific weight. The binary counting sequence is an example.

Decimal	BCD 8421	Excess-3	84-2-1	2421	5211	Bi-Quinary 5043210			5	0	4	3	2	1	0
0	0000	0011	0000	0000	0000	0100001		0		X					X
1	0001	0100	0111	0001	0001	0100010		1		X				X	
2	0010	0101	0110	0010	0011	0100100		2		X			X		
3	0011	0110	0101	0011	0101	0101000		3		X		X			
4	0100	0111	0100	0100	0111	0110000		4		X	X				
5	0101	1000	1011	1011	1000	1000001		5	X						X
6	0110	1001	1010	1100	1010	1000010		6	X					X	
7	0111	1010	1001	1101	1100	1000100		7	X				X		
8	1000	1011	1000	1110	1110	1001000		8	X			X			
9	1001	1111	1111	1111	1111	1010000		9	X		X				

Reflective Code

A code is said to be reflective when code for 9 is complement for the code for 0, and

so is for 8 and 1 codes, 7 and 2, 6 and 3, 5 and 4. Codes 2421, 5211, and excess-3 are reflective, whereas the 8421 code is not.

Sequential Codes

A code is said to be sequential when two subsequent codes, seen as numbers in binary representation, differ by one. This greatly aids mathematical manipulation of data. The 8421 and Excess-3 codes are sequential, whereas the 2421 and 5211 codes are not.

Non weighted codes

Non weighted codes are codes that are not positionally weighted. That is, each position within the binary number is not assigned a fixed value. Ex: Excess-3 code

Excess-3 Code

Excess-3 is a non weighted code used to express decimal numbers. The code derives its name from the fact that each binary code is the corresponding 8421 code plus 0011(3).

Gray Code

The gray code belongs to a class of codes called minimum change codes, in which only one bit in the code changes when moving from one code to the next. The Gray code is non-weighted code, as the position of bit does not contain any weight. The gray code is a reflective digital code which has the special property that any two subsequent numbers codes differ by only one bit. This is also called a unit- distance code. In digital Gray code has got a special place.

Decimal Number	Binary Code	Gray Code	Decimal Number	Binary Code	Gray Code
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

Binary to Gray Conversion

- ☐ Gray Code MSB is binary code MSB.
- ☐ Gray Code MSB-1 is the XOR of binary code MSB and MSB-1.
- ☐ MSB-2 bit of gray code is XOR of MSB-1 and MSB-2 bit of binary code.
- ☐ MSB-N bit of gray code is XOR of MSB-N-1 and MSB-N bit of binary code.

8421 BCD code (Natural BCD code):

Each decimal digit 0 through 9 is coded by a 4 bit binary no. called natural binary codes. Because of the 8,4,2,1 weights attached to it. It is a weighted code & also sequential . it is useful for mathematical operations. The advantage of this code is its ease of conversion to & from decimal. It is less efficient than the pure binary, it requires more bits.

Ex: 14 → 1110 in binary

But as 0001 0100 in 8421 code.

The disadvantage of the BCD code is that , arithmetic operations are more complex than they are in pure binary . There are 6 illegal combinations 1010,1011,1100,1101,1110,1111 in these codes, they are not part of the 8421 BCD code system . The disadvantage of 8421 code is, the rules of binary addition 8421 no, but only to the individual 4 bit groups.

BCD Addition:

It is individually adding the corresponding digits of the decimal no,s expressed in 4 bit binary groups starting from the LSD . If there is no carry & the sum term is not an illegal code , no correction is needed .If there is a carry out of one group to the next group or if the sum term is an illegal code then $6_{10}(0100)$ is added to the sum term of that group & the resulting carry is added to the next group.

Ex: Perform decimal additions in 8421 code

(a) 25+13

In BCD 25 = 0010 0101

In BCD +13 = +0001 0011

38 0011 1000

No carry , no illegal code .This is the corrected sum

(b). 679.6 + 536.8

679.6 = 0110 0111 1001 .0110 in BCD
 +536.8 = +0101 0011 0010 .1000 in BCD

 1216.4 1011 1010 0110 . 1110 illegal codes
 +0110 + 0011 +0110 . + 0110 add 0110 to each

(1)0001 (1)0000 (1)0101 . (1)0100 propagate carry
 / / / /
 +1 +1 +1 +1

 0001 0010 0001 0110 . 0100

 1 2 1 6 . 4

BCD Subtraction:

Performed by subtracting the digits of each 4 bit group of the subtrahend the digits from the corresponding 4- bit group of the minuend in binary starting from the LSD . if there is no borrow from the next group , then $6_{10}(0110)$ is subtracted from the difference term of this group.

(a)38-15

In BCD 38= 0011 1000
 In BCD -15 = -0001 0101

 23 0010 0011

No borrow, so correct difference.

(b) 206.7-147.8

206.7 = 0010 0000 0110 . 0111 in BCD
 -147.8 = -0001 0100 0111 . 0110 in BCD

 58.9 0000 1011 1110 . 1111 borrows are present
 -0110 -0110 . -0110 subtract 0110

 0101 1000 . 1001

BCD Subtraction using 9's & 10's compliment methods:

Form the 9's & 10's compliment of the decimal subtrahend & encode that no. in the 8421 code . the resulting BCD no.s are then added.

EX: 305.5 – 168.8

$$\begin{array}{r} 305.5 = 305.5 \\ -168.8 = +83.1 \quad \text{9's comp of -168.8} \\ \hline (1)136.6 \\ \quad +1 \quad \text{end around carry} \\ \quad \mathbf{136.7} \quad \text{corrected difference} \end{array}$$

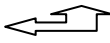
$$\begin{array}{r} 305.5_{10} = 0011 \ 0000 \ 0101 \ . \ 0101 \\ +831.1_{10} = +1000 \ 0011 \ 0001 \ . \ 0001 \quad \text{9's comp of 168.8 in BCD} \\ \hline +1011 \ 0011 \ 0110 \ . \ 0110 \quad \text{1011 is illegal code} \\ +0110 \quad \text{add 0110} \\ \hline (1)0001 \ 0011 \ 0110 \ . \ 0110 \quad +1 \quad \text{End around carry} \\ \hline 0001 \ 0011 \ 0110 \ . \ 0111 \\ \quad \quad \quad = 136.7 \end{array}$$

Excess three(xs-3)code:

It is a non-weighted BCD code .Each binary codeword is the corresponding 8421 codeword plus 0011(3).It is a sequential code & therefore , can be used for arithmetic operations..It is a self-complementing code.s o the subtraction by the method of compliment addition is more direct in xs-3 code than that in 8421 code. The xs-3 code has six invalid states 0000,0010,1101,1110,1111.. It has interesting properties when used in addition & subtraction.

Excess-3 Addition:

Add the xs-3 no.s by adding the 4 bit groups in each column starting from the LSD. If there is no carry starting from the addition of any of the 4-bit groups , subtract 0011 from the sum term of those groups (because when 2 decimal digits are added in xs-3 & there is no carry , result in xs-6). If there is a carry out, add 0011 to the sum term of those groups(because when there is a carry, the invalid states are skipped and the result is normal binary).

EX:	37	0110	1010	
	+28	+0101	1011	
	— — — — —			
	65	1011	(1)0101	carry generated
		+1		propagate carry
		— — — — —		
		1100	0101	add 0011 to correct 0101 &
		-0011	+0011	subtract 0011 to correct 1100
		— — — — —		
		1001	1000	=65 ₁₀

Excess -3 (XS-3) Subtraction:

Subtract the xs-3 no.s by subtracting each 4 bit group of the subtrahend from the corresponding 4 bit group of the minuend starting from the LSD .if there is no borrow from the next 4-bit group add 0011 to the difference term of such groups (because when decimal digits are subtracted in xs-3 & there is no borrow , result is normal binary). If there is a borrow , subtract 0011 from the difference term (because taking a borrow is equivalent to adding six invalid states , result is in xs-6)

Ex: 267-175

267 =	0101	1001	1010	
-175 =	-0100	1010	1000	
	— — — — —			
	0000	1111	0010	
	+0011	-0011	+0011	
	— — — — —			
	0011	1100	+0011	=92 ₁₀

Xs-3 subtraction using 9's & 10's compliment methods:

Subtraction is performed by the 9's compliment or 10's compliment

Ex: 687-348 The subtrahend (348) xs -3 code & its compliment are:

9's comp of 348 = 651

Xs-3 code of 348 = 0110 0111 1011

1's comp of 348 in xs-3 = 1001 1000 0100

Xs=3 code of 348 in xs=3 = 1001 1000 0100

687	→	687
-348		+651 9's compl of 348

—	—
339	(1)338
	+1 end around carry
—	—

—	—
339	corrected difference in decimal

1001	1011	1010	687 in xs-3
+1001	1000	0100	1's comp 348 in xs-3
—	—	—	
—			

```

      (1)0010  (1)0011      1110      carry generated
//
+1           +1           propagate carry
-----
(1)0011      0010  1110
                        +1      end around carry
-----

      0011      0011      1111      (correct 1111 by sub0011 and
+0011      +0011      +0011      correct both groups of 0011 by
-----
      adding 0011)
      --
      -
      0110      0110      1100      corrected diff in xs-3 = 33010

```

The Gray code (reflective –code):

Gray code is a non-weighted code & is not suitable for arithmetic operations. It is not a BCD code . It is a cyclic code because successive code words in this code differ in one bit position only i.e, it is a unit distance code. Popular of the unit distance code. It is also a reflective code i.e, both reflective & unit distance. The n least significant bits for 2^n through $2^{n+1}-1$ are the mirror images of those for 0 through 2^n-1 . An N bit gray code can be obtained by reflecting an $N-1$ bit code about an axis at the end of the code, & putting the MSB of 0 above the axis & the MSB of 1 below the axis.

Reflection of gray codes:

Gray Code				Decimal	4 bit binary
1 bit	2 bit	3 bit	4 bit		
0	00	000	0000	0	0000
1	01	001	0001	1	0001
	11	011	0011	2	0010
	10	010	0010	3	0011
		110	0110	4	0100
		111	0111	5	0101
		101	0101	6	0110
		110	0100	7	0111

			1100	8	1000
			1101	9	1001
			1111	10	1010
			1110	11	1011
			1010	12	1100
			1011	13	1101
			1001	14	1110
			1000	15	1111

Binary to Gray conversion:

N bit binary no is rep by $B_n B_{n-1} \dots B_1$

Gray code equivalent is by $G_n G_{n-1} \dots G_1$

B_n, G_n are the MSB's then the gray code bits are obtained from the binary code as

$G_n = B_n$	$G_{n-1} = B_n \oplus B_{n-1}$	$G_{n-2} = B_{n-1} \oplus B_{n-2}$	-----	$G_1 = B_2 \oplus B_1$	
-------------	--------------------------------	------------------------------------	-------	------------------------	--

$\oplus \rightarrow$ EX-or symbol

Procedure: ex-or the bits of the binary no with those of the binary no shifted one position to the right. The LSB of the shifted no. is discarded & the MSB of the gray code no. is the same as the MSB of the original binary no.

EX: 10001 $\oplus \oplus \oplus$

(a). Binary : 1 $\rightarrow$ 0 $\rightarrow$ 0 $\rightarrow$ 1

Gray : 1 1 0 1

(b). Binary: 1 0 0 1

Shifted binary: 0 0 (1)

— — — — —
1 1 0 1 $\rightarrow$ gray

Gray to Binary Conversion:

If an n bit gray no. is rep by $G_n G_{n-1} \dots G_1$

its binary equivalent by $B_n B_{n-1} \dots B_1$ then the binary bits are obtained from gray bits as

$B_n = G_n$	$B_{n-1} = B_n \oplus G_{n-1}$	$B_{n-2} = B_{n-1} \oplus G_{n-2}$	-----	$B_1 = B_2 \oplus G_1$
-------------	--------------------------------	------------------------------------	-------	------------------------

To convert no. in any system into given no. first convert it into binary & then binary to gray. To convert gray no into binary no & convert binary no into require no system.

Ex: $10110010(\text{gray}) = 11011100_2 = DC_{16} = 334_8 = 220_{10}$

EX: 1101

Gray: 1 1 0 1



Binary: 1 0 0 1

Ex: $3A7_{16} = 0011,1010,0111_2 = 1001110100(\text{gray})$

$527_8 = 101,011,011_2 = 111110110(\text{gray})$

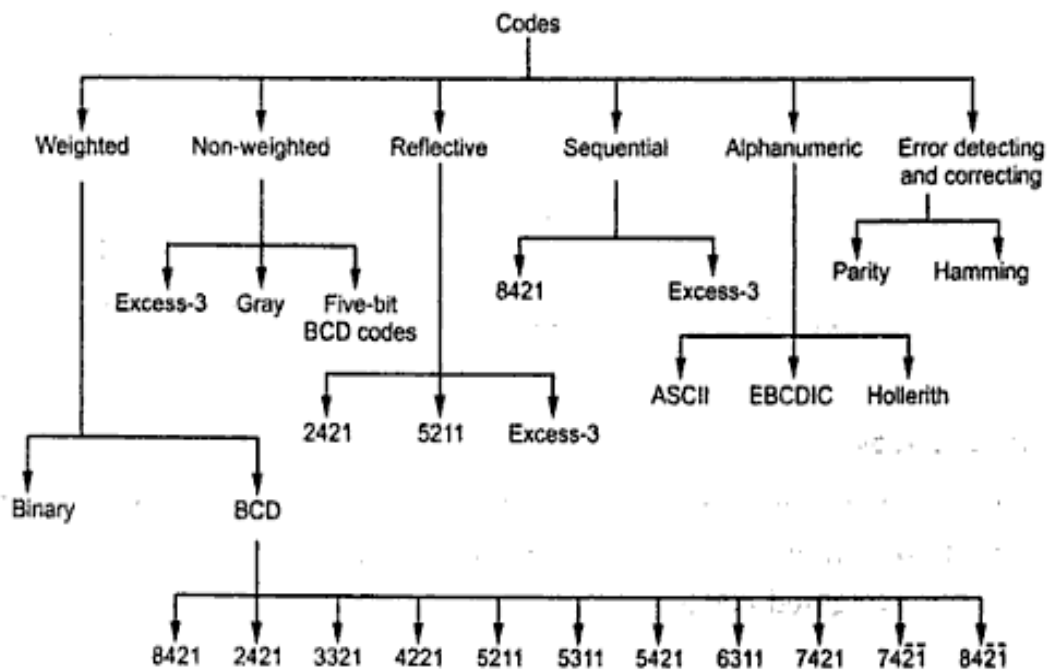
$652_{10} = 1010001100_2 = 1111001010(\text{gray})$

XS-3 gray code:

In a normal gray code , the bit patterns for 0(0000) & 9(1101) do not have a unit distance between them i.e, they differ in more than one position. In xs-3 gray code , each decimal digit is encoded with gray code patter of the decimal digit that is greater by 3. It has a unit distance between the patterns for 0 & 9.

XS-3 gray code for decimal digits 0 through 9

Decimal digit	Xs-3 gray code	Decimal digit	Xs-3 gray code
0	0010	5	1100
1	0110	6	1101
2	0111	7	1111
3	0101	8	1110
4	0100	9	1010



Binary codes block diagram

Error – Detecting codes: When binary data is transmitted & processed, it is susceptible to noise that can alter or distort its contents. The 1's may get changed to 0's & 1's. Because digital systems must be accurate to the digit, error can pose a problem. Several schemes have been devised to detect the occurrence of a single bit error in a binary word, so that whenever such an error occurs the concerned binary word can be corrected & retransmitted.

Parity: The simplest technique for detecting errors is that of adding an extra bit known as parity bit to each word being transmitted. Two types of parity: Odd parity, even parity. For odd parity, the parity bit is set to a _0_ or a _1_ at the transmitter such that the total no. of 1 bit in the word including the parity bit is an odd no. For even parity, the parity bit is set to a _0_ or a _1_ at the transmitter such that the parity bit is an even no.

Decimal	8421 code	Odd parity	Even parity
0	0000	1	0
1	0001	0	1
2	0010	0	1
3	0011	1	0
4	0100	0	1
5	0100	1	0
6	0110	1	0
7	0111	0	1
8	1000	0	1
9	1001	1	0

When the digit data is received, a parity checking circuit generates an error signal if the total no. of 1's is even in an odd parity system or odd in an even parity system. This parity check can always detect a single bit error but cannot detect 2 or more errors within the same word. Odd parity is used more often than even parity does not detect the situation. Where all 0's are created by a short ckt or some other fault condition.

Ex: Even parity scheme

(a) 10101010 (b) 11110110 (c) 10111001

Ans:

- (a) No. of 1's in the word is even is 4 so there is no error
- (b) No. of 1's in the word is even is 6 so there is no error
- (c) No. of 1's in the word is odd is 5 so there is error

Ex: odd parity

(a) 10110111 (b) 10011010 (c) 11101010

Ans:

- (a) No. of 1's in the word is even is 6 so word has error
- (b) No. of 1's in the word is even is 4 so word has error
- (c) No. of 1's in the word is odd is 5 so there is no error

Checksums:

Simple parity can't detect two errors within the same word. To overcome this, use a sort of 2 dimensional parity. As each word is transmitted, it is added to the sum of the previously transmitted words, and the sum retained at the transmitter end. At the end of transmission, the sum called the check sum. Up to that time sent to the receiver. The receiver can check its sum with the transmitted sum. If the two sums are the same, then no errors were detected at the receiver end. If there is an error, the receiving location can ask for retransmission of the entire data, used in teleprocessing systems.

Block parity:

Block of data shown is create the row & column parity bits for the data using odd parity. The parity bit 0 or 1 is added column wise & row wise such that the total no. of 1's in each column & row including the data bits & parity bit is odd as

Data	Parity bit	data
10110	0	10110
10001	1	10001
10101	0	10101
00010	0	00010
11000	1	11000
00000	1	00000
11010	0	11010

Error –Correcting Codes:

A code is said to be an error –correcting code, if the code word can always be deduced from an erroneous word. For a code to be a single bit error correcting code, the minimum distance of that code must be three. The minimum distance of that code is the smallest no. of bits by which any two code words must differ. A code with minimum distance of 3 can't only correct single bit errors but also detect (can't correct) two bit errors, The key to error correction is that it must be possible to detect & locate erroneous that it must be possible to detect & locate erroneous digits. If the location of an error has been determined. Then by complementing the erroneous digit, the message can be corrected , error correcting , code is the Hamming code , In this , to each group of m information or message or data bits, K parity checking bits denoted by P₁,P₂,-----p_k located at positions 2^{k-1} from left are added to form an (m+k) bit code word. To correct the error, k parity checks are performed on selected digits of each code word, & the position of the error bit is located by forming an error word, & the error bit is then complemented. The k bit error word is generated by putting a 0 or a 1 in the 2^{k-1} th position depending upon whether the check for parity involving the parity bit P_k is satisfied or not. Error positions & their corresponding values :

Error Position	For 15 bit code C ₄ C ₃ C ₂ C ₁	For 12 bit code C ₄ C ₃ C ₂ C ₁	For 7 bit code C ₃ C ₂ C ₁
0	0 0 0 0	0 0 0 0	0 0 0
1	0 0 0 1	0 0 0 1	0 0 1
2	0 0 1 0	0 0 1 0	0 1 0
3	0 0 1 1	0 0 1 1	0 1 1
4	0 1 0 0	0 1 0 0	1 0 0
5	0 1 0 1	0 1 0 1	1 0 1
6	0 1 1 0	0 1 1 0	1 1 0
7	0 1 1 1	0 1 1 1	1 1 1
8	1 0 0 0	1 0 0 0	
9	1 0 0 1	1 0 0 1	
10	1 0 1 0	1 0 1 0	
11	1 0 1 1	1 0 1 1	
12	1 1 0 0	1 1 0 0	
13	1 1 0 1		
14	1 1 1 0		
15	1 1 1 1		

7-bit Hamming code:

To transmit four data bits, 3 parity bits located at positions 2^0 , 2^1 & 2^2 from left are added to make a 7 bit codeword which is then transmitted.

The word format

P ₁	P ₂	D ₃	P ₄	D ₅	D ₆	D ₇
----------------	----------------	----------------	----------------	----------------	----------------	----------------

D—Data bits P-

Parity bits

Decimal Digit	For BCD P ₁ P ₂ D ₃ P ₄ D ₅ D ₆ D ₇	For Excess-3 P ₁ P ₂ D ₃ P ₄ D ₅ D ₆ D ₇
0	0 0 0 0 0 0 0	1 0 0 0 0 1 1
1	1 1 0 1 0 0 1	1 0 0 1 1 0 0
2	0 1 0 1 0 1 1	0 1 0 0 1 0 1
3	1 0 0 0 0 1 1	1 1 0 0 1 1 0
4	1 0 0 1 1 0 0	0 0 0 1 1 1 1
5	0 1 0 0 1 0 1	1 1 1 0 0 0 0
6	1 1 0 0 1 1 0	0 0 1 1 0 0 1
7	0 0 0 1 1 1 1	1 0 1 1 0 1 0
8	1 1 1 0 0 0 0	0 1 1 0 0 1 1
9	0 0 1 1 0 0 1	0 1 1 1 1 0 0

Ex: Encode the data bits 1101 into the 7 bit even parity Hamming Code

The bit pattern is

P₁P₂D₃P₄D₅D₆D₇

1 1 0 1

Bits 1,3,5,7 (P₁ 111) must have even parity, so P₁ =1

Bits 2, 3, 6, 7(P₂ 101) must have even parity, so P₂ =0

Bits 4,5,6,7 (P₄ 101) must have even parity, so P₄ =0

The final code is 1010101

EX: Code word is 1001001

Bits 1,3,5,7 (C₁ 1001) →no error →put a 0 in the 1's position→C₁=0

Bits 2, 3, 6, 7(C₂ 0001)) → error →put a 1 in the 2's position→C₂=1

Bits 4,5,6,7 (C₄ 1001)) →no error →put a 0 in the 4's position→C₃=0

15-bit Hamming Code: It transmit 11 data bits, 4 parity bits located $2^0 2^1 2^2 2^3$

Word format is

P ₁	P ₂	D ₃	P ₄	D ₅	D ₆	D ₇	P ₈	D ₉	D ₁₀	D ₁₁	D ₁₂	D ₁₃	D ₁₄	D ₁₅
----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	-----------------	-----------------	-----------------	-----------------	-----------------	-----------------

12-Bit Hamming Code:It transmit 8 data bits, 4 parity bits located at position $2^0 2^1 2^2 2^3$

Word format is

P ₁	P ₂	D ₃	P ₄	D ₅	D ₆	D ₇	P ₈	D ₉	D ₁₀	D ₁₁	D ₁₂
----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	-----------------	-----------------	-----------------

Alphanumeric Codes:

These codes are used to encode the characteristics of alphabet in addition to the decimal digits. It is used for transmitting data between computers & its I/O device such as printers, keyboards & video display terminals. Popular modern alphanumeric codes are ASCII code & EBCDIC code.

Boolean algebra

In 1854, George Boole developed an algebraic system now called Boolean algebra. In 1938, Claude E. Shannon introduced a two-valued Boolean algebra called switching algebra that represented the properties of bistable electrical switching circuits. For the formal definition of Boolean algebra, we shall employ the postulates formulated by E. V. Huntington in 1904.

Boolean algebra is a system of mathematical logic. It is an algebraic system consisting of the set of elements (0, 1), two binary operators called OR, AND, and one unary operator NOT. It is the basic mathematical tool in the analysis and synthesis of switching circuits. It is a way to express logic functions algebraically.

Axioms and laws of Boolean algebra

Axioms or Postulates of Boolean algebra are a set of logical expressions that we accept without proof and upon which we can build a set of useful theorems.

	AND Operation	OR Operation	NOT Operation
Axiom1 :	$0.0=0$	$0+0=0$	$\bar{0}=1$
Axiom2:	$0.1=0$	$0+1=1$	$\bar{1}=0$
Axiom3:	$1.0=0$	$1+0=1$	
Axiom4:	$1.1=1$	$1+1=1$	

Complementation law

Law1: $\bar{0}=1$	Law3: if $A=0$, then $\bar{A}=1$
Law2: $\bar{1}=0$	Law4: if $A=1$, then $\bar{A}=0$
	Law5: if $\bar{\bar{A}}=A$ (double inversion law)

AND Law

- Law1: $A.0=0$ (Null law)
- Law2: $A.1=A$ (Identity law)
- Law3: $A.A=A$ (Idempotence law)
- Law4: $A. \bar{A}=0$

OR Law

- Law1: $A+0=A$
- Law2: $A+1=1$
- Law3: $A+A=A$ (Idempotence law)
- Law4: $A+ \bar{A}=1$

Basic Theorems and Properties of Boolean algebra

Commutative law

$$\text{Law1: } A+B=B+A$$

$$\text{Law2: } A.B=B.A$$

Associative law

$$\text{Law1: } A + (B + C) = (A + B) + C$$

$$\text{Law2: } A(B.C) = (A.B)C$$

Distributive law

$$\text{Law1: } A.(B + C) = AB + AC$$

$$\text{Law2: } A + BC = (A + B).(A + C)$$

Absorption law

$$\text{Law1: } A + AB = A$$

$$\text{Law2: } A(A + B) = A$$

$$\begin{array}{l} \text{Solution: } A(1+B) \\ A \end{array}$$

$$\begin{array}{l} \text{Solution: } A.A + A.B \\ A + A.B \\ A(1+B) \\ A \end{array}$$

DeMorgan Theorems

$$\text{Theorem1: } \overline{(A + B)} = \bar{A} . \bar{B}$$

$$\text{Theorem2: } \overline{(A . B)} = \bar{A} + \bar{B}$$

Redundant Literal Rule

$$\text{Rule1: } A + \bar{A}.B = A + B$$

$$\text{Rule2: } A.(\bar{A} + B) = AB$$

$$\begin{array}{l} \text{Solution: } A + \bar{A}.B \\ (A + \bar{A}).(A + B) \quad \therefore A + BC = (A + B).(A + C) \\ A + B \quad \therefore A + \bar{A} = 1 \end{array}$$

$$\begin{array}{l} \text{Solution: } A.(\bar{A} + B) \\ A.\bar{A} + A.B \\ AB \end{array}$$

Consensus Theorem

$$\text{Theorem1. } AB + A'C + BC = AB + A'C \quad \text{Theorem2. } (A+B).(A'+C).(B+C) = (A+B).(A'+C)$$

The BC term is called the consensus term and is redundant. The consensus term is formed from a PAIR OF TERMS in which a variable (A) and its complement (A') are present; the consensus term is formed by multiplying the two terms and leaving out the selected variable and its complement

Consensus Theorem1 Proof:

$$\begin{aligned}
 AB+A'C+BC &= AB+A'C+(A+A')BC \\
 &= AB+A'C+ABC+A'BC \\
 &= AB(1+C)+A'C(1+B) \\
 &= AB+ A'C
 \end{aligned}$$

Principle of Duality

Each postulate consists of two expressions statement one expression is transformed into the other by interchanging the operations (+) and (·) as well as the identity elements 0 and 1.

Such expressions are known as duals of each other.

If some equivalence is proved, then its dual is also immediately true.

E.g. If we prove: $(x.x)+(x'+x')=1$, then we have by duality: $(x+x).(x'.x')=0$

The Huntington postulates were listed in pairs and designated by part (a) and part (b) in below table.

Table for Postulates and Theorems of Boolean algebra

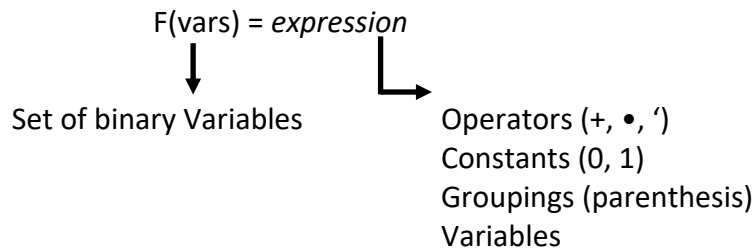
Part-A	Part-B
$A+0=A$	$A.0=0$
$A+1=1$	$A.1=A$
$A+A=A$ (Idempotence law)	$A.A=A$ (Idempotence law)
$A+\bar{A}=1$	$A.\bar{A}=0$
$\overline{\bar{A}}=A$ (double inversion law)	--
Commutative law: $A+B=B+A$	$A.B=B.A$
Associative law: $A+(B+C)=(A+B)+C$	$A(B.C)=(A.B)C$
Distributive law: $A.(B+C)=AB+AC$	$A+BC=(A+B).(A+C)$
Absorption law: $A+AB=A$	$A(A+B)=A$
DeMorgan Theorem: $\overline{(A+B)}=\bar{A}.\bar{B}$	$\overline{(A.B)}=\bar{A}+\bar{B}$
Redundant Literal Rule: $A+\bar{A}.B=A+B$	$A.(\bar{A}+B)=AB$
Consensus Theorem: $AB+A'C+BC=AB+A'C$	$(A+B).(A'+C).(B+C)=(A+B).(A'+C)$

Boolean Function

Boolean algebra is an algebra that deals with binary variables and logic operations.

A Boolean function described by an algebraic expression consists of binary variables, the constants 0 and 1, and the logic operation symbols.

For a given value of the binary variables, the function can be equal to either 1 or 0.



Consider an example for the Boolean function

$$F_1 = x + y'z$$

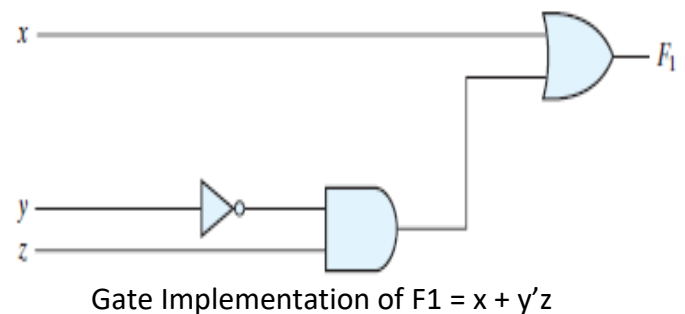
The function F_1 is equal to 1 if x is equal to 1 or if both y' and z are equal to 1. F_1 is equal to 0 otherwise. The complement operation dictates that when $y' = 1$, $y = 0$. Therefore, $F_1 = 1$ if $x = 1$ or if $y = 0$ and $z = 1$.

A Boolean function expresses the logical relationship between binary variables and is evaluated by determining the binary value of the expression for all possible values of the variables.

A Boolean function can be represented in a truth table. The number of rows in the truth table is 2^n , where n is the number of variables in the function. The binary combinations for the truth table are obtained from the binary numbers by counting from 0 through $2^n - 1$.

Truth Table for F_1

x	y	z	F_1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1



Note:

Q: Let a function $F()$ depend on n variables. How many rows are there in the truth table of $F()$?

A: 2^n rows, since there are 2^n possible binary patterns/combinations for the n variables.

Truth Tables

- Enumerates all possible combinations of variable values and the corresponding function value
- Truth tables for some arbitrary functions
F1(x,y,z), F2(x,y,z), and F3(x,y,z) are shown to the below.

x	y	z	F ₁	F ₂	F ₃
0	0	0	0	1	1
0	0	1	0	0	1
0	1	0	0	0	1
0	1	1	0	1	1
1	0	0	0	1	0
1	0	1	0	1	0
1	1	0	0	0	0
1	1	1	1	0	1

- Truth table: a unique representation of a Boolean function
- If two functions have identical truth tables, the functions are equivalent (and vice-versa).
- Truth tables can be used to prove equality theorems.
- However, the size of a truth table grows exponentially with the number of variables involved, hence unwieldy. This motivates the use of Boolean Algebra.

Boolean expressions-NOT unique

Unlike truth tables, expressions representing a Boolean function are NOT unique.

- Example:
 - $F(x,y,z) = x' \cdot y' \cdot z' + x' \cdot y \cdot z' + x \cdot y \cdot z'$
 - $G(x,y,z) = x' \cdot y' \cdot z' + y \cdot z'$
- The corresponding truth tables for F() and G() are to the right. They are identical.
- Thus, $F() = G()$

x	y	z	F	G
0	0	0	1	1
0	0	1	0	0
0	1	0	1	1
0	1	1	0	0
1	0	0	0	0
1	0	1	0	0
1	1	0	1	1
1	1	1	0	0

Algebraic Manipulation (Minimization of Boolean function)

- Boolean algebra is a useful tool for simplifying digital circuits.
- Why do it? Simpler can mean cheaper, smaller, faster.
- Example: Simplify $F = x'yz + x'yz' + xz$.

$$\begin{aligned} F &= x'yz + x'yz' + xz \\ &= x'y(z+z') + xz \\ &= x'y \bullet 1 + xz \\ &= x'y + xz \end{aligned}$$

- Example: Prove

$$x'y'z' + x'yz' + xyz' = x'z' + yz'$$

- **Proof:**

$$\begin{aligned} x'y'z' + x'yz' + xyz' \\ &= x'y'z' + x'yz' + x'yz' + xyz' \\ &= x'z'(y'+y) + yz'(x'+x) \\ &= x'z' \bullet 1 + yz' \bullet 1 \\ &= x'z' + yz' \end{aligned}$$

Complement of a Function

- The complement of a function is derived by interchanging ($\bullet$ and $+$), and (1 and 0), and complementing each variable.
- Otherwise, interchange 1s to 0s in the truth table column showing F.
- The *complement* of a function IS NOT THE SAME as the *dual* of a function.

Example

- Find $G(x,y,z)$, the complement of $F(x,y,z) = xy'z' + x'yz$

$$\begin{aligned} \text{Ans: } G = F' &= (xy'z' + x'yz)' \\ &= (xy'z')' \bullet (x'yz)' && \text{DeMorgan} \\ &= (x'+y+z) \bullet (x+y'+z') && \text{DeMorgan again} \end{aligned}$$

Note: The complement of a function can also be derived by finding the function's *dual*, and then complementing all of the literals

Canonical and Standard Forms

We need to consider formal techniques for the simplification of Boolean functions.

Identical functions will have exactly the same canonical form.

- Minterms and Maxterms
- Sum-of-Minterms and Product-of- Maxterms
- Product and Sum terms
- Sum-of-Products (SOP) and Product-of-Sums (POS)

Definitions

Literal: A variable or its complement

Product term: literals connected by •

Sum term: literals connected by +

Minterm: a product term in which all the variables appear exactly once, either complemented or uncomplemented.

Maxterm: a sum term in which all the variables appear exactly once, either complemented or uncomplemented.

Canonical form: Boolean functions expressed as a sum of Minterms or product of Maxterms are said to be in canonical form.

Minterm

- Represents exactly one combination in the truth table.
- Denoted by m_j , where j is the decimal equivalent of the minterm's corresponding binary combination (b_j).
- A variable in m_j is complemented if its value in b_j is 0, otherwise is uncomplemented.

Example: Assume 3 variables (A, B, C), and $j=3$. Then, $b_j = 011$ and its corresponding minterm is denoted by $m_j = A'BC$

Maxterm

- Represents exactly one combination in the truth table.
- Denoted by M_j , where j is the decimal equivalent of the maxterm's corresponding binary combination (b_j).
- A variable in M_j is complemented if its value in b_j is 1, otherwise is uncomplemented.

Example: Assume 3 variables (A, B, C), and $j=3$. Then, $b_j = 011$ and its corresponding maxterm is denoted by $M_j = A+B'+C'$

Truth Table notation for Minterms and Maxterms

- Minterms and Maxterms are easy to denote using a truth table.

Example: Assume 3 variables x,y,z (order is fixed)

x	y	z	Minterm	Maxterm
0	0	0	$x'y'z' = m_0$	$x+y+z = M_0$
0	0	1	$x'y'z = m_1$	$x+y+z' = M_1$
0	1	0	$x'yz' = m_2$	$x+y'+z = M_2$
0	1	1	$x'yz = m_3$	$x+y'+z' = M_3$
1	0	0	$xy'z' = m_4$	$x'+y+z = M_4$
1	0	1	$xy'z = m_5$	$x'+y+z' = M_5$
1	1	0	$xyz' = m_6$	$x'+y'+z = M_6$
1	1	1	$xyz = m_7$	$x'+y'+z' = M_7$

Canonical Forms

- Every function $F()$ has two canonical forms:
 - Canonical Sum-Of-Products (sum of minterms)
 - Canonical Product-Of-Sums (product of maxterms)

Canonical Sum-Of-Products:

The minterms included are those m_j such that $F() = 1$ in row j of the truth table for $F()$.

Canonical Product-Of-Sums:

The maxterms included are those M_j such that $F() = 0$ in row j of the truth table for $F()$.

Example

Consider a Truth table for $f_1(a,b,c)$ at right

The canonical sum-of-products form for f_1 is

$$f_1(a,b,c) = m_1 + m_2 + m_4 + m_6$$

$$= a'b'c + a'bc' + ab'c' + abc'$$

The canonical product-of-sums form for f_1 is

$$f_1(a,b,c) = M_0 \cdot M_3 \cdot M_5 \cdot M_7$$

$$= (a+b+c) \cdot (a+b'+c') \cdot (a'+b+c') \cdot (a'+b'+c').$$

- Observe that: $m_j = M_j'$

a	b	c	f_1
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Shorthand: Σ and $\prod$

- $f_1(a,b,c) = \Sigma m(1,2,4,6)$, where Σ indicates that this is a sum-of-products form, and $m(1,2,4,6)$ indicates that the minterms to be included are m_1 , m_2 , m_4 , and m_6 .
- $f_1(a,b,c) = \prod M(0,3,5,7)$, where $\prod$ indicates that this is a product-of-sums form, and $M(0,3,5,7)$ indicates that the maxterms to be included are M_0 , M_3 , M_5 , and M_7 .
- Since $m_j = M_j'$ for any j ,
 $\Sigma m(1,2,4,6) = \prod M(0,3,5,7) = f_1(a,b,c)$
-

Conversion between Canonical Forms

- Replace Σ with $\prod$ (or *vice versa*) and replace those j 's that appeared in the original form with those that do not.
- Example:

$$\begin{aligned} f_1(a,b,c) &= a'b'c + a'bc' + ab'c' + abc' \\ &= m_1 + m_2 + m_4 + m_6 \\ &= \Sigma(1,2,4,6) \\ &= \prod(0,3,5,7) \\ &= (a+b+c) \cdot (a+b'+c') \cdot (a'+b+c') \cdot (a'+b'+c') \end{aligned}$$

Standard Forms

Another way to express Boolean functions is in standard form. In this configuration, the terms that form the function may contain one, two, or any number of literals.

There are two types of standard forms: the sum of products and products of sums.

The sum of products is a Boolean expression containing AND terms, called product terms, with one or more literals each. The sum denotes the ORing of these terms. An example of a function expressed as a sum of products is

$$F1 = y' + xy + x'yz'$$

The expression has three product terms, with one, two, and three literals. Their sum is, in effect, an OR operation.

A product of sums is a Boolean expression containing OR terms, called sum terms. Each term may have any number of literals. The product denotes the ANDing of these terms. An example of a function expressed as a product of sums is

$$F2 = x(y' + z)(x' + y + z')$$

This expression has three sum terms, with one, two, and three literals. The product is an AND operation.

Conversion of SOP from standard to canonical form

Example-1.

Express the Boolean function $F = A + B'C$ as a sum of minterms.

Solution: The function has three variables: A, B, and C. The first term A is missing two variables; therefore,

$$A = A(B + B') = AB + AB'$$

This function is still missing one variable, so

$$\begin{aligned} A &= AB(C + C') + AB'(C + C') \\ &= ABC + ABC' + AB'C + AB'C' \end{aligned}$$

The second term $B'C$ is missing one variable; hence,

$$B'C = B'C(A + A') = AB'C + A'B'C$$

Combining all terms, we have

$$\begin{aligned} F &= A + B'C \\ &= ABC + ABC' + AB'C + AB'C' + A'B'C \end{aligned}$$

But $AB'C$ appears twice, and according to theorem $(x + x = x)$, it is possible to remove one of those occurrences. Rearranging the minterms in ascending order, we finally obtain

$$\begin{aligned} F &= A'B'C + AB'C + AB'C' + ABC' + ABC \\ &= m_1 + m_4 + m_5 + m_6 + m_7 \end{aligned}$$

When a Boolean function is in its sum-of-minterms form, it is sometimes convenient to express the function in the following brief notation:

$$F(A, B, C) = \sum m(1, 4, 5, 6, 7)$$

Example-2.

Express the Boolean function $F = xy + x'z$ as a product of maxterms.

Solution: First, convert the function into OR terms by using the distributive law:

$$\begin{aligned} F &= xy + x'z = (xy + x')(xy + z) \\ &= (x + x')(y + x')(x + z)(y + z) \\ &= (x' + y)(x + z)(y + z) \end{aligned}$$

The function has three variables: x, y, and z. Each OR term is missing one variable; therefore,

$$\begin{aligned} x' + y &= x' + y + zz' = (x' + y + z)(x' + y + z') \\ x + z &= x + z + yy' = (x + y + z)(x + y' + z) \\ y + z &= y + z + xx' = (x + y + z)(x' + y + z) \end{aligned}$$

Combining all the terms and removing those which appear more than once, we finally obtain

$$\begin{aligned} F &= (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z) \\ F &= M_0 M_2 M_4 M_5 \end{aligned}$$

A convenient way to express this function is as follows:

$$F(x, y, z) = \pi M(0, 2, 4, 5)$$

The product symbol, π , denotes the ANDing of maxterms; the numbers are the indices of the maxterms of the function.

Digital Logic Gates

Boolean functions are expressed in terms of AND, OR, and NOT operations, it is easier to implement a Boolean function with these type of gates.

Name	Graphic symbol	Algebraic function	Truth table															
AND		$F = x \cdot y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = x + y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	1
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
Inverter		$F = x'$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	x	F	0	1	1	0									
x	F																	
0	1																	
1	0																	
Buffer		$F = x$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	x	F	0	0	1	1									
x	F																	
0	0																	
1	1																	
NAND		$F = (xy)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = (x + y)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	0
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
Exclusive-OR (XOR)		$F = xy' + x'y$ $= x \oplus y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Exclusive-NOR or equivalence		$F = xy + x'y'$ $= (x \oplus y)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

Properties of XOR Gates

- XOR (also $\oplus$) : the “not-equal” function
- $\text{XOR}(X,Y) = X \oplus Y = X'Y + XY'$
- Identities:
 - $X \oplus 0 = X$
 - $X \oplus 1 = X'$
 - $X \oplus X = 0$
 - $X \oplus X' = 1$
- Properties:
 - $X \oplus Y = Y \oplus X$
 - $(X \oplus Y) \oplus W = X \oplus (Y \oplus W)$

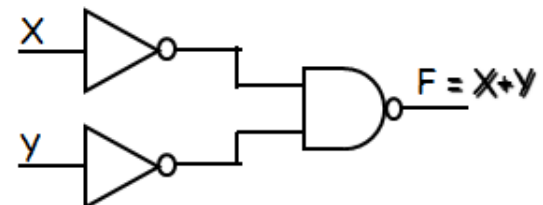
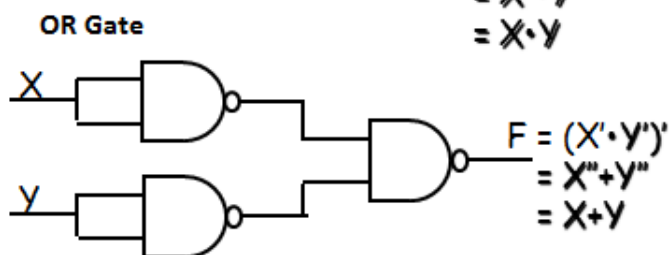
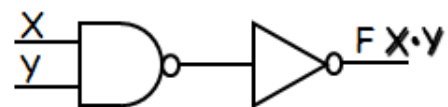
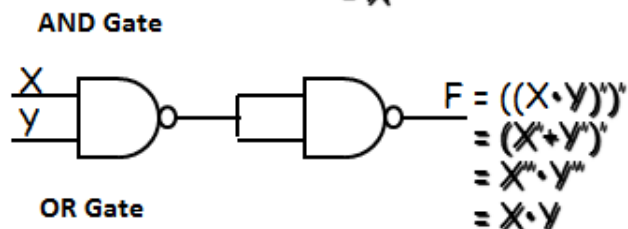
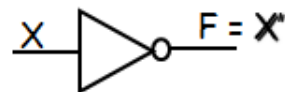
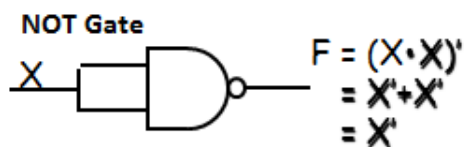
Universal Logic Gates

NAND and NOR gates are called Universal gates. All fundamental gates (NOT, AND, OR) can be realized by using either only NAND or only NOR gate. A universal gate provides flexibility and offers enormous advantage to logic designers.

NAND as a Universal Gate

NAND Known as a “universal” gate because ANY digital circuit can be implemented with NAND gates alone.

To prove the above, it suffices to show that AND, OR, and NOT can be implemented using NAND gates only.



Unit-II

Minimization and design of Combinational circuits

Two-variable k-map:

A two-variable k-map can have $2^2=4$ possible combinations of the input variables A and B. Each of these combinations, $A B$, $A B$, AB , AB (in the SOP form) is called a minterm. The minterm may be represented in terms of their decimal designations – m_0 for $A B$, m_1 for $A B$, m_2 for AB and m_3 for AB , assuming that A represents the MSB. The letter m stands for minterm and the subscript represents the decimal designation of the minterm. The presence or absence of a minterm in the expression indicates that the output of the logic circuit assumes logic 1 or logic 0 level for that combination of input variables.

The expression $f = A B + A B + AB + AB$, it can be expressed using min

$$\text{term as } F = m_0 + m_2 + m_3 = \sum m(0, 2, 3)$$

Using Truth Table:

Minterm	Inputs		Output
	A	B	F
0	0	0	1
1	0	1	0
2	1	0	1
3	1	1	1

A 1 in the output contains that particular minterm in its sum and a 0 in that column indicates that the particular minterm does not appear in the expression for output. This information can also be indicated by a two-variable k-map.

Mapping of SOP Expressions:

A two-variable k-map has $2^2=4$ squares. These squares are called cells. Each square on the k-map represents a unique minterm. The minterm designation of the squares are placed in any square, indicates that the corresponding minterm does output expressions. And a 0 or no entry in any square indicates that the corresponding minterm does not appear in the expression for output.

A \ B	0	1
0	$A'B'$	$A'B$
1	AB'	AB

The minterms of a two-variable k-map

The mapping of the expressions $=\sum m(0,2,3)$ is

		B		0	1	
A	0		⁰	1	¹	0
	1		²	1	³	1

k-map of $\sum m(0,2,3)$

EX: Map the expressions $f = A B + A B$

$F = m_1 + m_2 = \sum m(1,2)$ The k-map is

		B		0	1	
A	0		⁰	0	¹	1
	1		²	1	³	0

Minimizations of SOP expressions:

To minimize Boolean expressions given in the SOP form by using the k-map, look for adjacent adjacent squares having 1's minterms adjacent to each other, and combine them to form larger squares to eliminate some variables. Two squares are said to be adjacent to each other, if their minterms differ in only one variable. (i.e, $A B$ & $A B$ differ only in one variable. so they may be combined to form a 2-square to eliminate the variable B. similarly all other.

The necessary condition for adjacency of minterms is that their decimal designations must differ by a power of 2. A minterm can be combined with any number of minterms adjacent to it to form larger squares. Two minterms which are adjacent to each other can be combined to form a bigger square called a 2-square or a pair. This eliminates one variable – the variable that is not common to both the minterms. For EX:

m_0 and m_1 can be combined to yield,

$$f_1 = m_0 + m_1 = AB + A B = A (B + B$$

$) = A$ m_0 and m_2 can be combined to yield,

$$f_2 = m_0 + m_2 = A B + AB = B (A + A) = B$$

m_1 and m_3 can be combined to yield,

$$f_3 = m_1 + m_3 = A'B + AB = B(A' + A) = B$$

m_2 and m_3 can be combined to yield,

$$f_4 = m_2 + m_3 = AB + AB = A(B + B) = A$$

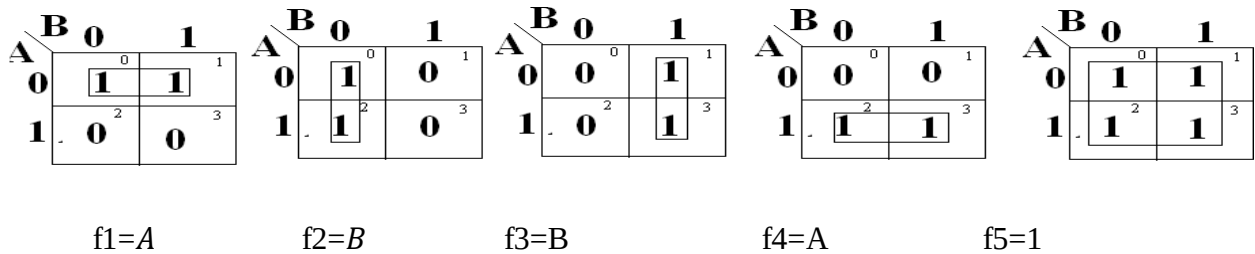
m_0, m_1, m_2 and m_3 can be combined to yield,

$$= A'B + AB + AB + AB$$

$$= A'(B + B) + A(B + B)$$

$$= A' + A$$

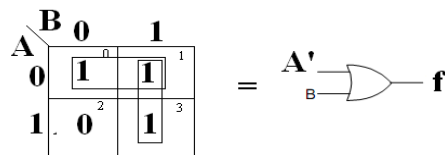
$$= 1$$



The possible minterm groupings in a two-variable k-map.

Two 2-squares adjacent to each other can be combined to form a 4-square. A 4-square eliminates 2 variables. A 4-square is called a quad. To read the squares on the map after minimization, consider only those variables which remain constant through the square, and ignore the variables which are varying. Write the non complemented variable if the variable is remaining constant as a 1, and the complemented variable if the variable is remaining constant as a 0, and write the variables as a product term. In the above figure f_1 read as A , because, along the square, A remains constant as a 0, that is, as A' , where as B is changing from 0 to 1.

EX: Reduce the minterm $f = A'B + AB + AB$ using mapping Expressed in terms of minterms, the given expression is $F = m_0 + m_1 + m_2 + m_3 = m \sum(0, 1, 2, 3)$ & the figure shows the k-map for f and its reduction. In one 2-square, A is constant as a 0 but B varies from a 0 to a 1, and in the other 2-square, B is constant as a 1 but A varies from a 0 to a 1. So, the reduced expressions is $A' + B$.



It requires two gate inputs for realization as

$$f = A' + B \quad (\text{k-map in SOP form, and logic diagram.})$$

The main criterion in the design of a digital circuit is that its cost should be as low as possible. For that the expression used to realize that circuit must be minimal. Since the cost is proportional to number of gate inputs in the circuit, an expression is considered minimal only if it corresponds to the least possible number of gate inputs. & there is no guarantee for that k-map in SOP is the real minimal. To obtain real minimal expression, obtain the minimal expression both in SOP & POS form by using k-maps and take the minimal of these two minimals.

The 1's on the k-map indicate the presence of minterms in the output expressions, where as the 0s indicate the absence of minterms. Since the absence of a minterm in the SOP expression means the presence of the corresponding maxterm in the POS expression of the same. when a SOP expression is plotted on the k-map, 0s or no entries on the k-map represent the maxterms. To obtain the minimal expression in the POS form, consider the 0s on the k-map and follow the procedure used for combining 1s. Also, since the absence of a maxterm in the POS expression means the presence of the corresponding minterm in the SOP expression of the same, when a POS expression is plotted on the k-map, 1s or no entries on the k-map represent the minterms.

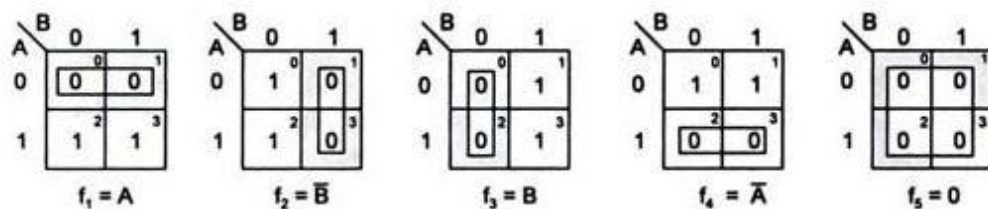
Mapping of POS expressions:

Each sum term in the standard POS expression is called a maxterm. A function in two variables (A, B) has four possible maxterms, $A+B$, $A+B$, $A+B$, $A+B$

. They are represented as M_0 , M_1 , M_2 , and M_3 respectively. The uppercase letter M stands for maxterm and its subscript denotes the decimal designation of that maxterm obtained by treating the non-complemented variable as a 0 and the complemented variable as a 1 and putting them side by side for reading the decimal equivalent of the binary number so formed.

For mapping a POS expression on to the k-map, 0s are placed in the squares corresponding to the maxterms which are presented in the expression and 1s are placed in the squares corresponding to the maxterm which are not present in the expression. The decimal designation of the squares of the squares for maxterms is the same as that for the minterms. A two-variable k-map & the associated maxterms are as the maxterms of a two-variable k-map

The possible maxterm groupings in a two-variable k-map



Minimization of POS Expressions:

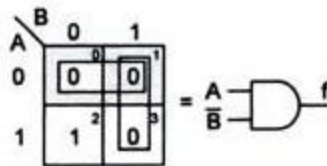
To obtain the minimal expression in POS form, map the given POS expression on to the K-map and combine the adjacent 0s into as large squares as possible. Read the squares putting the complemented variable if its value remains constant as a 1 and the non-complemented variable if its value remains constant as a 0 along the entire square (ignoring the variables which do not remain constant throughout the square) and then write them as a sum term.

Various maxterm combinations and the corresponding reduced expressions are shown in figure. In this f_1 read as A because A remains constant as a 0 throughout the square and B changes from a 0 to a 1. f_2 is read as B' because B remains constant along the square as a 1 and A changes from a 0 to a 1. f_3

Is read as a 0 because both the variables are changing along the square.

Ex: Reduce the expression $f = (A+B)(A+B')(A'+B')$ using mapping.

The given expression in terms of maxterms is $f = \pi M(0,1,3)$. It requires two gates inputs for realization of the reduced expression as



$$F = AB'$$

K-map in POS form and logic diagram

In this given expression ,the maxterm M_2 is absent. This is indicated by a 1 on the k-map. The corresponding SOP expression is $\sum m_2$ or AB' . This realization is the same as that for the POS form.

Three-variable K-map:

A function in three variables (A, B, C) expressed in the standard SOP form can have eight possible combinations: $A B C$, $A B C'$, $A B C$, $A B C'$, $A B C$, $A B C'$, $A B C$, and $A B C$. Each one of these combinations designate d by $m_0, m_1, m_2, m_3, m_4, m_5, m_6$, and m_7 , respectively, is called a minterm. A is the MSB of the minterm designator and C is the LSB.

In the standard POS form, the eight possible combinations are: $A+B+C$, $A+B+C$, $A+B+C$, $A+B+C$, $A+B+C$, $A+B+C$, $A+B+C$, and $A+B+C$. Each one of these combinations designated by $M_0, M_1, M_2, M_3, M_4, M_5, M_6$, and M_7 respectively is called a maxterm. A is the MSB of the maxterm designator and C is the LSB.

A three-variable k-map has, therefore, $8(=2^3)$ squares or cells, and each square on the map represents a minterm or maxterm as shown in figure. The small number on the top right corner of each cell indicates the minterm or maxterm designation.

		BC			
		00	01	11	10
A	0	$\bar{A}\bar{B}\bar{C}^0$	$\bar{A}\bar{B}C^1$	$\bar{A}BC^3$	$\bar{A}B\bar{C}^2$
	1	$A\bar{B}\bar{C}^4$	$A\bar{B}C^5$	ABC^7	$AB\bar{C}^6$

(a) Minterms

		BC			
		00	01	11	10
A	0	$A+B+C^0$	$A+B+\bar{C}^1$	$A+\bar{B}+\bar{C}^3$	$A+\bar{B}+C^2$
	1	$\bar{A}+B+C^4$	$\bar{A}+B+\bar{C}^5$	$\bar{A}+\bar{B}+\bar{C}^7$	$\bar{A}+\bar{B}+C^6$

(b) Maxterms

The three-variable k-map.

The binary numbers along the top of the map indicate the condition of B and C for each column. The binary number along the left side of the map against each row indicates the condition of A for that row. For example, the binary number 01 on top of the second column in fig indicates that the variable B appears in complemented form and the variable C in non-complemented form in all the minterms in that column. The binary number 0 on the left of the first row indicates that the variable A appears in complemented form in all the minterms in that row, the binary numbers along the top of the k-map are not in normal binary order. They are, infact, in the Gray code. This is to ensure that twophysically adjacent squares are really adjacent, i.e., their minterms or maxterms differ by only one variable.

Ex: Map the expression $f = A\bar{B}C + A\bar{B}\bar{C} + ABC + ABC + ABC$

In the given expression , the minterms are : $A\bar{B}C=001=m_1$; $ABC=101=m_5$; $A\bar{B}\bar{C}=010=m_2$;

$ABC=110=m_6$; $ABC=111=m_7$.

So the expression is $f = \sum m(1,5,2,6,7) = \sum m(1,2,5,6,7)$. The corresponding k-map is

		BC			
		00	01	11	10
A	0	0 ⁰	1 ¹	0 ³	1 ²
	1	0 ⁴	1 ⁵	1 ⁷	1 ⁶

K-map in SOP form

Ex: Map the expression $f = (A+B+C)(A + B + C)(A + B + C)(A + B + C)(A + B + C)$

In the given expression the maxterms are : $A+B+C=000=M_0$; $A + B + C = 101=M_5$; $A + B + C = 111=M_7$; $A + B + C = 011=M_3$; $A + B + C = 110=M_6$.

So the expression is $f = \pi M(0,5,7,3,6) = \pi M(0,3,5,6,7)$. The mapping of the expression is

BC A	00	01	11	10
	0	1	0	1
0	0 ⁰	1 ¹	0 ³	1 ²
1	1 ⁴	0 ⁵	0 ⁷	0 ⁶

K-map in POS form.

Minimization of SOP and POS expressions:

For reducing the Boolean expressions in SOP (POS) form plotted on the k-map, look at the 1s (0s) present on the map. These represent the minterms (maxterms). Look for the minterms (maxterms) adjacent to each other, in order to combine them into larger squares. Combining of adjacent squares in a k-map containing 1s (or 0s) for the purpose of simplification of a SOP (or POS) expression is called *looping*. Some of the minterms (maxterms) may have many adjacencies. Always start with the minterms (maxterm) with the least number of adjacencies and try to form as large as large a square as possible. The larger must form a geometric square or rectangle. They can be formed even by wrapping around, but cannot be formed by using diagonal configurations. Next consider the minterm (maxterm) with next to the least number of adjacencies and form as large a square as possible. Continue this till all the minterms (maxterms) are taken care of . A minterm (maxterm) can be part of any number of squares if it is helpful in reduction. Read the minimal expression from the k-map, corresponding to the squares formed. There can be more than one minimal expression.

Two squares are said to be adjacent to each other (since the binary designations along the top of the map and those along the left side of the map are in Gray code), if they are physically adjacent to each other, or can be made adjacent to each other by wrapping around. For squares to be combinable into bigger squares it is essential but not sufficient that their minterm designations must differ by a power of two.

General procedure to simplify the Boolean expressions:

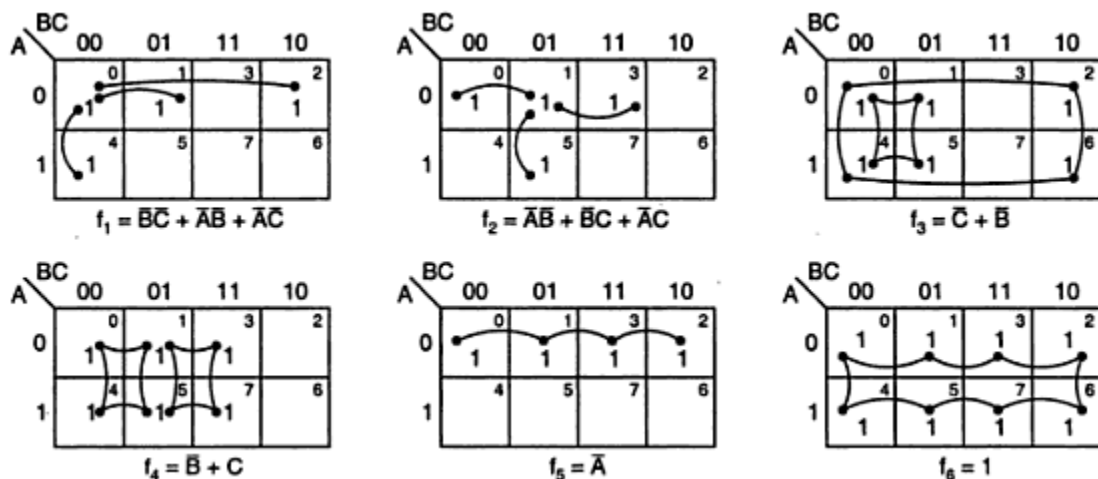
1. Plot the k-map and place 1s(0s) corresponding to the minterms (maxterms) of the SOP (POS) expression.
2. Check the k-map for 1s(0s) which are not adjacent to any other 1(0). They are isolated minterms(maxterms) . They are to be read as they are because they cannot be combined even into a 2-square.
3. Check for those 1s(0s) which are adjacent to only one other 1(0) and make them pairs (2 squares).
4. Check for quads (4 squares) and octets (8 squares) of adjacent 1s (0s) even if they contain some 1s(0s) which have already been combined. They must geometrically form a square or a rectangle.
5. Check for any 1s(0s) that have not been combined yet and combine them into bigger squares if possible.
6. Form the minimal expression by summing (multiplying) the product the product (sum) terms of all the groups.

Reading the K-maps:

While reading the reduced k-map in SOP (POS) form, the variable which remains constant as 0 along the square is written as the complemented (non-complemented) variable and the one which remains constant as 1 along the square is written as non-complemented (complemented) variable and the term as a product (sum) term. All the product (sum) terms are added (multiplied).

Some possible combinations of minterms and the corresponding minimal expressions read from the k-maps are shown in fig: Here f_6 is read as 1, because along the 8-square no variable remains constant. F_5 is read as $\bar{A}$, because, along the 4-square formed by m_0, m_1, m_2 and m_3 , the variables B and C are changing, and A remains constant as a 0. Algebraically,

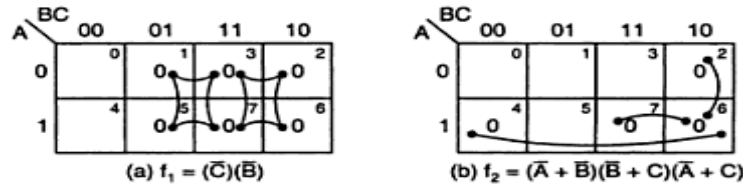
$$\begin{aligned} f_5 &= m_0 + m_1 + m_2 + m_3 \\ &= \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + \bar{A}BC + \bar{A}B\bar{C} \\ &= \bar{A}B(\bar{C} + C) + \bar{A}B(\bar{C} + C) \\ &= \bar{A}B + \bar{A}B \\ &= \bar{A}(B + B) = \bar{A} \end{aligned}$$



f_3 is read as $C + B$, because in the 4-square formed by m_0, m_2, m_4 , and m_6 , the variable A and B are changing, whereas the variable C remains constant as a 0. So it is read as $\bar{C}$. In the 4-square formed by m_0, m_1, m_4, m_5 , A and C are changing but B remains constant as a 0. So it is read as $\bar{B}$. So, the resultant expression for f_3 is the sum of these two, i.e., $\bar{C} + \bar{B}$.

f_1 is read as $\bar{B}\bar{C} + A\bar{B} + A\bar{C}$, because in the 2-square formed by m_0 and m_4 , A is changing from a 0 to a 1. Whereas B and C remain constant as a 0. So it is read as $\bar{B}\bar{C}$. In the 2-square formed by m_0 and m_1 , C is changing from a 0 to a 1, whereas A and B remain constant as a 0. So it is read as $\bar{A}\bar{B}$. In the 2-square formed by m_0 and m_2 , B is changing from a 0 to a 1 whereas A and C remain constant as a 0. So, it is read as $\bar{A}\bar{C}$. Therefore, the resultant SOP expression is $\bar{B}\bar{C} + A\bar{B} + A\bar{C}$.

Some possible maxterm groupings and the corresponding minimal POS expressions read from the k-map are



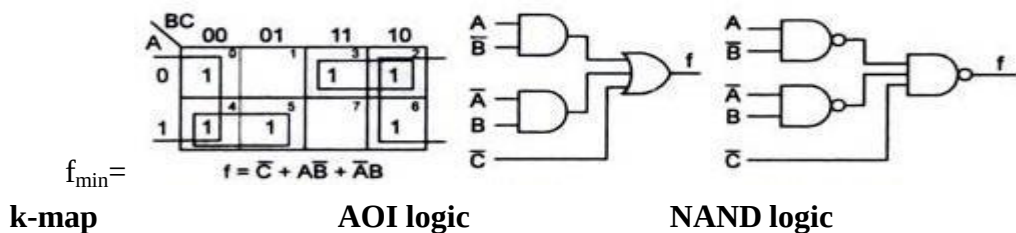
In this figure, along the 4-square formed by M_1, M_3, M_7, M_5 , A and B are changing from a 0 to a 1, where as C remains constant as a 1. SO it is read as C . Along the 4-squad formed by M_3, M_2, M_7 , and M_6 , variables A and C are changing from a 0 to a 1. But B remains constant as a 1. So it is read as B . The minimal expression is the product of these two terms, i.e., $f_1 = (C)(B)$.also in this figure, along the 2-square formed by M_4 and M_6 , variable B is changing from a 0 to a 1, while variable A remains constant as a 1 and variable C remains constant as a 0. SO, read it as

$A + C$. Similarly, the 2-square formed by M_7 and M_6 is read as $A + B$, while the 2-square formed by M_2 and M_6 is read as $B + C$. The minimal expression is the product of these sum terms, i.e, $f_2 = (A + C)(A + B)(B + C)$

Ex:Reduce the expression $f = \sum m(0,2,3,4,5,6)$ using mapping and implement it in AOI logic as well as in NAND logic.The Sop k-map and its reduction, and the implementation of the minimal expression using AOI logic and the corresponding NAND logic are shown in figures below

In SOP k-map, the reduction is done as:

- 1 m_5 has only one adjacency m_4 , so combine m_5 and m_4 into a square. Along this 2-square A remains constant as 1 and B remains constant as 0 but C varies from 0 to 1. So read it as AB .
- 2 m_3 has only one adjacency m_2 , so combine m_3 and m_2 into a square. Along this 2-square A remains constant as 0 and B remains constant as 1 but C varies from 1 to 0. So read it as $A\bar{B}$.
- 3 m_6 can form a 2-square with m_2 and m_4 can form a 2-square with m_0 , but observe that by wrapping the map from left to right m_0, m_4, m_2, m_6 can form a 4-square. Out of these m_2 and m_4 have already been combined but they can be utilized again. So make it. Along this 4-square, A is changing from 0 to 1 and B is also changing from 0 to 1 but C is remaining constant as 0. so read it as $\bar{C}$.
- 4 Write all the product terms in SOP form. So the minimal SOP expression is



Four variable k-maps:

Four variable k-map expressions can have $2^4=16$ possible combinations of input variables such as $A B C D, A B C \bar{D}, \dots, A B C D$ with minterm designations $m_0, m_1, \dots, m_{15}$ respectively in SOP form & $A+B+C+D, A+B+C+\bar{D}, \dots, A+B+\bar{C}+\bar{D}$ with maxterms $M_0, M_1, \dots, M_{15}$ respectively in POS form. It has $2^4=16$ squares or cells. The binary number designations of rows & columns are in the gray code. Here follows 01 & 10 follows 11 called Adjacency ordering.

AB \ CD	CD			
	00	01	11	10
00	0 $\bar{A}\bar{B}\bar{C}\bar{D}$	1 $\bar{A}\bar{B}\bar{C}D$	3 $\bar{A}\bar{B}C\bar{D}$	2 $\bar{A}\bar{B}CD$
01	4 $\bar{A}B\bar{C}\bar{D}$	5 $\bar{A}B\bar{C}D$	7 $\bar{A}BC\bar{D}$	6 $\bar{A}BCD$
11	12 $A\bar{B}\bar{C}\bar{D}$	13 $A\bar{B}\bar{C}D$	15 $AB\bar{C}\bar{D}$	14 $AB\bar{C}D$
10	8 $A\bar{B}C\bar{D}$	9 $A\bar{B}CD$	11 $AB\bar{C}\bar{D}$	10 $AB\bar{C}D$

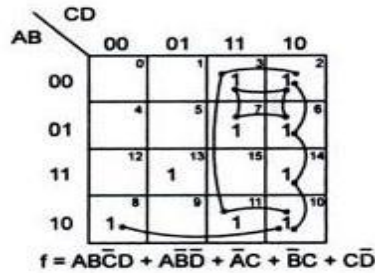
SOP form

AB \ CD	CD			
	00	01	11	10
00	0 $A+B+C+D$	1 $A+B+C+\bar{D}$	3 $A+B+\bar{C}+\bar{D}$	2 $A+B+\bar{C}+D$
01	4 $A+\bar{B}+C+D$	5 $A+\bar{B}+C+\bar{D}$	7 $A+\bar{B}+\bar{C}+\bar{D}$	6 $A+\bar{B}+\bar{C}+D$
11	12 $\bar{A}+\bar{B}+C+D$	13 $\bar{A}+\bar{B}+C+\bar{D}$	15 $\bar{A}+\bar{B}+\bar{C}+\bar{D}$	14 $\bar{A}+\bar{B}+\bar{C}+D$
10	8 $\bar{A}+B+C+D$	9 $\bar{A}+B+C+\bar{D}$	11 $\bar{A}+B+\bar{C}+\bar{D}$	10 $\bar{A}+B+\bar{C}+D$

POS form

EX: Reduce using mapping the expression $\Sigma m(2, 3, 6, 7, 8, 10, 11, 13, 14)$.

Start with the minterm with the least number of adjacencies. The minterm m_{13} has no adjacency. Keep it as it is. The m_8 has only one adjacency, m_{10} . Expand m_8 into a 2-square with m_{10} . The m_7 has two adjacencies, m_6 and m_3 . Hence m_7 can be expanded into a 4-square with m_6, m_3 and m_2 . Observe that, m_7, m_6, m_2 , and m_3 form a geometric square. The m_{11} has 2 adjacencies, m_{10} and m_3 . Observe that, m_{11}, m_{10}, m_3 , and m_2 form a geometric square on wrapping the K-map. So expand m_{11} into a 4-square with m_{10}, m_3 and m_2 . Note that, m_2 and m_3 have already become a part of the 4-square m_7, m_6, m_2 , and m_3 . But if m_{11} is expanded only into a 2-square with m_{10} , only one variable is eliminated. So m_2 and m_3 are used again to make another 4-square with m_{11} and m_{10} to eliminate two variables. Now only m_6 and m_{14} are left uncovered. They can form a 2-square that eliminates only one variable. Don't do that. See whether they can be expanded into a larger square. Observe that, m_2, m_6, m_{14} , and m_{10} form a rectangle. So m_6 and m_{14} can be expanded into a 4-square with m_2 and m_{10} . This eliminates two variables.



Five variable k-map:

Five variable k-map can have $2^5 = 32$ possible combinations of input variable as $ABCDE, ABCDE, \dots, ABCDE$ with minterms $m_0, m_1, \dots, m_{31}$ respectively in SOP & $A+B+C+D+E, A+B+C+D+E, \dots, A+B+C+D+E$ with maxterms $M_0, M_1, \dots, M_{31}$ respectively in POS form. It has $2^5 = 32$ squares or cells of the k-map are divided into 2 blocks of

16 squares each. The left block represents minterms from m_0 to m_{15} in which A is a 0, and the right block represents minterms from m_{16} to m_{31} in which A is 1. The 5-variable k-map may contain 2-squares, 4-squares, 8-squares, 16-squares or 32-squares involving these two blocks. Squares are also considered adjacent in these two blocks, if when superimposing one block on top of another, the squares coincide with one another.

Some possible 2-squares in a five-variable map are $m_0, m_{16}; m_2, m_{18}; m_5, m_{21}; m_{15}, m_{31}; m_{11}, m_{27}$.

Some possible 4-squares are $m_0, m_2, m_{16}, m_{18}; m_0, m_1, m_{16}, m_{17}; m_0, m_4, m_{16}, m_{20}; m_{13}, m_{15}, m_{29}, m_{31}; m_5, m_{13}, m_{21}, m_{29}$.

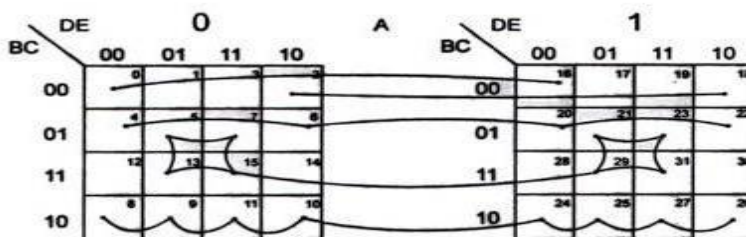
Some possible 8-squares are $m_0, m_1, m_3, m_2, m_{16}, m_{17}, m_{19}, m_{18}; m_0, m_4, m_{12}, m_8, m_{16}, m_{20}, m_{28}, m_{24}; m_5, m_7, m_{13}, m_{15}, m_{21}, m_{23}, m_{29}, m_{31}$.

The squares are read by dropping out the variables which change. Some possible

Groupings is

- (a) $m_0, m_{16} = \overline{B}\overline{C}\overline{D}\overline{E}$
- (b) $m_2, m_{18} = \overline{B}\overline{C}D\overline{E}$
- (c) $m_4, m_6, m_{20}, m_{22} = \overline{B}C\overline{E}$
- (d) $m_5, m_7, m_{13}, m_{15}, m_{21}, m_{23}, m_{29}, m_{31} = CE$
- (e) $m_8, m_9, m_{10}, m_{11}, m_{24}, m_{25}, m_{26}, m_{27} = B\overline{C}$

- $M_0, M_{16} = B + C + D + E$
- $M_2, M_{18} = B + C + \overline{D} + E$
- $M_4, M_6, M_{20}, M_{22} = B + \overline{C} + E$
- $M_5, M_7, M_{13}, M_{15}, M_{21}, M_{23}, M_{29}, M_{31} = \overline{C} + \overline{E}$
- $M_8, M_9, M_{10}, M_{11}, M_{24}, M_{25}, M_{26}, M_{27} = \overline{B} + C$



Ex: $F = \sum m(0,1,4,5,6,13,14,15,22,24,25,28,29,30,31)$ is SOP

POS is $F = \pi M(2,3,7,8,9,10,11,12,16,17,18,19,20,21,23,26,27)$

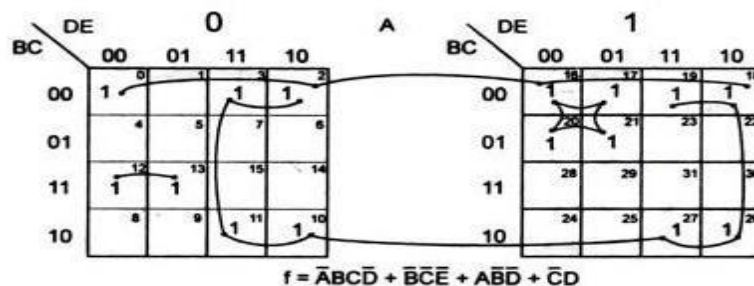
The real minimal expression is the minimal of the SOP and POS forms.

The reduction is done as

1. There is no isolated 1s
2. M_{12} can go only with m_{13} . Form a 2-square which is read as $A'B'CD'$
3. M_0 can go with m_2, m_{16} and m_{18} . so form a 4-square which is read as $B'C'E'$
4. M_{20}, m_{21}, m_{17} and m_{16} form a 4-square which is read as $AB'D'$
5. $M_2, m_3, m_{18}, m_{19}, m_{10}, m_{11}, m_{26}$ and m_{27} form an 8-square which is read as $C'D$
6. Write all the product terms in SOP form.

So the minimal expression is

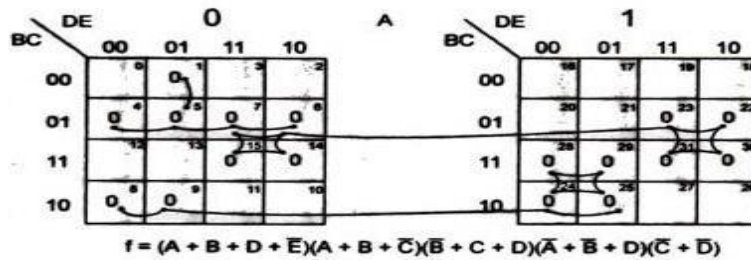
$$F_{\min} = A'B'CD' + B'C'E' + AB'D' + C'D \text{ (16 inputs)}$$



In the POS k-map ,the reduction is done as:

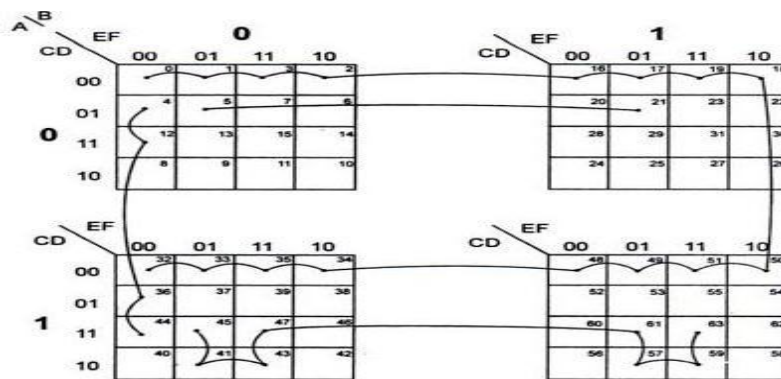
1. There are no isolated 0s
2. M_1 can go only with M_5 . So, make a 2-square, which is read as $(A + B + D + \bar{E})$.
3. M_4 can go with $M_5, M_7,$ and M_6 to form a 4-square, which is read as $(A + B + \bar{C})$.
4. M_8
5. M_{28}
6. M_{30}
7. Sum terms in POS form. So the minimal expression in POS is

$$F_{\min} = A'BcD' + B'C'E' + AB'D' + C'D$$



Six variable k-map:

Six variable k-map can have $2^6 = 64$ combinations as $ABCDEF, ABCDEF, \dots$ --- $ABCDEF$ with minterms $m_0, m_1, \dots, m_{63}$ respectively in SOP & $(A+B+C+D+E+F), \dots$ $(A + B + C + D + E + F)$ with maxterms $M_0, M_1, \dots, M_{63}$ respectively in POS form. It has $2^6 = 64$ squares or cells of the k-map are divided into 4 blocks of 16 squares each.



Some possible groupings in a six variable k-map

Don't care combinations: For certain input combinations, the value of the output is unspecified either because the input combinations are invalid or because the precise value of the output is of no consequence. The combinations for which the value of experiments are not specified are called don't care combinations are invalid or because the precise value of the output is of no consequence. The combinations for which the value of expressions is not specified are called don't care combinations or Optional Combinations, such expressions stand incompletely specified. The output is a don't care for these invalid combinations.

Ex: In XS-3 code system, the binary states 0000, 0001, 0010, 1101, 1110, 1111 are unspecified. & never occur called don't cares.

A standard SOP expression with don't cares can be converted into a standard POS form by keeping the don't cares as they are & writing the missing minterms of the SOP form as the maxterms of the POS form viceversa.

Don't cares denoted by $_X'$ or $_\phi'$

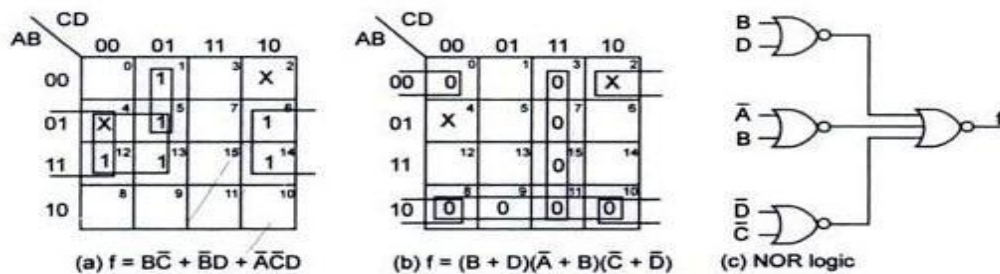
$$\text{Ex: } f = \sum m(1, 5, 6, 12, 13, 14) + d(2, 4)$$

$$\text{Or } f = \pi M(0, 3, 7, 9, 10, 11, 15) \cdot \pi d(2, 4)$$

$$\text{SOP minimal form } f_{\min} = BC + BD + ACD$$

$$\text{POS minimal form } f_{\min} = (B + D)(A + B)(C + D)$$

$$= B + D + A + B + (C + D)$$



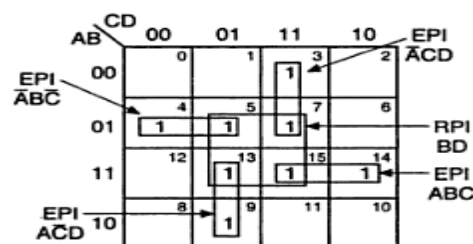
Prime implicants, Essential Prime implicants, Redundant prime implicants:

Each square or rectangle made up of the bunch of adjacent minterms is called a subcube. Each of these subcubes is called a Prime implicant (PI). The PI which contains at least one minterm which cannot be covered by any other prime implicants is called as Essential Prime implicant (EPI). The PI whose each 1 is covered at least by one EPI is called a Redundant Prime implicant (RPI). A PI which is neither an EPI nor a RPI is called a Selective Prime implicant (SPI).

The function has unique MSP comprising EPI is

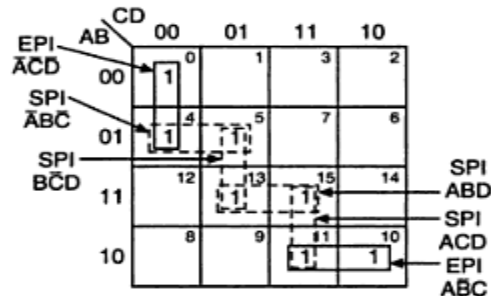
$$F(A, B, C, D) = ACD + ABC + AC\bar{D} + A\bar{B}C$$

The RPI $\bar{B}D$ may be included without changing the function but the resulting expression would not be in minimal SOP(MSP) form.



Essential and Redundant Prime Implicants

$F(A,B,C,D)=\sum m(0,4,5,10,11,13,15)$ SPI are marked by dotted squares, shows MSP form of a function need not be unique.



Essential and Selective Prime Implicants

Here, the MSP form is obtained by including two EPI's & selecting a set of SPI's to cover remaining uncovered minterms 5,13,15. & these can be covered as

(A) (4,5) & (13,15) ----- $A BC + ABD$

(B) (5,13) & (13,15) ----- $BC D + ABD$

(C) (5,13) & (15,11) ----- $BC D + ACD$

$$F(A,B,C,D) = A C D + ABC \text{-----EPI's} + A BC + ABD$$

(OR) $F(A,B,C,D) = A C D + ABC \text{-----EPI's} + BC D + ABD$

(OR) $F(A,B,C,D) = A C D + ABC \text{-----EPI's} + BC D + ACD$

False PI's Essential False PI's, Redundant False PI's & Selective False PI's:

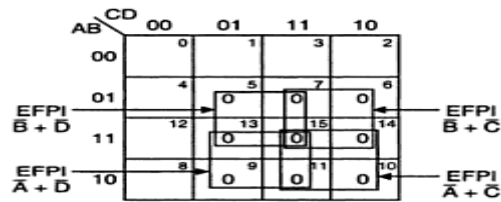
The maxterms are called false minterms. The PI's is obtained by using the maxterms are called False PI's (FPI). The FPI which contains at least one $\underline{0}$ which can't be covered by only other FPI is called an Essential False Prime implicant (ESPI)

$$F(A,B,C,D) = \sum m(0,1,2,3,4,8,12)$$

$$= \pi M(5,6,7,9,10,11,13,14,15)$$

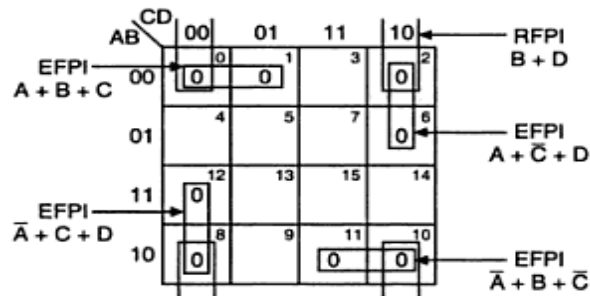
$$F_{\min} = (B+C)(A+C)(A+D)(B+D)$$

All the FPI, EFPI's as each of them contain atleast one $\underline{0}$ which can't be covered by any other FPI



Essential False Prime implicants

Consider Function $F(A,B,C,D) = \pi M(0,1,2,6,8,10,11,12)$



Essential and Redundant False Prime Implicants

Mapping when the function is not expressed in minterms (maxterms):

An expression in k-map must be available as a sum (product) of minterms (maxterms). However if not so expressed, it is not necessary to expand the expression algebraically into its minterms (maxterms). Instead, expansion into minterms (maxterms) can be accomplished in the process of entering the terms of the expression on the k-map.

Limitations of Karnaugh maps:

- Convenient as long as the number of variables does not exceed six.
- Manual technique, simplification process is heavily dependent on the human abilities.

Quine-Mccluskey Method:

It also known as *Tabular method*. It is more systematic method of minimizing expressions of even larger number of variables. It is suitable for hand computation as well as computation by machines i.e., programmable. . The procedure is based on repeated application of the combining theorem.

$PA + PA = P$ (P is set of literals) on all adjacent pairs of terms, yields the set of all PI's from which a minimal sum may be selected.

Consider expression

$$\sum m(0,1,4,5) = A\bar{B}\bar{C} + A\bar{B}C + A\bar{B}C + A\bar{B}C$$

First, second terms & third, fourth terms can be combined

$$A B(C + C) + A B(C + C) = A B + A B$$

Reduced to

$$B(A + A) = B$$

The same result can be obtained by combining m_0 & m_4 & m_1 & m_5 in first step & resulting terms in the second step .

Procedure:

- Decimal Representation
- Don't cares
- PI chart
- EPI
- Dominating Rows & Columns
- Determination of Minimal expressions in complex cases.

Branching Method:

EXAMPLE 3.29 Obtain the set of prime implicants for the Boolean expression
 $f = \sum m(0, 1, 6, 7, 8, 9, 13, 14, 15)$ using the tabular method.

Solution

Group the minterms in terms of the number of 1s present in them and write their binary designations. The procedure to obtain the prime implicants is shown in Table 3.3.

Table 3.3 Example 3.29

Column 1			Column 2		Column 3
	Minterm	Binary designation	A B C D		A B C D
Index 0	0	0 0 0 0 ✓	0, 1 (1)	0 0 0 - ✓	0, 1, 8, 9 (1, 8) - 0 0 - Q
Index 1	1	0 0 0 1 ✓	0, 8 (8)	- 0 0 0 ✓	...
	8	1 0 0 0 ✓	1, 9 (8)	- 0 0 1 ✓	...
Index 2	6	0 1 1 0 ✓	8, 9 (1)	1 0 0 - ✓	6, 7, 14, 15 (1, 8) - 1 1 - P
	9	1 0 0 1 ✓	6, 7 (1)	0 1 1 - ✓	
Index 3	7	0 1 1 1 ✓	6, 14 (8)	- 1 1 0 ✓	
	13	1 1 0 1 ✓	9, 13 (4)	1 - 0 1 S	
	14	1 1 1 0 ✓	7, 15 (8)	- 1 1 1 ✓	
Index 4	15	1 1 1 1 ✓	13, 15 (2)	1 1 - 1 R	
			14, 15 (1)	1 1 1 - ✓	

Comparing the terms of index 0 with the terms of index 1 of column 1, $m_0(0000)$ is combined with $m_1(0001)$ to yield 0, 1 (1), i.e. 000–. This is recorded in column 2 and 0000 and 0001 are checked off in column 1. $m_0(0000)$ is combined with $m_8(1000)$ to yield 0, 8 (8), i.e. –000. This is recorded in column 2 and 1000 is checked off in column 1. Note that 0000 of column 1 has already been checked off. No more combinations of terms of index 0 and index 1 are possible. So, draw a line below the last combination of these groups, i.e. below 0, 8 (8), –000 in column 2. Now 0, 1 (1), i.e. 000– and 0, 8 (8), i.e. –000 are the terms in the first group of column 2.

Comparing the terms of index 1 with the terms of index 2 in column 1, $m_1(0001)$ is combined with $m_9(1001)$ to yield 1, 9 (8), i.e. –001. This is recorded in column 2 and 1001 is checked off in column 1 because 0001 has already been checked off. $m_8(1000)$ is combined with $m_9(1001)$ to yield 8, 9 (1), i.e. 100–. This is recorded in column 2. 1000 and 1001 of column 1 have already been checked off. So, no need to check them off again. No more combinations of terms of index 1 and index 2 are possible. So, draw a line below the last combination of these groups, i.e. 8, 9 (1),

–001 in column 2. Now 1, 9 (8), i.e. –001 and 8, 9 (1), i.e. 100– are the terms in the second group of column 2.

Similarly, comparing the terms of index 2 with the terms of index 3 in column 1,

$m_6(0110)$ and $m_7(0111)$ yield 6, 7 (1), i.e. 011–. Record it in column 2 and check off 6(0110) and 7(0111).

$m_6(0110)$ and $m_{14}(1110)$ yield 6, 14 (8), i.e. –110. Record it in column 2 and check off 6(0110) and 14(1110).

$m_9(1001)$ and $m_{13}(1101)$ yield 9, 13 (4), i.e. 1–01. Record it in column 2 and check off 9(1001) and 13(1101).

So, 6, 7 (1), i.e. 011–, and 6, 14 (8), i.e. –110 and 9, 13 (4), i.e. 1–01 are the terms in group 3 of column 2. Draw a line at the end of 9, 13 (4), i.e. 1–01.

Also, comparing the terms of index 3 with the terms of index 4 in column 1,

$m_7(0111)$ and $m_{15}(1111)$ yield 7, 15 (8), i.e. –111. Record it in column 2 and check off 7(0111) and 15(1111).

$m_{13}(1101)$ and $m_{15}(1111)$ yield 13, 15 (2), i.e. 11–1. Record it in column 2 and check off 13 and 15.

$m_{14}(1110)$ and $m_{15}(1111)$ yield 14, 15 (1), i.e. 111–. Record it in column 2 and check off 14 and 15.

So, 7, 15 (8), i.e. –111, and 13, 15 (2), i.e. 11–1 and 14, 15 (1), i.e. 111– are the terms in group 4 of column 2. Column 2 is completed now.

Comparing the terms of group 1 with the terms of group 2 in column 2, the terms 0, 1 (1), i.e. 000– and 8, 9 (1), i.e. 100– are combined to form 0, 1, 8, 9 (1, 8), i.e. –00–. Record it in group 1 of column 3 and check off 0, 1 (1), i.e. 000–, and 8, 9 (1), i.e. 100– of column 2. The terms 0, 8 (8), i.e. –000 and 1, 9 (8), i.e. –001 are combined to form 0, 1, 8, 9 (1, 8), i.e. –00–. This has already been recorded in column 3. So, no need to record again. Check off 0, 8 (8), i.e. –000 and 1, 9 (8), i.e. –001 of column 2. Draw a line below 0, 1, 8, 9 (1, 8), i.e. –00–. This is the only term in group 1 of column 3. No term of group 2 of column 2 can be combined with any term of group 3 of column 2. So, no entries are made in group 2 of column 2.

Comparing the terms of group 3 of column 2 with the terms of group 4 of column 2, the terms 6, 7 (1), i.e. 011–, and 14, 15 (1), i.e. 111– are combined to form 6, 7, 14, 15 (1, 8), i.e. –11–. Record it in group 3 of column 3 and check off 6, 7 (1), i.e. 011– and 14, 15 (1), i.e. 111– of column 2. The terms 6, 14 (8), i.e. –110 and 7, 15 (8), i.e. –111 are combined to form 6, 7, 14, 15 (1, 8), i.e. –11–. This has already been recorded in column 3; so, check off 6, 14 (8), i.e. –110 and 7, 15 (8), i.e. –111 of column 2.

Observe that the terms 9, 13 (4), i.e. 1–01 and 13, 15 (2), i.e. 11–1 cannot be combined with any other terms. Similarly in column 3, the terms 0, 1, 8, 9 (1, 8), i.e. –00– and 6, 7, 14, 15 (1, 8), i.e. –11– cannot also be combined with any other terms. So, these 4 terms are the prime implicants.

The terms, which cannot be combined further, are labelled as P, Q, R, and S. These form the set of prime implicants.

EX:

Obtain the minimal expression for $f = \sum m(1, 2, 3, 5, 6, 7, 8, 9, 12, 13, 15)$ using the tabular method.

Solution

The procedure to obtain the set of prime implicants is illustrated in Table 3.4.

Table 3.4 Example 3.30

Step 1		Step 2	Step 3	
Index 1	1 ✓	1, 3 (2) ✓	1, 3, 5, 7 (2, 4)	T
	2 ✓	1, 5 (4) ✓	1, 5, 9, 13 (4, 8)	S
	8 ✓	1, 9 (8) ✓	2, 3, 6, 7 (1, 4)	R
Index 2	3 ✓	2, 3 (1) ✓	8, 9, 12, 13 (1, 4)	Q
	5 ✓	2, 6 (4) ✓	5, 7, 13, 15 (2, 8)	P
	6 ✓	8, 9 (1) ✓		
	9 ✓	8, 12 (4) ✓		
	12 ✓	3, 7 (4) ✓		
Index 3	7 ✓	5, 7 (2) ✓		
	13 ✓	5, 13 (8) ✓		
Index 4	15 ✓	6, 7 (1) ✓		
		9, 13 (4) ✓		
		12, 13 (1) ✓		
		7, 15 (8) ✓		
		13, 15 (2) ✓		

The non-combinable terms P, Q, R, S and T are recorded as prime implicants.

$$P \rightarrow 5, 7, 13, 15 (2, 8) = X \ 1 \ X \ 1 = BD$$

(Literals with weights 2 and 8, i.e. C and A are deleted. The lowest minterm is $m_5 (5 = 4 + 1)$. So, literals with weights 4 and 1, i.e. B and D are present in non-complemented form. So, read it as BD.)

$$Q \rightarrow 8, 9, 12, 13 (1, 4) = 1 \ X \ 0 \ X = A\bar{C}$$

(Literals with weights 1 and 4, i.e. D and B are deleted. The lowest minterm is m_8 . So, literal with weight 8 is present in non-complemented form and literal with weight 2 is present in complemented form. So, read it as $A\bar{C}$.)

$$R \rightarrow 2, 3, 6, 7 (1, 4) = 0 \ X \ 1 \ X = \bar{A}C$$

(Literals with weights 1 and 4, i.e. D and B are deleted. The lowest minterm is m_2 . So, literal with weight 2 is present in non-complemented form and literal with weight 8 is present in complemented form. So, read it as $\bar{A}C$.)

$$S \rightarrow 1, 5, 9, 13 (4, 8) = X \ X \ 0 \ 1 = \bar{C}D$$

(Literals with weights 4 and 8, i.e. B and A are deleted. The lowest minterm is m_1 . So, literal with weight 1 is present in non-complemented form and literal with weight 2 is present in complemented form. So, read it as $\bar{C}D$.)

$$T \rightarrow 1, 3, 5, 7 (2, 4) = 0 \ X \ X \ 1 = \bar{A}D$$

(Literals with weights 2 and 4, i.e. C and B are deleted. The lowest minterm is 1. So, literal with weight 1 is present in non-complemented form and literal with weight 8 is present in complemented form. So, read it as $\bar{A}D$.)

The prime implicant chart of the expression

$$f = \sum m(1, 2, 3, 5, 6, 7, 8, 9, 12, 13, 15)$$

is as shown in Table 3.5. It consists of 11 columns corresponding to the number of minterms and 5 rows corresponding to the prime implicants P, Q, R, S, and T generated. Row R contains four x's at the intersections with columns 2, 3, 6, and 7, because these minterms are covered by the prime implicant R. A row is said to cover the columns in which it has x's. The problem now is to select a minimal subset of prime implicants, such that each column contains at least one x in the rows corresponding to the selected subset and the total number of literals in the prime implicants selected is as small as possible. These requirements guarantee that the number of unions of the selected prime implicants is equal to the original number of minterms and that, no other expression containing fewer literals can be found.

Table 3.5 Example 3.30: Prime implicant chart

		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	1	2	3	5	6	7	8	9	12	13	15
*P → 5, 7, 13, 15 (2, 8)				x		x				x	x
*Q → 8, 9, 12, 13 (1, 4)							x	x	x	x	
*R → 2, 3, 6, 7 (1, 4)		x	x		x	x					
S → 1, 5, 9, 13 (4, 8)	x			x				x		x	
T → 1, 3, 5, 7 (2, 4)	x		x	x		x					

In the prime implicant chart of Table 3.5, m_2 and m_6 are covered by R only. So, R is an essential prime implicant. So, check off all the minterms covered by it, i.e. m_2, m_3, m_6 , and m_7 . Q is also an essential prime implicant because only Q covers m_8 and m_{12} . Check off all the minterms covered by it, i.e. m_8, m_9, m_{12} , and m_{13} . P is also an essential prime implicant, because m_{15} is covered only by P. So check off m_{15}, m_5, m_7 , and m_{13} covered by it. Thus, only minterm 1 is not covered. Either row S or row T can cover it and both have the same number of literals. Thus, two minimal expressions are possible.

$$P + Q + R + S = BD + A\bar{C} + \bar{A}C + \bar{C}D$$

or

$$P + Q + R + T = BD + A\bar{C} + \bar{A}C + \bar{A}D$$

Combinational Logic Design

Logic circuits for digital systems may be combinational or sequential. The output of a combinational circuit depends on its present inputs only. Combinational circuit processing operation fully specified logically by a set of Boolean functions. A combinational circuit consists of input variables, logic gates and output variables. Both input and output data are represented by signals, i.e., they exist in two possible values. One is logic –1 and the other logic 0.

Combinational Circuits

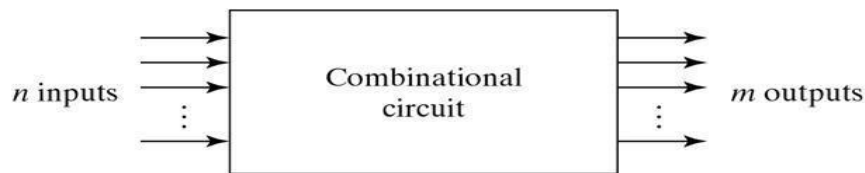


Fig. Block Diagram of Combinational Circuit

For n input variables, there are 2^n possible combinations of binary input variables. For each possible input combination, there is one and only one possible output combination. A combinational circuit can be described by m Boolean functions, one for each output variable. Usually the inputs come from flip-flops and outputs go to flip-flops.

Design Procedure:

1. The problem is stated
2. The number of available input variables and required output variables is determined.
3. The input and output variables are assigned letter symbols.
4. The truth table that defines the required relationship between inputs and outputs is derived.
5. The simplified Boolean function for each output is obtained.
6. The logic diagram is drawn.

Adders:

Digital computers perform variety of information processing tasks, the one is arithmetic operations. And the most basic arithmetic operation is the addition of two binary digits. i.e, 4 basic possible operations are:

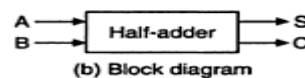
$$0+0=0, 0+1=1, 1+0=1, 1+1=10$$

The first three operations produce a sum whose length is one digit, but when augends and addend bits are equal to 1, the binary sum consists of two digits. The higher significant bit of this result is called a carry. A combinational circuit that performs the addition of two bits is called a half-adder. One that performs the addition of 3 bits (two significant bits & previous carry) is called a full adder. & 2 half adder can employ as a full-adder.

The Half Adder: A Half Adder is a combinational circuit with two binary inputs (augends and addend bits) and two binary outputs (sum and carry bits.) It adds the two inputs (A and B) and produces the sum (S) and the carry (C) bits. It is an arithmetic operation of addition of two single bit words.

Inputs		Outputs	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

(a) Truth table



(b) Block diagram

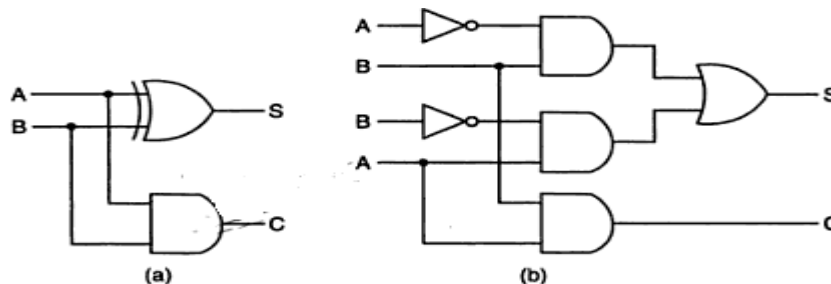
The Sum(S) bit and the carry (C) bit, according to the rules of binary addition, the sum (S) is the X-OR of A and B (It represents the LSB of the sum). Therefore,

$$S = A \oplus B$$

The carry (C) is the AND of A and B (it is 0 unless both the inputs are 1). Therefore,

$$C = AB$$

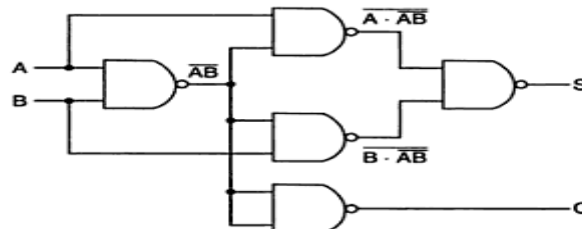
A half-adder can be realized by using one X-OR gate and one AND gate a



Logic diagrams of half-adder

NAND LOGIC:

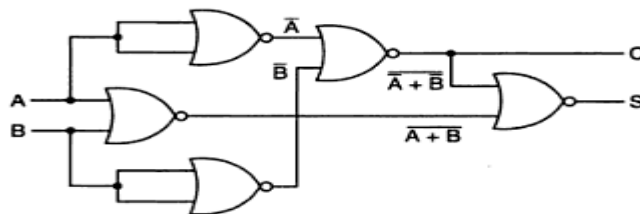
$$\begin{aligned}
 S &= A\bar{B} + \bar{A}B = A\bar{B} + A\bar{A} + \bar{A}B + B\bar{B} \\
 &= A(\bar{A} + \bar{B}) + B(\bar{A} + \bar{B}) \\
 &= A \cdot \bar{A}\bar{B} + B \cdot \bar{A}\bar{B} \\
 &= \overline{A \cdot AB \cdot B \cdot AB} \\
 C &= AB = \overline{AB}
 \end{aligned}$$



Logic diagram of a half-adder using only 2-input NAND gates.

NOR Logic:

$$\begin{aligned}
 S &= A\bar{B} + \bar{A}B = A\bar{B} + A\bar{A} + \bar{A}B + B\bar{B} \\
 &= A(\bar{A} + \bar{B}) + B(\bar{A} + \bar{B}) \\
 &= (A + B)(\bar{A} + \bar{B}) \\
 &= \overline{A + B + \bar{A} + \bar{B}} \\
 C &= AB = \overline{AB} = \overline{A + B}
 \end{aligned}$$



Logic diagram of a half-adder using only 2-input NOR gates.

The Full Adder:

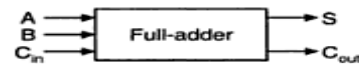
A Full-adder is a combinational circuit that adds two bits and a carry and outputs a sum bit and a carry bit. To add two binary numbers, each having two or more bits, the LSBs can be added by using a half-adder. The carry resulted from the addition of the LSBs is carried over to the next significant column and added to the two bits in that column. So, in the second and higher columns, the two data bits of that column and the carry bit generated from the addition in the previous column need to be added.

The full-adder adds the bits A and B and the carry from the previous column called the carry-in C_{in} and outputs the sum bit S and the carry bit called the carry-out C_{out} . The variable S gives the value of the least significant bit of the sum. The variable C_{out} gives the output carry. The

eight rows under the input variables designate all possible combinations of 1s and 0s that these variables may have. The 1s and 0s for the output variables are determined from the arithmetic sum of the input bits. When all the bits are 0s, the output is 0. The S output is equal to 1 when only 1 input is equal to 1 or when all the inputs are equal to 1. The C_{out} has a carry of 1 if two or three inputs are equal to 1.

Inputs			Sum	Carry
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

(a) Truth table



(b) Block diagram

Full-adder.

From the truth table, a circuit that will produce the correct sum and carry bits in response to every possible combination of A, B and C_{in} is described by

$$S = \overline{A}\overline{B}C_{in} + \overline{A}B\overline{C_{in}} + A\overline{B}\overline{C_{in}} + ABC_{in}$$

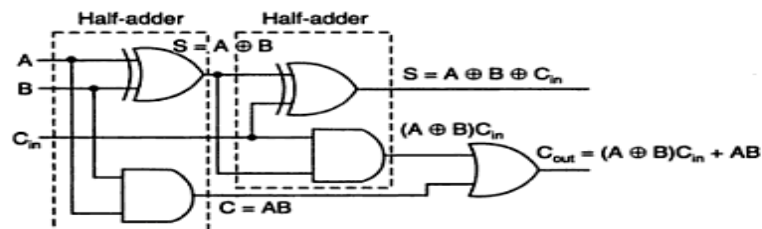
$$C_{out} = \overline{A}BC_{in} + A\overline{B}C_{in} + AB\overline{C_{in}} + ABC_{in}$$

and

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AC_{in} + BC_{in} + AB$$

The sum term of the full-adder is the X-OR of A, B, and C_{in} , i.e, the sum bit the modulo sum of the data bits in that column and the carry from the previous column. The logic diagram of the full-adder using two X-OR gates and two AND gates (i.e, Two half adders) and one OR gate is



Logic diagram of a full-adder using two half-adders.

The block diagram of a full-adder using two half-adders is :



Block diagram of a full-adder using two half-adders.

Even though a full-adder can be constructed using two half-adders, the disadvantage is that the bits must propagate through several gates in accession, which makes the total propagation delay greater than that of the full-adder circuit using AOI logic.

The Full-adder neither can also be realized using universal logic, i.e., either only NAND gates or only NOR gates as

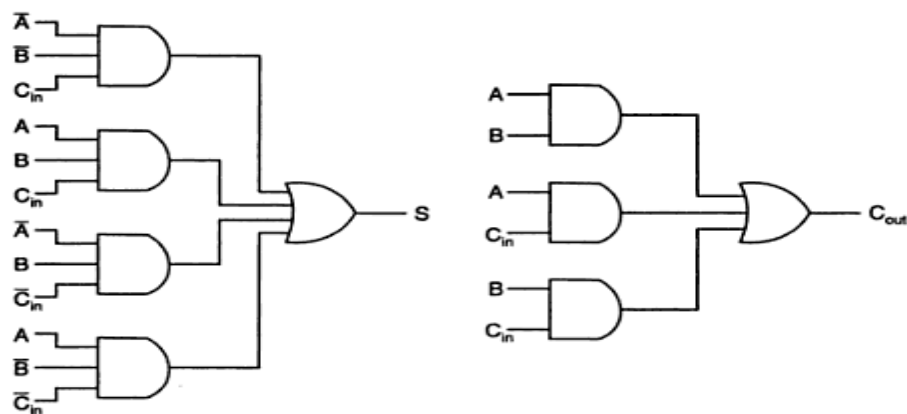
$$A \oplus B = \overline{\overline{A \cdot AB \cdot B \cdot AB}}$$

Then

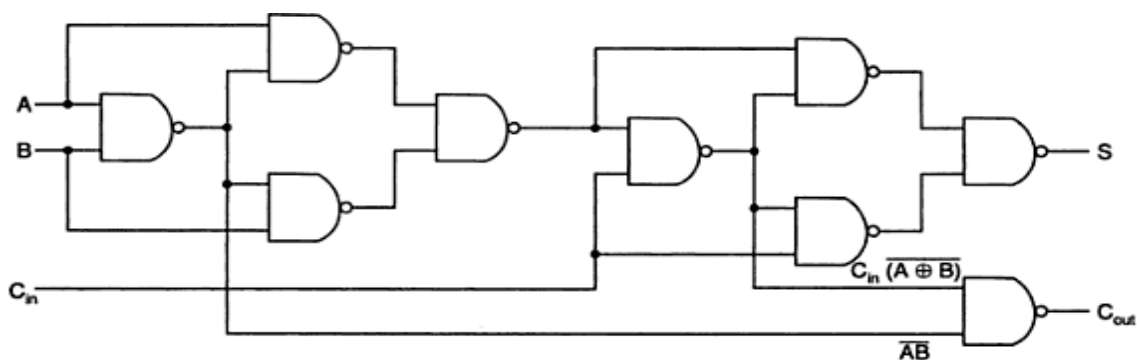
$$S = A \oplus B \oplus C_{in} = \overline{\overline{(A \oplus B) \cdot (A \oplus B)C_{in} \cdot C_{in} \cdot (A \oplus B)C_{in}}}$$

NAND Logic:

$$C_{out} = C_{in}(A \oplus B) + AB = \overline{\overline{C_{in}(A \oplus B) \cdot AB}}$$

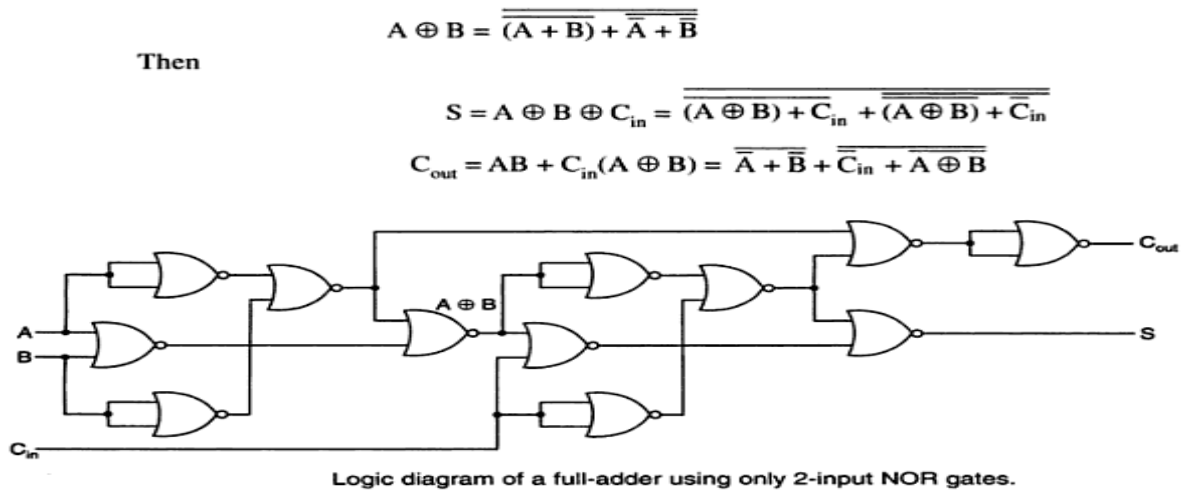


Sum and carry bits of a full-adder using AOI logic.



Logic diagram of a full-adder using only 2-input NAND gates.

NOR Logic:



Subtractors:

The subtraction of two binary numbers may be accomplished by taking the complement of the subtrahend and adding it to the minuend. By this, the subtraction operation becomes an addition operation and instead of having a separate circuit for subtraction, the adder itself can be used to perform subtraction. This results in reduction of hardware. In subtraction, each subtrahend bit of the number is subtracted from its corresponding significant minuend bit to form a difference bit. If the minuend bit is smaller than the subtrahend bit, a 1 is borrowed from the next significant position., that has been borrowed must be conveyed to the next higher pair of bits by means of a signal coming out (output) of a given stage and going into (input) the next higher stage.

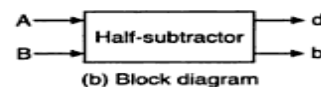
The Half-Subtractor:

A Half-subtractor is a combinational circuit that subtracts one bit from the other and produces the difference. It also has an output to specify if a 1 has been borrowed. . It is used to subtract the LSB of the subtrahend from the LSB of the minuend when one binary number is subtracted from the other.

A Half-subtractor is a combinational circuit with two inputs A and B and two outputs d and b. d indicates the difference and b is the output signal generated that informs the next stage that a 1 has been borrowed. When a bit B is subtracted from another bit A, a difference bit (d) and a borrow bit (b) result according to the rules given as

Inputs		Outputs	
A	B	d	b
0	0	0	0
1	0	1	0
1	1	0	0
0	1	1	1

(a) Truth table



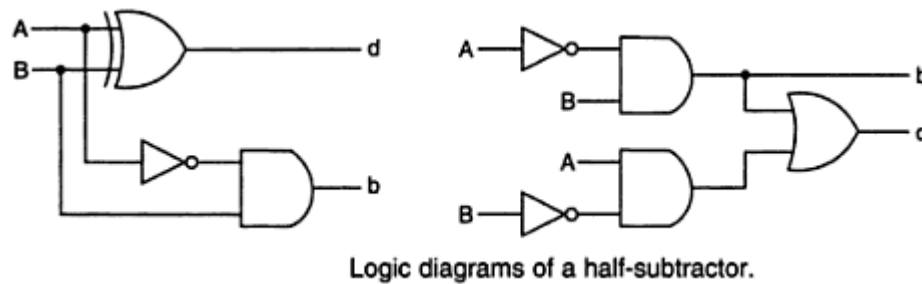
Half-subtractor.

The output borrow b is a 0 as long as $A \geq B$. It is a 1 for $A=0$ and $B=1$. The d output is the result of the arithmetic operation $2b+A-B$.

A circuit that produces the correct difference and borrow bits in response to every possible combination of the two 1-bit numbers is , therefore ,

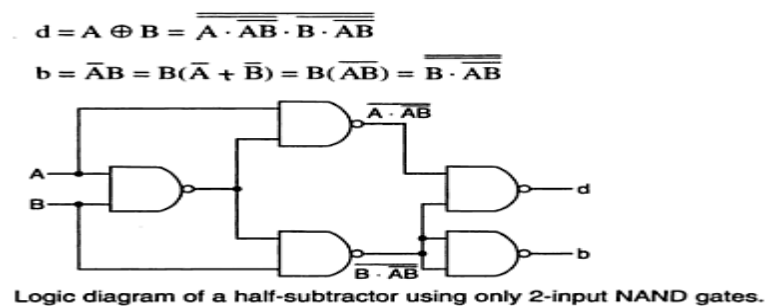
$$d = AB + A\bar{B} = A \oplus B \text{ and } b = A\bar{B}$$

That is, the difference bit is obtained by X-OR ing the two inputs, and the borrow bit is obtained by ANDing the complement of the minuend with the subtrahend. Note that logic for this exactly the same as the logic for output S in the half-adder.



A half-subtractor can also be realized using universal logic either using only NAND gates or using NOR gates as:

NAND Logic:

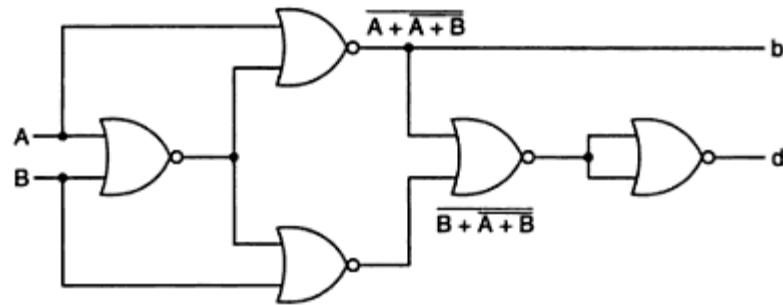


NOR Logic:

$$d = A \oplus B = A\bar{B} + \bar{A}B = A\bar{B} + B\bar{B} + \bar{A}B + A\bar{A}$$

$$= \bar{B}(A + B) + \bar{A}(A + B) = \overline{B + A + B + A + A + B}$$

$$d = \bar{A}B = \bar{A}(A + B) = \overline{\bar{A}(A + B)} = A + (A + B)$$



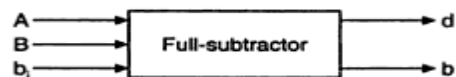
Logic diagram of a half-subtractor using only 2-input NOR gates.

The Full-Subtractor:

The half-subtractor can be only for LSB subtraction. IF there is a borrow during the subtraction of the LSBs, it affects the subtraction in the next higher column; the subtrahend bit is subtracted from the minuend bit, considering the borrow from that column used for the subtraction in the preceding column. Such a subtraction is performed by a full-subtractor. It subtracts one bit (B) from another bit (A), when already there is a borrow b_i from this column for the subtraction in the preceding column, and outputs the difference bit (d) and the borrow bit(b) required from the next d and b. The two outputs present the difference and output borrow. The 1s and 0s for the output variables are determined from the subtraction of $A-B-b_i$.

Inputs			Difference	Borrow
A	B	b_i	d	b
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

(a) Truth table



(b) Block diagram

Full-subtractor.

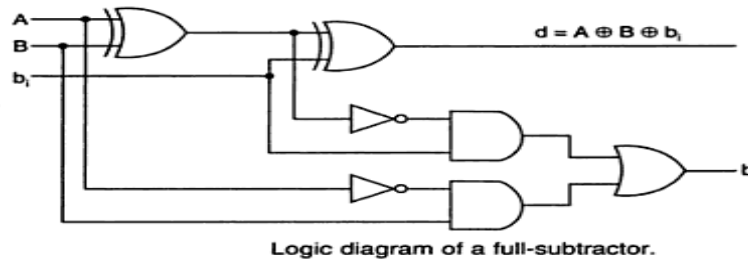
From the truth table, a circuit that will produce the correct difference and borrow bits in response to every possible combinations of A,B and b_i is

$$\begin{aligned}
 d &= \bar{A}\bar{B}b_i + \bar{A}B\bar{b}_i + A\bar{B}\bar{b}_i + ABb_i \\
 &= b_i(AB + \bar{A}\bar{B}) + \bar{b}_i(A\bar{B} + \bar{A}B) \\
 &= b_i(\overline{A \oplus B}) + \bar{b}_i(A \oplus B) = A \oplus B \oplus b_i
 \end{aligned}$$

and

$$\begin{aligned}
 b &= \bar{A}\bar{B}b_i + \bar{A}B\bar{b}_i + \bar{A}Bb_i + ABb_i = \bar{A}B(b_i + \bar{b}_i) + (AB + \bar{A}\bar{B})b_i \\
 &= \bar{A}B + (A \oplus B)b_i
 \end{aligned}$$

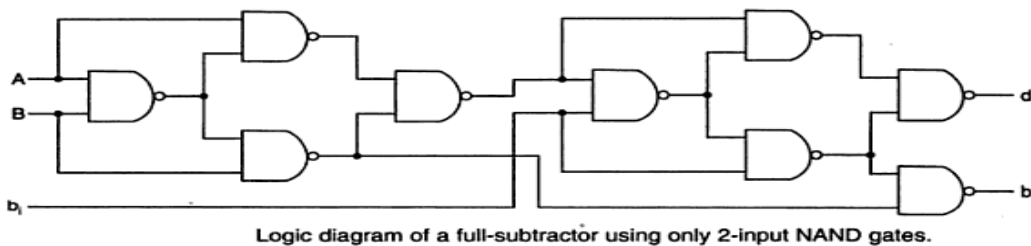
A full-subtractor can be realized using X-OR gates and AOI gates as



The full subtractor can also be realized using universal logic either using only NAND gates or using NOR gates as:

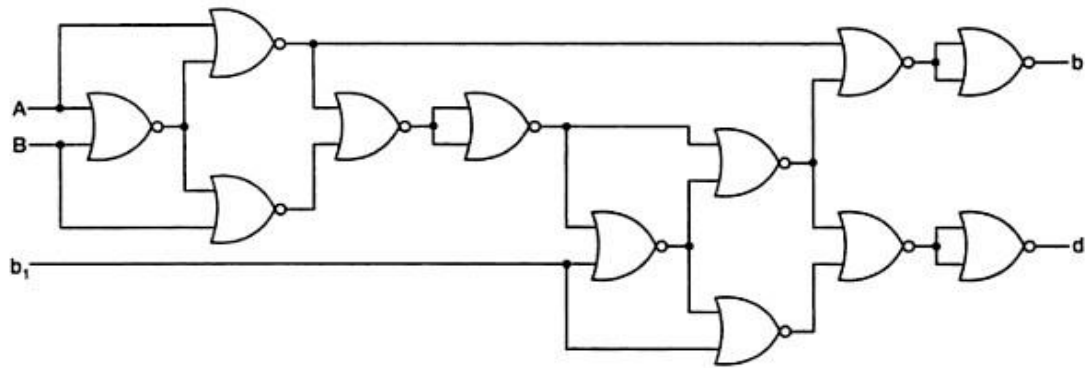
NAND Logic:

$$\begin{aligned}
 d &= A \oplus B \oplus b_i = \overline{\overline{(A \oplus B)} \oplus b_i} = \overline{\overline{(A \oplus B)}(A \oplus B)b_i \cdot b_i(A \oplus B)b_i} \\
 b &= \overline{AB} + b_i(\overline{A \oplus B}) = \overline{AB + b_i(A \oplus B)} \\
 &= \overline{AB \cdot b_i(A \oplus B)} = \overline{B(\overline{A + B}) \cdot b_i(b_i + (A \oplus B))} \\
 &= \overline{B \cdot AB \cdot b_i[b_i \cdot (A \oplus B)]}
 \end{aligned}$$



NOR Logic:

$$\begin{aligned}
 d &= A \oplus B \oplus b_i = \overline{\overline{(A \oplus B)} \oplus b_i} \\
 &= \overline{(A \oplus B)b_i + (\overline{A \oplus B})\overline{b_i}} \\
 &= \overline{[(A \oplus B) + (\overline{A \oplus B})\overline{b_i}][b_i + (\overline{A \oplus B})\overline{b_i}]} \\
 &= \overline{(A \oplus B) + (\overline{A \oplus B}) + b_i + b_i + (\overline{A \oplus B}) + b_i} \\
 &= \overline{(A \oplus B) + (\overline{A \oplus B}) + b_i + b_i + (\overline{A \oplus B}) + b_i} \\
 b &= \overline{AB} + b_i(\overline{A \oplus B}) \\
 &= \overline{A(A + B) + (\overline{A \oplus B})[(A \oplus B) + b_i]} \\
 &= \overline{A + (A + B) + (A \oplus B) + (A \oplus B) + b_i}
 \end{aligned}$$

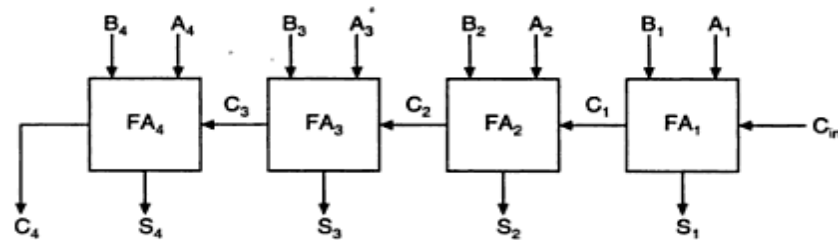


Logic diagram of a full subtractor using only 2-input NOR gates.

Binary Parallel Adder:

A binary parallel adder is a digital circuit that adds two binary numbers in parallel form and produces the arithmetic sum of those numbers in parallel form. It consists of full adders connected in a chain, with the output carry from each full-adder connected to the input carry of the next full-adder in the chain.

The interconnection of four full-adder (FA) circuits to provide a 4-bit parallel adder. The augends bits of A and addend bits of B are designated by subscript numbers from right to left, with subscript 1 denoting the lower –order bit. The carries are connected in a chain through the full-adders. The input carry to the adder is C_{in} and the output carry is C_4 . The S output generates the required sum bits. When the 4-bit full-adder circuit is enclosed within an IC package, it has four terminals for the augends bits, four terminals for the addend bits, four terminals for the sum bits, and two terminals for the input and output carries. An n-bit parallel adder requires n-full adders. It can be constructed from 4-bit, 2-bit and 1-bit full adder ICs by cascading several packages. The output carry from one package must be connected to the input carry of the one with the next higher –order bits. The 4-bit full adder is a typical example of an MSI function.



Logic diagram of a 4-bit binary parallel adder.

Ripple carry adder:

In the parallel adder, the carry –out of each stage is connected to the carry-in of the next stage. The sum and carry-out bits of any stage cannot be produced, until sometime after the carry-in of that stage occurs. This is due to the propagation delays in the logic circuitry,

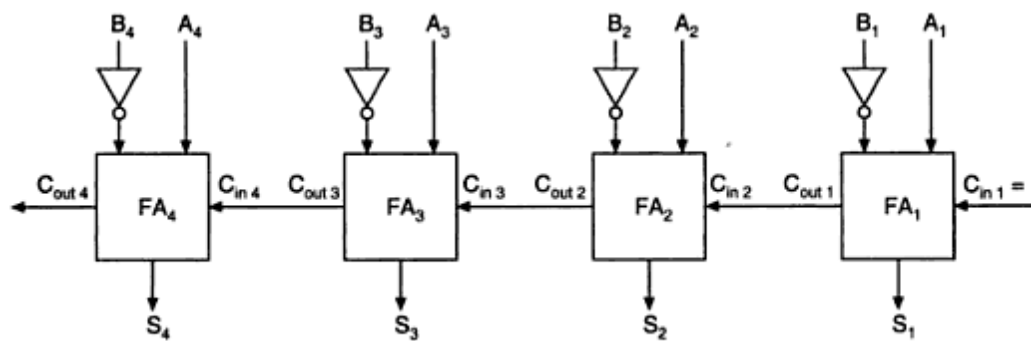
which lead to a time delay in the addition process. The carry propagation delay for each full-adder is the time between the application of the carry-in and the occurrence of the carry-out.

The 4-bit parallel adder, the sum (S_1) and carry-out (C_1) bits given by FA_1 are not valid, until after the propagation delay of FA_1 . Similarly, the sum S_2 and carry-out (C_2) bits given by FA_2 are not valid until after the cumulative propagation delay of two full adders (FA_1 and FA_2), and so on. At each stage, the sum bit is not valid until after the carry bits in all the preceding stages are valid. Carry bits must propagate or ripple through all stages before the most significant sum bit is valid. Thus, the total sum (the parallel output) is not valid until after the cumulative delay of all the adders.

The parallel adder in which the carry-out of each full-adder is the carry-in to the next most significant adder is called a ripple carry adder.. The greater the number of bits that a ripple carry adder must add, the greater the time required for it to perform a valid addition. If two numbers are added such that no carries occur between stages, then the add time is simply the propagation time through a single full-adder.

4-Bit Parallel Subtractor:

The subtraction of binary numbers can be carried out most conveniently by means of complements, the subtraction $A-B$ can be done by taking the 2's complement of B and adding it to A . The 2's complement can be obtained by taking the 1's complement and adding 1 to the least significant pair of bits. The 1's complement can be implemented with inverters as

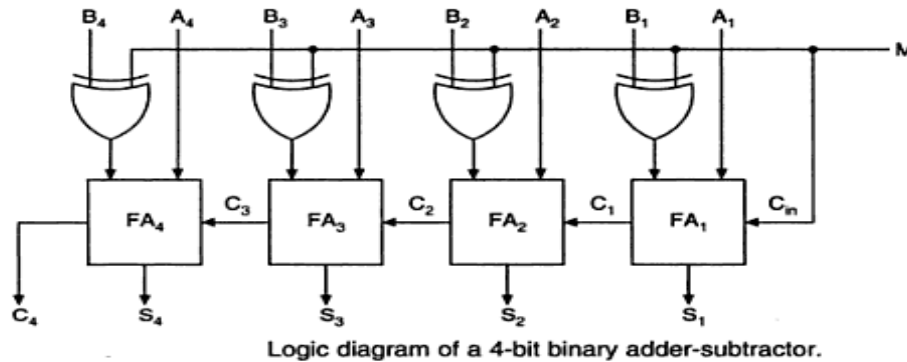


Logic diagram of a 4-bit parallel subtractor.

Binary-Adder Subtractor:

A 4-bit adder-subtractor, the addition and subtraction operations are combined into one circuit with one common binary adder. This is done by including an X-OR gate with each full-adder. The mode input M controls the operation. When $M=0$, the circuit is an adder, and when $M=1$, the circuit becomes a subtractor. Each X-OR gate receives input M and one of the inputs of B . When $M=0$, $B \oplus 0 = B$. The full-adder receives the value of B , the input carry is 0

and the circuit performs $A+B$. when $B \oplus 1 = B'$ and $C_1=1$. The B inputs are complemented and a 1 is through the input carry. The circuit performs the operation A plus the 2's complement of B .

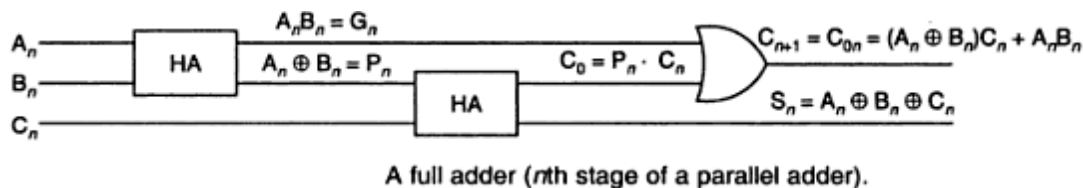


The Look-Ahead –Carry Adder:

In parallel-adder, the speed with which an addition can be performed is governed by the time required for the carries to propagate or ripple through all of the stages of the adder. The look-ahead carry adder speeds up the process by eliminating this ripple carry delay. It examines all the input bits simultaneously and also generates the carry-in bits for all the stages simultaneously.

The method of speeding up the addition process is based on the two additional functions of the full-adder, called the carry generate and carry propagate functions.

Consider one full adder stage; say the n th stage of a parallel adder as shown in fig. we know that is made by two half adders and that the half adder contains an X-OR gate to produce the sum and an AND gate to produce the carry. If both the bits A_n and B_n are 1s, a carry has to be generated in this stage regardless of whether the input carry C_{in} is a 0 or a 1. This is called generated carry, expressed as $G_n = A_n \cdot B_n$ which has to appear at the output through the OR gate as shown in fig.



There is another possibility of producing a carry out. X-OR gate inside the half-adder at the input produces an intermediary sum bit- call it P_n -which is expressed as $P_n = A_n \oplus B_n$. Next P_n and C_n are added using the X-OR gate inside the second half adder to produce the final

sum bit and $S_n = P_n \oplus C_n$ where $P_n = A_n \oplus B_n$ and output carry $C_0 = P_n \cdot C_n = (A_n \oplus B_n) C_n$ which becomes carry for the (n+1) th stage.

Consider the case of both P_n and C_n being 1. The input carry C_n has to be propagated to the output only if P_n is 1. If P_n is 0, even if C_n is 1, the and gate in the second half-adder will inhibit C_n . the carry out of the nth stage is 1 when either $G_n=1$ or $P_n \cdot C_n=1$ or both G_n and $P_n \cdot C_n$ are equal to 1.

For the final sum and carry outputs of the nth stage, we get the following Boolean expressions.

$$S_n = P_n \oplus C_n \text{ where } P_n = A_n \oplus B_n$$

$$C_{out} = C_{n+1} = G_n + P_n C_n \text{ where } G_n = A_n \cdot B_n$$

Observe the recursive nature of the expression for the output carry at the nth stage which becomes the input carry for the (n+1)st stage .it is possible to express the output carry of a higher significant stage is the carry-out of the previous stage.

Based on these , the expression for the carry-outs of various full adders are as follows,

$$C_1 = G_0 + P_0 \cdot C_0$$

$$C_2 = G_1 + P_1 \cdot C_1 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$$

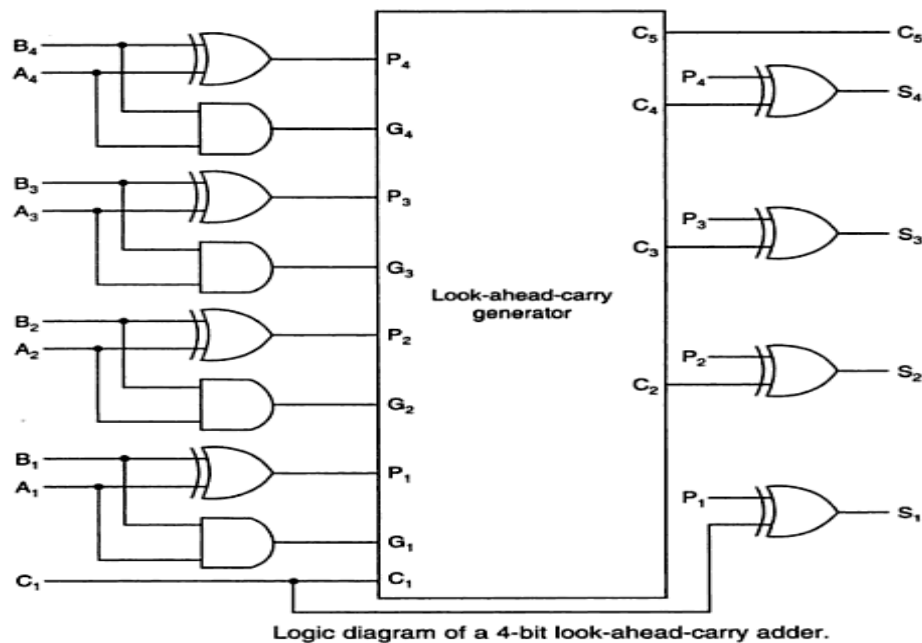
$$C_3 = G_2 + P_2 \cdot C_2 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

$$C_4 = G_3 + P_3 \cdot C_3 = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

The general expression for n stages designated as 0 through $(n - 1)$ would be

$$C_n = G_{n-1} + P_{n-1} \cdot C_{n-1} = G_{n-1} + P_{n-1} \cdot G_{n-2} + P_{n-1} \cdot P_{n-2} \cdot G_{n-3} + \dots + P_{n-1} \cdot \dots \cdot P_0 \cdot C_0$$

Observe that the final output carry is expressed as a function of the input variables in SOP form. Which is two level AND-OR or equivalent NAND-NAND form. Observe that the full look-ahead scheme requires the use of OR gate with (n+1) inputs and AND gates with number of inputs varying from 2 to (n+1).



2's complement Addition and Subtraction using Parallel Adders:

Most modern computers use the 2's complement system to represent negative numbers and to perform subtraction operations of signed numbers can be performed using only the addition operation, if we use the 2's complement form to represent negative numbers.

The circuit shown can perform both addition and subtraction in the 2's complement. This adder/subtractor circuit is controlled by the control signal ADD/SUB'. When the ADD/SUB' level is HIGH, the circuit performs the addition of the numbers stored in registers A and B. When the ADD/Sub' level is LOW, the circuit subtracts the number in register B from the number in register A. The operation is:

When ADD/SUB' is a 1:

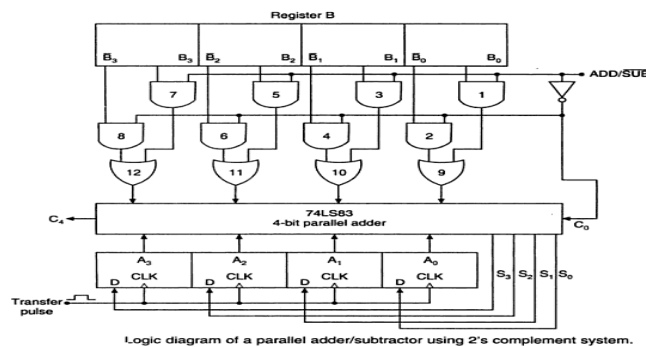
1. AND gates 1,3,5 and 7 are enabled, allowing B_0, B_1, B_2 and B_3 to pass to the OR gates 9,10,11,12. AND gates 2,4,6 and 8 are disabled, blocking $B_0', B_1', B_2',$ and B_3' from reaching the OR gates 9,10,11 and 12.
2. The two levels B_0 to B_3 pass through the OR gates to the 4-bit parallel adder, to be added to the bits A_0 to A_3 . The sum appears at the output S_0 to S_3 .
3. Add/SUB' = 1 causes no carry into the adder.

When ADD/SUB' is a 0:

1. AND gates 1,3,5 and 7 are disabled, allowing B_0, B_1, B_2 and B_3 from reaching the OR gates 9,10,11,12. AND gates 2,4,6 and 8 are enabled, blocking $B_0', B_1', B_2',$ and B_3' from reaching the OR gates.

- 2 The two levels B_0' to B_3' pass through the OR gates to the 4-bit parallel adder, to be added to the bits A_0 to A_3 . The C_0 is now 1. thus the number in register B is converted to its 2's complement form.
- 3 The difference appears at the output S_0 to S_3 .

Adders/Subtractors used for adding and subtracting signed binary numbers. In computers, the output is transferred into the register A (accumulator) so that the result of the addition or subtraction always ends up stored in the register A. This is accomplished by applying a transfer pulse to the CLK inputs of register A.



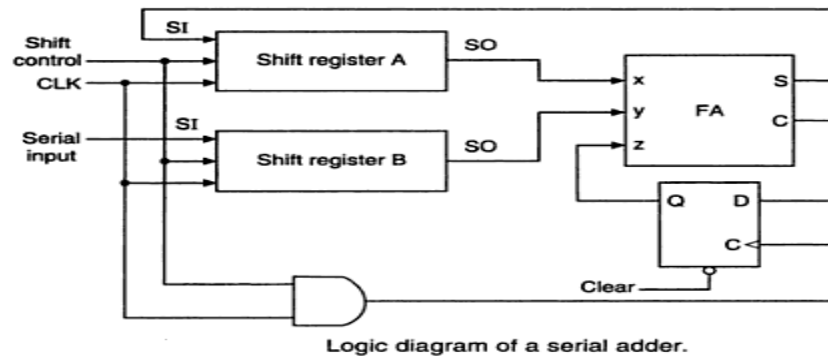
Serial Adder:

A serial adder is used to add binary numbers in serial form. The two binary numbers to be added serially are stored in two shift registers A and B. Bits are added one pair at a time through a single full adder (FA) circuit as shown. The carry out of the full-adder is transferred to a D flip-flop. The output of this flip-flop is then used as the carry input for the next pair of significant bits. The sum bit from the S output of the full-adder could be transferred to a third shift register. By shifting the sum into A while the bits of A are shifted out, it is possible to use one register for storing both augend and the sum bits. The serial input register B can be used to transfer a new binary number while the addend bits are shifted out during the addition.

The operation of the serial adder is:

Initially register A holds the augend, register B holds the addend and the carry flip-flop is cleared to 0. The outputs (SO) of A and B provide a pair of significant bits for the full-adder at x and y. The shift control enables both registers and carry flip-flop, so, at the clock pulse both registers are shifted once to the right, the sum bit from S enters the left most flip-flop of A, and the output carry is transferred into flip-flop Q. The shift control enables the registers for a number of clock pulses equal to the number of bits of the registers. For each succeeding clock pulse a new sum bit is transferred to A, a new carry is transferred to Q, and both registers are shifted once to the right. This process continues until the shift control is disabled. Thus the addition is accomplished by passing each pair of bits together with the previous carry through a single full adder circuit and transferring the sum, one bit at a time, into register A.

Initially, register A and the carry flip-flop are cleared to 0 and then the first number is added from B. While B is shifted through the full adder, a second number is transferred to it through its serial input. The second number is then added to the content of register A while a third number is transferred serially into register B. This can be repeated to form the addition of two, three, or more numbers and accumulate their sum in register A.



Difference between Serial and Parallel Adders:

The parallel adder registers with parallel load, whereas the serial adder uses shift registers. The number of full adder circuits in the parallel adder is equal to the number of bits in the binary numbers, whereas the serial adder requires only one full adder circuit and a carry flip-flop. Excluding the registers, the parallel adder is a combinational circuit, whereas the serial adder is a sequential circuit. The sequential circuit in the serial adder consists of a full-adder and a flip-flop that stores the output carry.

BCD Adder:

The BCD addition process:

1. Add the 4-bit BCD code groups for each decimal digit position using ordinary binary addition.
2. For those positions where the sum is 9 or less, the sum is in proper BCD form and no correction is needed.
3. When the sum of two digits is greater than 9, a correction of 0110 should be added to that sum, to produce the proper BCD result. This will produce a carry to be added to the next decimal position.

A BCD adder circuit must be able to operate in accordance with the above steps. In other words, the circuit must be able to do the following:

1. Add two 4-bit BCD code groups, using straight binary addition.

- Determine, if the sum of this addition is greater than 1101 (decimal 9); if it is , add 0110 (decimal 6) to this sum and generate a carry to the next decimal position.

The first requirement is easily met by using a 4- bit binary parallel adder such as the 74LS83 IC .For example , if the two BCD code groups $A_3A_2A_1A_0$ and $B_3B_2B_1B_0$ are applied to a 4-bit parallel adder, the adder will output $S_4S_3S_2S_1S_0$, where S_4 is actually C_4 , the carry –out of the MSB bits.

The sum outputs $S_4S_3S_2S_1S_0$ can range anywhere from 00000 to 100109 when both the BCD code groups are 1001=9). The circuitry for a BCD adder must include the logic needed to detect whenever the sum is greater than 01001, so that the correction can be added in. Those cases , where the sum is greater than 1001 are listed as:

S_4	S_3	S_2	S_1	S_0	Decimal number
0	1	0	1	0	10
0	1	0	1	1	11
0	1	1	0	0	12
0	1	1	0	1	13
0	1	1	1	0	14
0	1	1	1	1	15
1	0	0	0	0	16
1	0	0	0	1	17
1	0	0	1	0	18

Let us define a logic output X that will go HIGH only when the sum is greater than 01001 (i.e, for the cases in table). If examine these cases ,see that X will be HIGH for either of the following conditions:

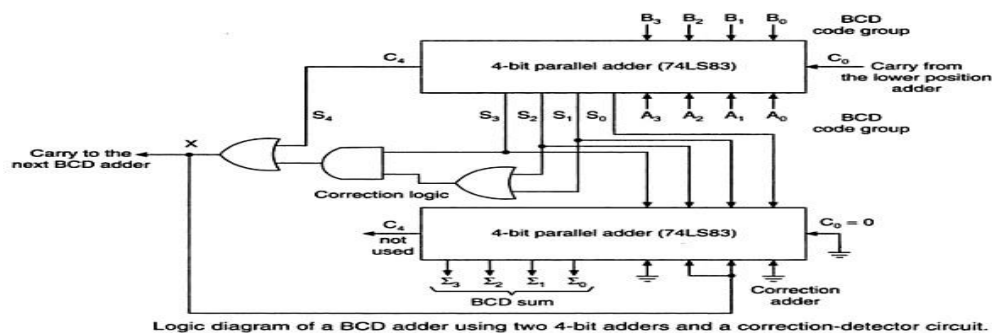
- Whenever $S_4 = 1$ (sum greater than 15)
- Whenever $S_3 = 1$ and either S_2 or S_1 or both are 1 (sum 10 to 15)

This condition can be expressed as

$$X = S_4 + S_3(S_2 + S_1)$$

Whenever $X=1$, it is necessary to add the correction factor 0110 to the sum bits, and to generate a carry. The circuit consists of three basic parts. The two BCD code groups $A_3A_2A_1A_0$ and $B_3B_2B_1B_0$ are added together in the upper 4-bit adder, to produce the sum $S_4S_3S_2S_1S_0$. The logic gates shown implement the expression for X. The lower 4-bit adder will add the correction 0110 to the sum bits, only when $X=1$, producing the final BCD sum output represented by $\sum_3\sum_2\sum_1\sum_0$. The X is also the carry-out that is produced when the sum is greater than 01001. When $X=0$, there is no carry and no addition of 0110. In such cases, $\sum_3\sum_2\sum_1\sum_0 = S_3S_2S_1S_0$.

Two or more BCD adders can be connected in cascade when two or more digit decimal numbers are to be added. The carry-out of the first BCD adder is connected as the carry-in of the second BCD adder, the carry-out of the second BCD adder is connected as the carry-in of the third BCD adder and so on.



EXCESS-3(XS-3) ADDER:

To perform Excess-3 additions,

1. Add two xs-3 code groups
2. If carry=1, add 0011(3) to the sum of those two code groups
If carry =0, subtract 0011(3) i.e., add 1101 (13 in decimal) to the sum of those two code groups.

Ex: Add 9 and 5

	1100	9 in Xs-3
	+1000	5 in xs-3

1	0100	there is a carry
+0011	0011	add 3 to each group
-----	-----	
0100	0111	14 in xs-3
(1)	(4)	

EX:

(b)	0 1 1 1	4 in XS-3
	+ 0 1 1 0	3 in XS-3

	1 1 0 1	no carry
	+ 1 1 0 1	Subtract 3 (i.e. add 13)

Ignore carry	1 1 0 1 0	7 in XS-3
	(7)	

Implementation of xs-3 adder using 4-bit binary adders is shown. The augend ($A_3A_2A_1A_0$) and addend ($B_3B_2B_1B_0$) in xs-3 are added using the 4-bit parallel adder. If the carry is a 1, then 0011(3) is added to the sum bits $S_3S_2S_1S_0$ of the upper adder in the lower 4-bit parallel

adder. If the carry is a 0, then 1101(3) is added to the sum bits (This is equivalent to subtracting 0011(3) from the sum bits. The correct sum in xs-3 is obtained

Excess-3 (XS-3) Subtractor:

To perform Excess-3 subtraction,

1. Complement the subtrahend
2. Add the complemented subtrahend to the minuend.
3. If carry =1, result is positive. Add 3 and end around carry to the result . If carry=0, the result is negative. Subtract 3, i.e, and take the 1's complement of the result.

Ex: Perform 9-4

	1100	9 in xs-3
	+1000	Complement of 4 n Xs-3

(1)	0100	There is a carry
	+0011	Add 0011(3)

	0111	
	1	End around carry

	1000	5 in xs-3

The minuend and the 1's complement of the subtrahend in xs-3 are added in the upper 4-bit parallel adder. If the carry-out from the upper adder is a 0, then 1101 is added to the sum bits of the upper adder in the lower adder and the sum bits of the lower adder are complemented to get the result. If the carry-out from the upper adder is a 1, then 3=0011 is added to the sum bits of the lower adder and the sum bits of the lower adder give the result.

Binary Multipliers:

In binary multiplication by the paper and pencil method, is modified somewhat in digital machines because a binary adder can add only two binary numbers at a time.

In a binary multiplier, instead of adding all the partial products at the end, they are added two at a time and their sum accumulated in a register (the accumulator register). In addition, when the multiplier bit is a 0, 0s are not written down and added because it does not affect the final result. Instead, the multiplicand is shifted left by one bit.

The multiplication of 1110 by 1001 using this process is

Multiplicand 1110

Multiplier 1001

1110

The LSB of the multiplier is a 1; write down the multiplicand; shift the multiplicand one position to the left (1 1 1 0 0)

1110

The second multiplier bit is a 0; write down the previous result 1110; shift the multiplicand to the left again (1 1 1 0 0 0)

+1110000

The fourth multiplier bit is a 1 write down the new multiplicand add it to the first partial product to obtain the final product.

1111110

This multiplication process can be performed by the serial multiplier circuit, which multiplies two 4-bit numbers to produce an 8-bit product. The circuit consists of following elements

X register: A 4-bit shift register that stores the multiplier --- it will shift right on the falling edge of the clock. Note that 0s are shifted in from the left.

B register: An 8-bit register that stores the multiplicand; it will shift left on the falling edge of the clock. Note that 0s are shifted in from the right.

A register: An 8-bit register, i.e., the accumulator that accumulates the partial products.

Adder: An 8-bit parallel adder that produces the sum of A and B registers. The adder outputs S_7 through S_0 are connected to the D inputs of the accumulator so that the sum can be transferred to the accumulator only when a clock pulse gets through the AND gate.

The circuit operation can be described by going through each step in the multiplication of 1110 by 1001. The complete process requires 4 clock cycles.

1. Before the first clock pulse: Prior to the occurrence of the first clock pulse, the register A is loaded with 00000000, the register B with the multiplicand 00001110, and the register X with the multiplier 1001. Assume that each of these registers is loaded using its asynchronous inputs (i.e., PRESET and CLEAR). The output of the adder will be the sum of A and B, i.e., 00001110.

2. First Clock pulse: Since the LSB of the multiplier (X_0) is a 1, the first clock pulse gets through the AND gate and its positive going transition transfers the sum outputs into the accumulator. The subsequent negative going transition causes the X and B registers to shift right and left, respectively. This produces a new sum of A and B.

3. Second Clock Pulse: The second bit of the original multiplier is now in X_0 . Since this bit is a 0, the second clock pulse is inhibited from reaching the accumulator. Thus, the sum outputs are not transferred into the accumulator and the number in the accumulator does not change. The negative going transition of the clock pulse will again shift the X and B registers. Again a new sum is produced.

4. Third Clock Pulse: The third bit of the original multiplier is now in X_0 ; since this bit is a 0, the third clock pulse is inhibited from reaching the accumulator. Thus, the sum outputs are not transferred into the accumulator and the number in the accumulator does not change. The negative going transition of the clock pulse will again shift the X and B registers. Again a new sum is produced.

5. Fourth Clock Pulse: The last bit of the original multiplier is now in X_0 , and since it is a 1, the positive going transition of the fourth pulse transfers the sum into the accumulator. The accumulator now holds the final product. The negative going transition of the clock pulse shifts X and B again. Note that, X is now 0000, since all the multiplier bits have been shifted out.

Code converters:

The availability of a large variety of codes for the same discrete elements of information results in the use of different codes by different digital systems. It is sometimes necessary to use the output of one system as the input to another. A conversion circuit must be inserted between the two systems if each uses different codes for the same information. Thus a

code converter is a logic circuit whose inputs are bit patterns representing numbers (or character) in one code and whose outputs are the corresponding representation in a different code. Code converters are usually multiple output circuits.

To convert from binary code A to binary code B, the input lines must supply the bit combination of elements as specified by code A and the output lines must generate the corresponding bit combination of code B. A combinational circuit performs this transformation by means of logic gates.

For example, a binary –to-gray code converter has four binary input lines B_4, B_3, B_2, B_1 and four gray code output lines G_4, G_3, G_2, G_1 . When the input is 0010, for instance, the output should be 0011 and so forth. To design a code converter, we use a code table treating it as a truth table to express each output as a Boolean algebraic function of all the inputs.

In this example, of binary –to-gray code conversion, we can treat the binary to the gray code table as four truth tables to derive expressions for G_4, G_3, G_2 , and G_1 . Each of these four expressions would, in general, contain all the four input variables B_4, B_3, B_2 , and B_1 . Thus, this code converter is actually equivalent to four logic circuits, one for each of the truth tables.

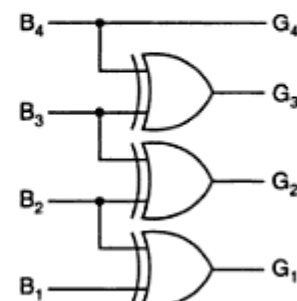
The logic expression derived for the code converter can be simplified using the usual techniques, including ‘don’t cares’ if present. Even if the input is an unweighted code, the same cell numbering method which we used earlier can be used, but the cell numbers --must correspond to the input combinations as if they were an 8-4-2-1 weighted code. s

Design of a 4-bit binary to gray code converter:

$$\begin{aligned} G_4 &= \sum m(8, 9, 10, 11, 12, 13, 14, 15) & G_4 &= B_4 \\ G_3 &= \sum m(4, 5, 6, 7, 8, 9, 10, 11) & G_3 &= \bar{B}_4 B_3 + B_4 \bar{B}_3 = B_4 \oplus B_3 \\ G_2 &= \sum m(2, 3, 4, 5, 10, 11, 12, 13) & G_2 &= \bar{B}_3 B_2 + B_3 \bar{B}_2 = B_3 \oplus B_2 \\ G_1 &= \sum m(1, 2, 5, 6, 9, 10, 13, 14) & G_1 &= \bar{B}_2 B_1 + B_2 \bar{B}_1 = B_2 \oplus B_1 \end{aligned}$$

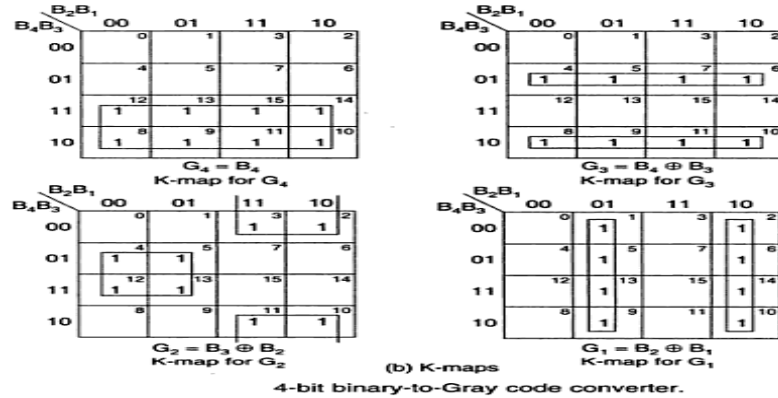
4-bit binary				4-bit Gray			
B_4	B_3	B_2	B_1	G_4	G_3	G_2	G_1
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

(a) Conversion table



(c) Logic diagram

4-bit binary-to-Gray code converter



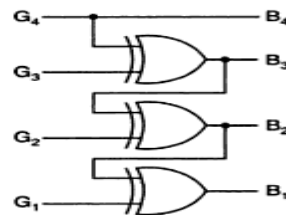
Design of a 4-bit gray to Binary code converter:

$$\begin{aligned}
 B_4 &= \Sigma m(12, 13, 15, 14, 10, 11, 9, 8) = \Sigma m(8, 9, 10, 11, 12, 13, 14, 15) \\
 B_3 &= \Sigma m(6, 7, 5, 4, 10, 11, 9, 8) = \Sigma m(4, 5, 6, 7, 8, 9, 10, 11) \\
 B_2 &= \Sigma m(3, 2, 5, 4, 15, 14, 9, 8) = \Sigma m(2, 3, 4, 5, 8, 9, 14, 15) \\
 B_1 &= \Sigma m(1, 2, 7, 4, 13, 14, 11, 8) = \Sigma m(1, 2, 4, 7, 8, 11, 13, 14)
 \end{aligned}$$

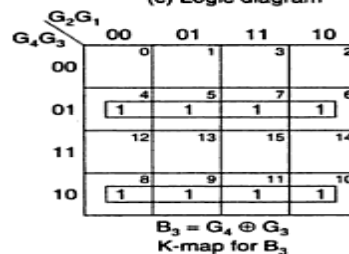
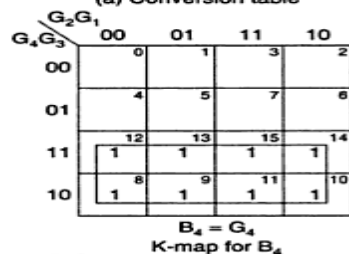
$$\begin{aligned}
 B_4 &= G_4 \\
 B_3 &= \bar{G}_4 G_3 + G_4 \bar{G}_3 = G_4 \oplus G_3 \\
 B_2 &= \bar{G}_4 G_3 \bar{G}_2 + \bar{G}_4 \bar{G}_3 G_2 + G_4 \bar{G}_3 \bar{G}_2 + G_4 G_3 G_2 \\
 &= \bar{G}_4 (G_3 \oplus G_2) + G_4 (\bar{G}_3 \oplus \bar{G}_2) = G_4 \oplus G_3 \oplus G_2 = B_3 \oplus G_2 \\
 B_1 &= \bar{G}_4 \bar{G}_3 \bar{G}_2 G_1 + \bar{G}_4 \bar{G}_3 G_2 \bar{G}_1 + \bar{G}_4 G_3 G_2 G_1 + \bar{G}_4 G_3 \bar{G}_2 \bar{G}_1 + G_4 G_3 \bar{G}_2 G_1 \\
 &\quad + G_4 G_3 G_2 \bar{G}_1 + G_4 \bar{G}_3 G_2 G_1 + G_4 \bar{G}_3 \bar{G}_2 \bar{G}_1 \\
 &= \bar{G}_4 \bar{G}_3 (G_2 \oplus G_1) + G_4 G_3 (G_2 \oplus G_1) + \bar{G}_4 G_3 (\bar{G}_2 \oplus \bar{G}_1) + G_4 \bar{G}_3 (\bar{G}_2 \oplus \bar{G}_1) \\
 &= (G_2 \oplus G_1) (\bar{G}_4 \oplus G_3) + (\bar{G}_2 \oplus \bar{G}_1) (G_4 \oplus G_3) \\
 &= G_4 \oplus G_3 \oplus G_2 \oplus G_1
 \end{aligned}$$

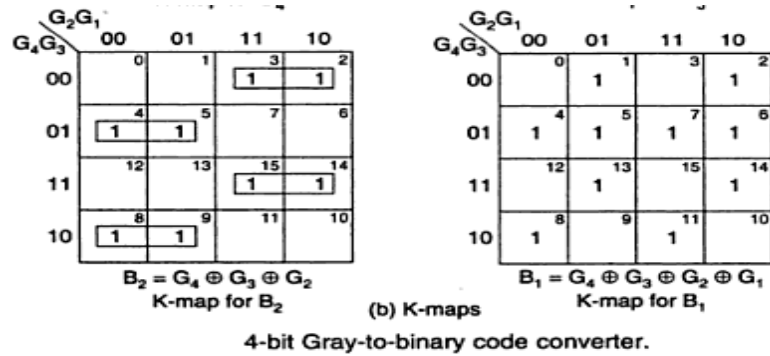
4-bit Gray				4-bit binary			
G_4	G_3	G_2	G_1	B_4	B_3	B_2	B_1
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
0	1	0	1	0	1	1	0
0	1	0	0	0	1	1	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	1	1	0	1	0
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	0	0	1	1	1	1	0
1	0	0	0	1	1	1	1

(a) Conversion table



(c) Logic diagram





Design of a 4-bit BCD to XS-3 code converter:

8421 code				XS-3 code			
B_4	B_3	B_2	B_1	X_4	X_3	X_2	X_1
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

(a) Conversion table

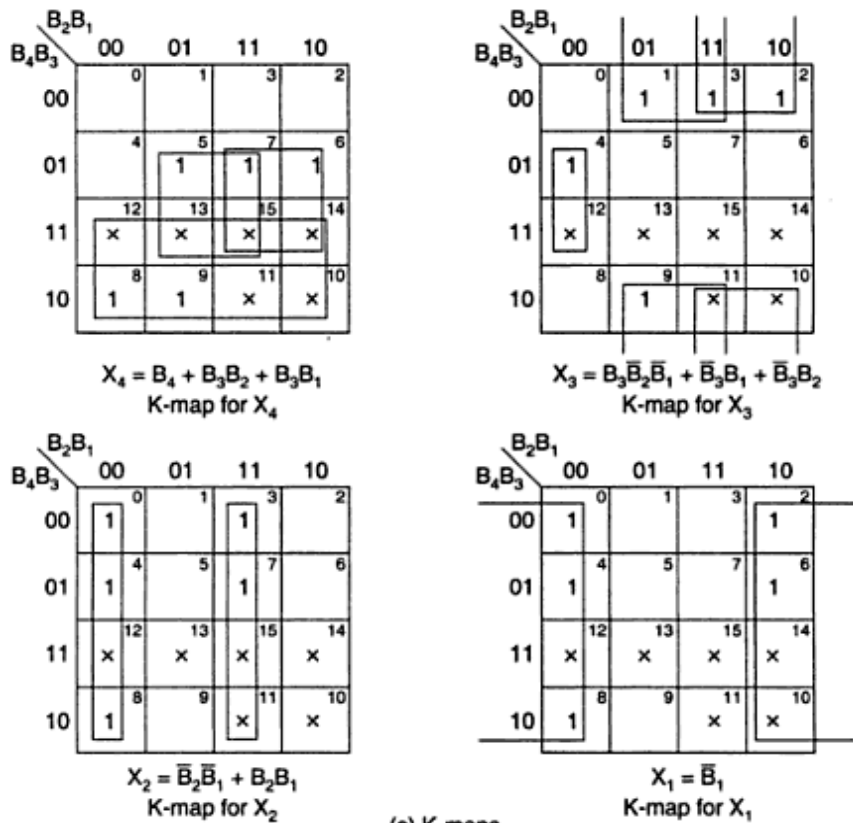
$$\begin{aligned}
 X_4 &= \sum m(5, 6, 7, 8, 9) + d(10, 11, 12, 13, 14, 15) \\
 X_3 &= \sum m(1, 2, 3, 4, 9) + d(10, 11, 12, 13, 14, 15) \\
 X_2 &= \sum m(0, 3, 4, 7, 8) + d(10, 11, 12, 13, 14, 15) \\
 X_1 &= \sum m(0, 2, 4, 6, 8) + d(10, 11, 12, 13, 14, 15)
 \end{aligned}$$

The minimal expressions are

$$\begin{aligned}
 X_4 &= B_4 + B_3B_2 + B_3B_1 \\
 X_3 &= B_3\bar{B}_2\bar{B}_1 + \bar{B}_3B_1 + \bar{B}_3B_2 \\
 X_2 &= \bar{B}_2\bar{B}_1 + B_2B_1 \\
 X_1 &= \bar{B}_1
 \end{aligned}$$

(b) Minimal expressions

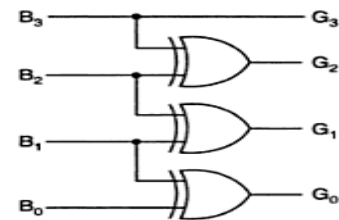
4-bit BCD-to-XS-3 code converter



Design of a BCD to gray code converter:

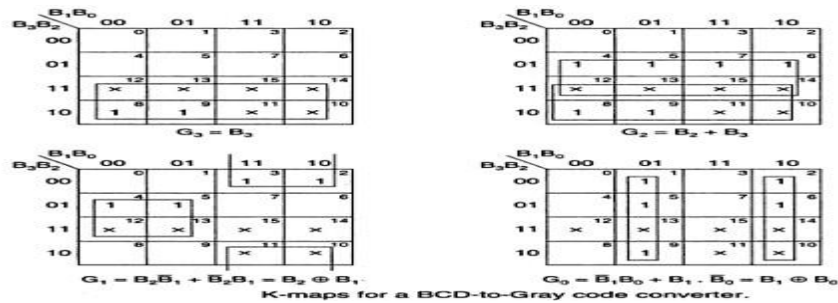
BCD code				Gray code			
B ₃	B ₂	B ₁	B ₀	G ₃	G ₂	G ₁	G ₀
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1

(a) BCD-to-Gray code conversion table



(b) Logic diagram

BCD-to-Gray code converter.

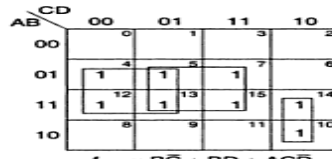


K-maps for a BCD-to-Gray code converter.

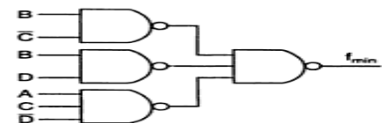
Design of a SOP circuit to Detect the Decimal numbers 5 through 12 in a 4-bit gray code Input:

Decimal number	4-bit Gray code				Output f
	A	B	C	D	
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	1	0
3	0	0	1	0	0
4	0	1	1	0	0
5	0	1	1	1	1
6	1	1	0	1	1
7	0	1	0	0	1
8	1	1	0	0	1
9	1	1	0	1	1
10	1	1	1	1	1
11	1	1	1	0	1
12	1	0	1	0	1
13	1	0	1	1	0
14	1	0	0	1	0
15	1	0	0	0	0

(a) Truth table



(b) K-map



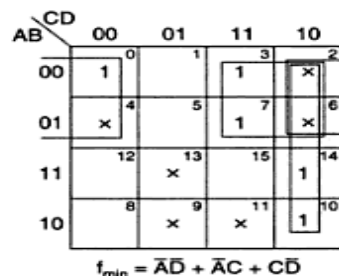
(c) NAND logic

Truth table, K-map and logic diagram for the SOP circuit.

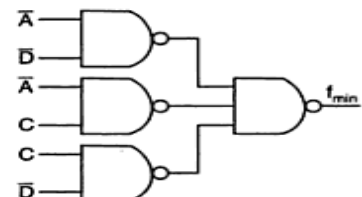
Design of a SOP circuit to detect the decimal numbers 0,2,4,6,8 in a 4-bit 5211 BCD code input:

Decimal number	5211 code				Output f
	A	B	C	D	
0	0	0	0	0	1
1	0	0	0	1	0
2	0	0	1	1	1
3	0	1	0	1	0
4	0	1	1	1	1
5	1	0	0	0	0
6	1	0	1	0	1
7	1	1	0	0	0
8	1	1	1	0	1
9	1	1	1	1	0

(a) Truth table



(b) K-map



(c) Logic diagram

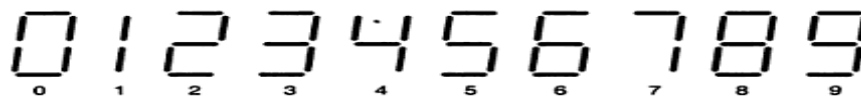
Truth table, K-map and logic diagram for the SOP circuit.

Design of a Combinational circuit to produce the 2's complement of a 4-bit binary number:

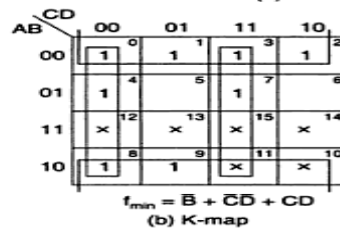
Input				Output			
A	B	C	D	E	F	G	H
0	0	0	0	0	0	0	0
0	0	0	1	1	1	1	1
0	0	1	0	1	1	1	0
0	0	1	1	1	1	0	1
0	1	0	0	1	1	0	0
0	1	0	1	1	0	1	1
0	1	1	0	1	0	1	0
0	1	1	1	1	0	0	1
1	0	0	0	1	0	0	0
1	0	0	1	0	1	1	1
1	0	1	0	0	1	1	0
1	0	1	1	0	1	0	1
1	1	0	0	0	1	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	0	1	0
1	1	1	1	0	0	0	1

(a) Conversion table

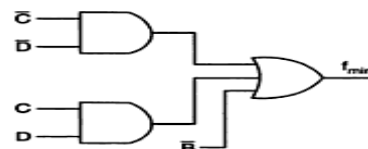
Conversion table and K-maps for the circuit



(a) Seven-segment display



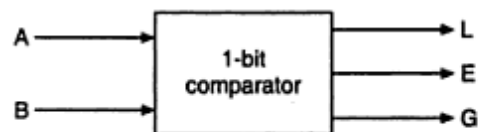
(b) K-map



(c) Logic diagram

Comparators:

$$\text{EQUALITY} = (A_3 \odot B_3)(A_2 \odot B_2)(A_1 \odot B_1)(A_0 \odot B_0)$$



Block diagram of a 1-bit comparator.

1. Magnitude Comparator:

The logic for a 1-bit magnitude comparator: Let the 1-bit numbers be $A = A_0$ and $B = B_0$.

If $A_0 = 1$ and $B_0 = 0$, then $A > B$.

Therefore,

$$A > B: G = A_0 \bar{B}_0$$

If $A_0 = 0$ and $B_0 = 1$, then $A < B$.

Therefore,

$$A < B: L = \bar{A}_0 B_0$$

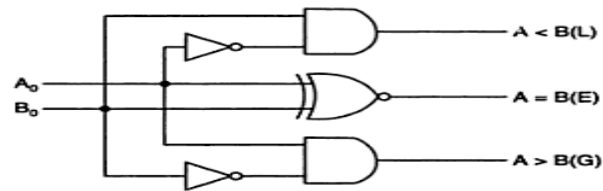
If A_0 and B_0 coincide, i.e. $A_0 = B_0 = 0$ or if $A_0 = B_0 = 1$, then $A = B$.

Therefore,

$$A = B: E = A_0 \odot B_0$$

A_0	B_0	L	E	G
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

(a) Truth table



(b) Logic diagram
1-bit comparator.

1-bit Magnitude Comparator:

The logic for a 2-bit magnitude comparator: Let the two 2-bit numbers be $A = A_1 A_0$ and $B = B_1 B_0$.

1. If $A_1 = 1$ and $B_1 = 0$, then $A > B$ or

2. If A_1 and B_1 coincide and $A_0 = 1$ and $B_0 = 0$, then $A > B$. So the logic expression for $A > B$ is

$$A > B: G = A_1 \bar{B}_1 + (A_1 \odot B_1) A_0 \bar{B}_0$$

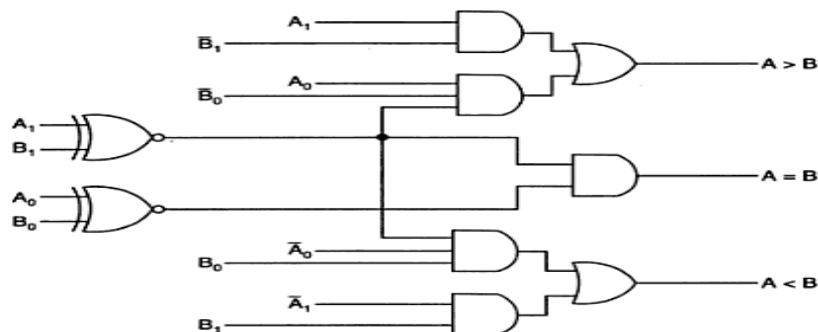
1. If $A_1 = 0$ and $B_1 = 1$, then $A < B$ or

2. If A_1 and B_1 coincide and $A_0 = 0$ and $B_0 = 1$, then $A < B$. So the expression for $A < B$ is

$$A < B: L = \bar{A}_1 B_1 + (A_1 \odot B_1) \bar{A}_0 B_0$$

If A_1 and B_1 coincide and if A_0 and B_0 coincide then $A = B$. So the expression for $A = B$ is

$$A = B: E = (A_1 \odot B_1)(A_0 \odot B_0)$$



Logic diagram of a 2-bit magnitude comparator.

4-Bit Magnitude Comparator:

The logic for a 4-bit magnitude comparator: Let the two 4-bit numbers be $A = A_3A_2A_1A_0$ and $B = B_3B_2B_1B_0$.

1. If $A_3 = 1$ and $B_3 = 0$, then $A > B$. Or
2. If A_3 and B_3 coincide, and if $A_2 = 1$ and $B_2 = 0$, then $A > B$. Or
3. If A_3 and B_3 coincide, and if A_2 and B_2 coincide, and if $A_1 = 1$ and $B_1 = 0$, then $A > B$. Or
4. If A_3 and B_3 coincide, and if A_2 and B_2 coincide, and if A_1 and B_1 coincide, and if $A_0 = 1$ and $B_0 = 0$, then $A > B$.

From these statements, we see that the logic expression for $A > B$ can be written as

$$(A > B) = A_3\bar{B}_3 + (A_3 \odot B_3)A_2\bar{B}_2 + (A_3 \odot B_3)(A_2 \odot B_2)A_1\bar{B}_1 + (A_3 \odot B_3)(A_2 \odot B_2)(A_1 \odot B_1)A_0\bar{B}_0$$

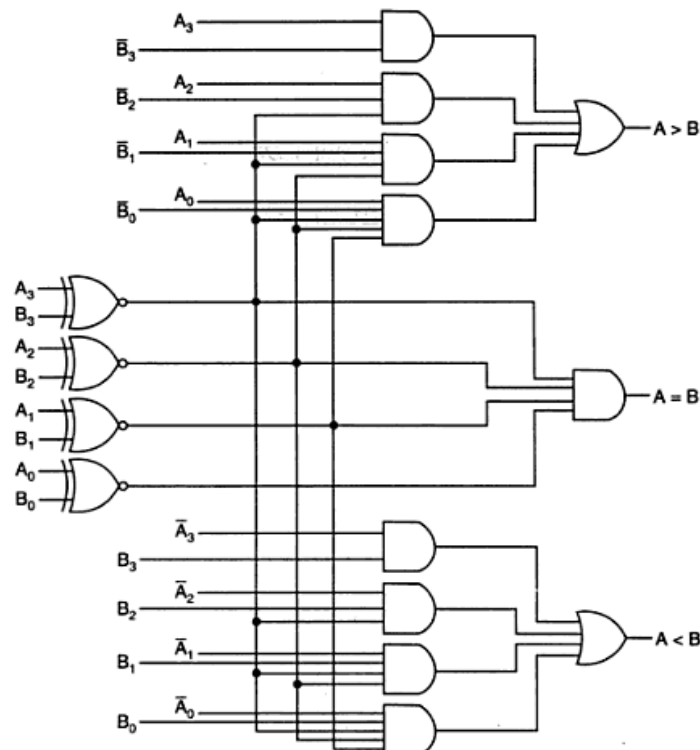
Similarly, the logic expression for $A < B$ can be written as

$$A < B = \bar{A}_3B_3 + (A_3 \odot B_3)\bar{A}_2B_2 + (A_3 \odot B_3)(A_2 \odot B_2)\bar{A}_1B_1 + (A_3 \odot B_3)(A_2 \odot B_2)(A_1 \odot B_1)\bar{A}_0B_0$$

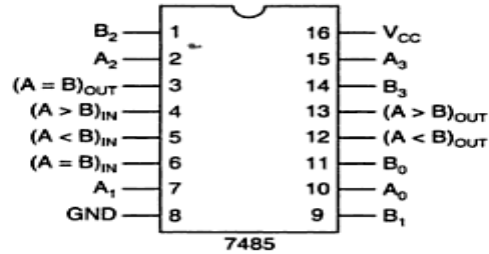
If A_3 and B_3 coincide and if A_2 and B_2 coincide and if A_1 and B_1 coincide and if A_0 and B_0 coincide, then $A = B$.

So the expression for $A = B$ can be written as

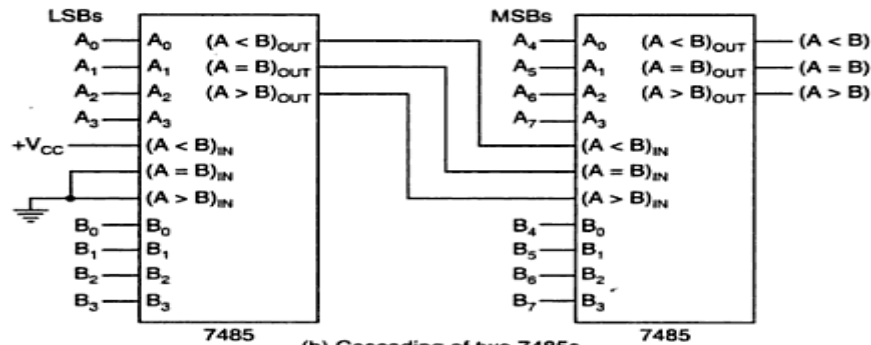
$$(A = B) = (A_3 \odot B_3)(A_2 \odot B_2)(A_1 \odot B_1)(A_0 \odot B_0)$$



IC Comparator:



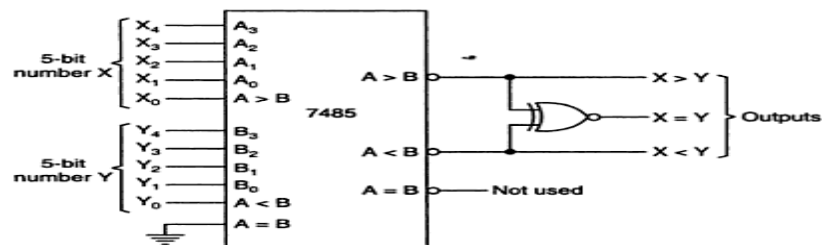
(a) Pin diagram of 7485



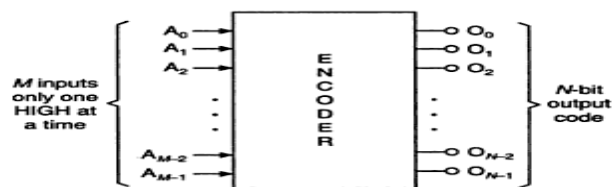
(b) Cascading of two 7485s

Pin diagram and cascading of 7485 4-bit comparators.

ENCODERS:



Use of 7485 as a 5-bit comparator.

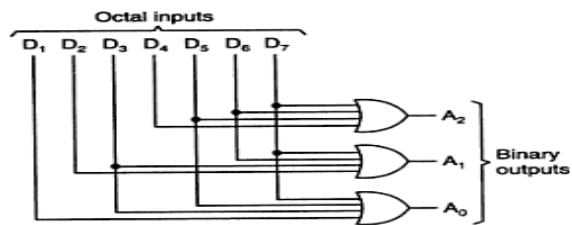


Block diagram of encoder.

Octal to Binary Encoder:

Octal digits		Binary		
		A ₂	A ₁	A ₀
D ₀	0	0	0	0
D ₁	1	0	0	1
D ₂	2	0	1	0
D ₃	3	0	1	1
D ₄	4	1	0	0
D ₅	5	1	0	1
D ₆	6	1	1	0
D ₇	7	1	1	1

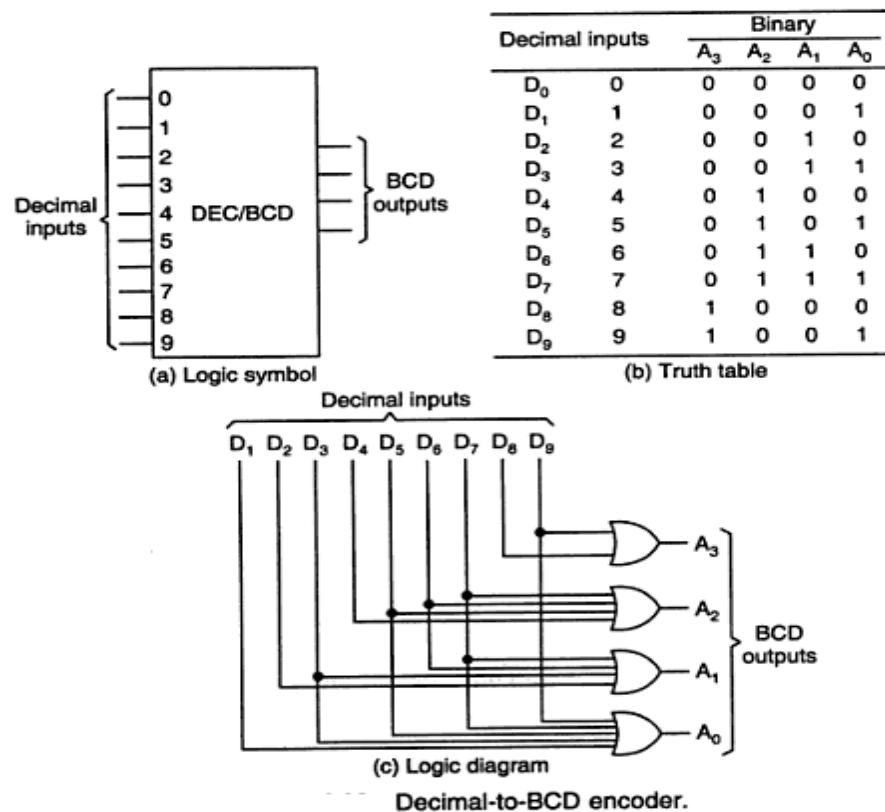
(a) Truth table



(b) Logic diagram

Octal-to-binary encoder.

Decimal to BCD Encoder:

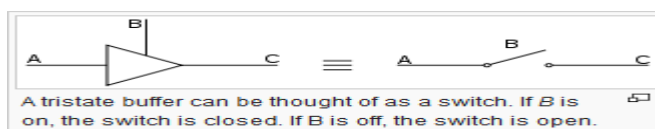


Tristate bus system:

In digital electronics **three-state**, **tri-state**, or **3-state** logic allows an output port to assume a high impedance state in addition to the 0 and 1 logic levels, effectively removing the output from the circuit.

This allows multiple circuits to share the same output line or lines (such as a bus which cannot listen to more than one device at a time).

Three-state outputs are implemented in many registers, bus drivers, and flip-flops in the 7400 and 4000 series as well as in other types, but also internally in many integrated circuits. Other typical uses are internal and external buses in microprocessors, computer memory, and peripherals. Many devices are controlled by an active-low input called OE (Output Enable) which dictates whether the outputs should be held in a high-impedance state or drive their respective loads (to either 0- or 1-level).



INPUT		OUTPUT
A	B	C
0		0
1	1	1
X	0	Z (high impedance)

Unit III

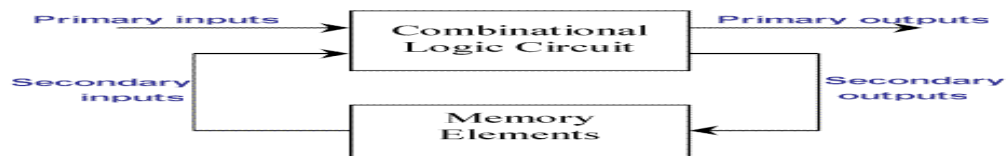
Sequential machine fundamentals

Sequential circuits

Classification of sequential circuits: Sequential circuits may be classified as two types.

1. Synchronous sequential circuits
2. Asynchronous sequential circuits

Combinational logic refers to circuits whose output is strictly depended on the present value of the inputs. As soon as inputs are changed, the information about the previous inputs is lost, that is, combinational logics circuits have no memory. Although every digital system is likely to have combinational circuits, most systems encountered in practice also include memory elements, which require that the system be described in terms of sequential logic. Circuits whose output depends not only on the present input value but also the past input value are known as **sequential logic circuits**. The mathematical model of a sequential circuit is usually referred to as a **sequential machine**.



Comparison between combinational and sequential circuits

Combinational circuit	Sequential circuit
1. In combinational circuits, the output variables at any instant of time are dependent only on the present input variables 2. memory unit is not required in combinational circuit 3. these circuits are faster because the delay between the i/p and o/p is due to propagation delay of gates only 4. easy to design	1. in sequential circuits the output variables at any instant of time are dependent not only on the present input variables, but also on the present state 2. memory unit is required to store the past history of the input variables 3. sequential circuits are slower than combinational circuits 4. comparatively hard to design

Level mode and pulse mode asynchronous sequential circuits:

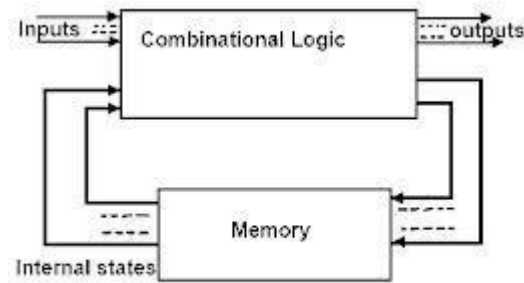


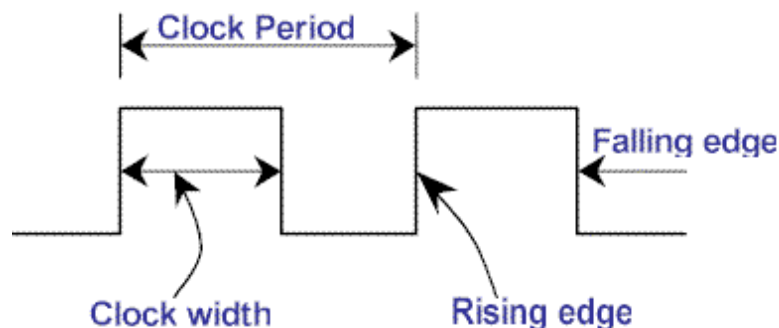
Figure 1: Asynchronous Sequential Circuit

Fig shows a block diagram of an asynchronous sequential circuit. It consists of a combinational circuit and delay elements connected to form the feedback loops. The present state and next state variables in asynchronous sequential circuits called secondary variables and excitation variables respectively..

There are two types of asynchronous circuits: fundamental mode circuits and pulse mode circuits.

Synchronous and Asynchronous Operation:

Sequential circuits are divided into two main types: **synchronous** and **asynchronous**. Their classification depends on the timing of their signals. **Synchronous** sequential circuits change their states and output values at discrete instants of time, which are specified by the rising and falling edge of a free-running **clock signal**. The clock signal is generally some form of square wave as shown in Figure below.



From the diagram you can see that the **clock period** is the time between successive transitions in the same direction, that is, between two rising or two falling edges. State transitions in synchronous sequential circuits are made to take place at times when the clock is making a transition from 0 to 1 (rising edge) or from 1 to 0 (falling edge). Between successive clock pulses there is no change in the information stored in memory.

The reciprocal of the clock period is referred to as the **clock frequency**. The **clock width** is defined as the time during which the value of the clock signal is equal to 1. The ratio of the clock width and clock period is referred to as the duty cycle. A clock signal is said to

be **active high** if the state changes occur at the clock's rising edge or during the clock width. Otherwise, the clock is said to be **active low**. Synchronous sequential circuits are also known as **clocked sequential circuits**.

The memory elements used in synchronous sequential circuits are usually flip-flops. These circuits are binary cells capable of storing one bit of information. A flip-flop circuit has two outputs, one for the normal value and one for the complement value of the bit stored in it. Binary information can enter a flip-flop in a variety of ways, a fact which give rise to the different types of flip-flops. For information on the different types of basic flip-flop circuits and their logical properties, see the previous tutorial on flip-flops.

In **asynchronous** sequential circuits, the transition from one state to another is initiated by the change in the primary inputs; there is no external synchronization. The memory commonly used in asynchronous sequential circuits are time-delayed devices, usually implemented by feedback among logic gates. Thus, asynchronous sequential circuits may be regarded as combinational circuits with feedback. Because of the feedback among logic gates, asynchronous sequential circuits may, at times, become unstable due to transient conditions. The instability problem imposes many difficulties on the designer. Hence, they are not as commonly used as synchronous systems.

Fundamental Mode Circuits assumes that:

1. The input variables change only when the circuit is stable
2. Only one input variable can change at a given time
3. Inputs are levels are not pulses

A pulse mode circuit assumes that:

1. The input variables are pulses instead of levels
2. The width of the pulses is long enough for the circuit to respond to the input
3. The pulse width must not be so long that is still present after the new state is reached.

Latches and flip-flops

Latches and flip-flops are the basic elements for storing information. One latch or flip-flop can store one bit of information. The main difference between latches and flip-flops is that for latches, their outputs are constantly affected by their inputs as long as the enable signal is asserted. In other words, when they are enabled, their content changes immediately when their inputs change. Flip-flops, on the other hand, have their content change only either at the rising or falling edge of the enable signal. This enable signal is usually the controlling clock signal. After the rising or falling edge of the clock, the flip-flop content remains constant even if the input changes.

There are basically four main types of latches and flip-flops: SR, D, JK, and T. The major differences in these flip-flop types are the number of inputs they have and how they change state. For each type, there are also different variations that enhance their operations. In this chapter, we

will look at the operations of the various latches and flip-flops. the flip-flops has two outputs, labeled Q and Q'. the Q output is the normal output of the flip flop and Q' is the inverted output.

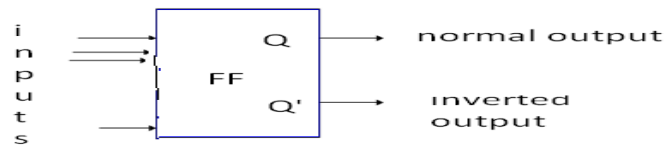


Figure: basic symbol of flipflop

A latch may be an active-high input latch or an active –LOW input latch. active –HIGH means that the SET and RESET inputs are normally resting in the low state and one of them will be pulsed high whenever we want to change latch outputs.

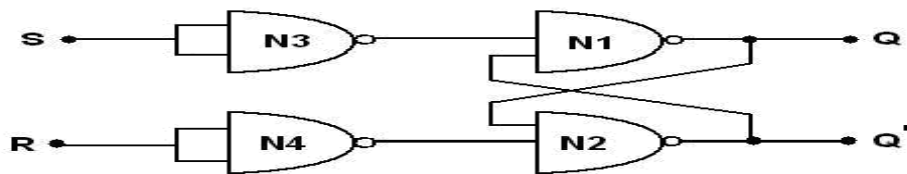
SR latch:

The latch has two outputs Q and Q'. When the circuit is switched on the latch may enter into any state. If Q=1, then Q'=0, which is called SET state. If Q=0, then Q'=1, which is called RESET state. Whether the latch is in SET state or RESET state, it will continue to remain in the same state, as long as the power is not switched off. But the latch is not an useful circuit, since there is no way of entering the desired input. It is the fundamental building block in constructing flip-flops, as explained in the following sections

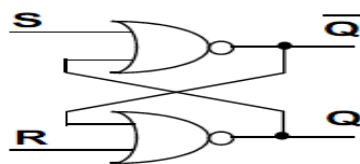
NAND latch

NAND latch is the fundamental building block in constructing a flip-flop. It has the property of holding on to any previous output, as long as it is not disturbed.

The operation of NAND latch is the reverse of the operation of NOR latch. if 0's are replaced by 1's and 1's are replaced by 0's we get the same truth table as that of the NOR latch shown



NOR latch



S	R	Q	$\bar{Q}$	Function
0	0	Q^+	$\bar{Q}^+$	Storage State
0	1	0	1	Reset
1	0	1	0	Set
1	1	0-?	0-?	Indeterminate State

The analysis of the operation of the active-HIGHNOR latch can be summarized as follows.

1. SET=0, RESET=0: this is normal resting state of the NOR latch and it has no effect on the output state. Q and Q' will remain in whatever state they were prior to the occurrence of this input condition.
2. SET=1, RESET=0: this will always set Q=1, where it will remain even after SET returns to 0
3. SET=0, RESET=1: this will always reset Q=0, where it will remain even after RESET returns to 0
4. SET=1,RESET=1; this condition tries to SET and RESET the latch at the same time, and it produces Q=Q'=0. If the inputs are returned to zero simultaneously, the resulting output state is erratic and unpredictable. This input condition should not be used.

The SET and RESET inputs are normally in the LOW state and one of them will be pulsed HIGH. Whenever we want to change the latch outputs..

RS Flip-flop:

The basic flip-flop is a one bit memory cell that gives the fundamental idea of memory device. It constructed using two NAND gates. The two NAND gates N1 and N2 are connected such that, output of N1 is connected to input of N2 and output of N2 to input of N1. These form the feedback path the inputs are S and R, and outputs are Q and Q'. The logic diagram and the block diagram of R-S flip-flop with clocked input

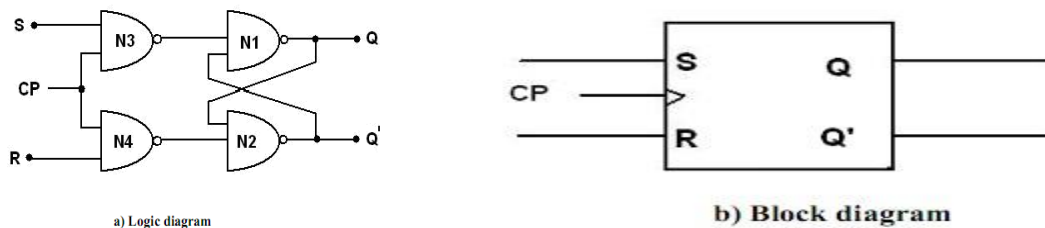


Figure: RS Flip-flop

The flip-flop can be made to respond only during the occurrence of clock pulse by adding two NAND gates to the input latch. So synchronization is achieved. i.e., flip-flops are allowed to change their states only at particular instant of time. The clock pulses are generated by a clock pulse generator. The flip-flops are affected only with the arrival of clock pulse.

Operation:

1. When CP=0 the output of N3 and N4 are 1 regardless of the value of S and R. This is given as input to N1 and N2. This makes the previous value of Q and Q' unchanged.
2. When CP=1 the information at S and R inputs are allowed to reach the latch and change of state in flip-flop takes place.
3. CP=1, S=1, R=0 gives the SET state i.e., Q=1, Q'=0.

4. CP=1, S=0, R=1 gives the RESET state i.e., Q=0, Q'=1.
5. CP=1, S=0, R=0 does not affect the state of flip-flop.
6. CP=1, S=1, R=1 is not allowed, because it is not able to determine the next state. This condition is said to be a -race condition.

In the logic symbol CP input is marked with a triangle. It indicates the circuit responds to an input change from 0 to 1. The characteristic table gives the operation conditions of flip-flop. $Q(t)$ is the present state maintained in the flip-flop at time t . $Q(t+1)$ is the state after the occurrence of clock pulse.

Truth table

S	R	$Q_{(t+1)}$	Comments
0	0	Q_t	No change
0	1	0	Reset / clear
1	0	1	Set
1	1	*	Not allowed

Edge triggered RS flip-flop:

Some flip-flops have an RC circuit at the input next to the clock pulse. By the design of the circuit the R-C time constant is much smaller than the width of the clock pulse. So the output changes will occur only at specific level of clock pulse. The capacitor gets fully charged when clock pulse goes from low to high. This change produces a narrow positive spike. Later at the trailing edge it produces narrow negative spike. This operation is called edge triggering, as the flip-flop responds only at the changing state of clock pulse. If output transition occurs at rising edge of clock pulse (0 $\rightarrow$ 1), it is called positively edge triggering. If it occurs at trailing edge (1 $\rightarrow$ 0) it is called negative edge triggering. Figure shows the logic and block diagram.

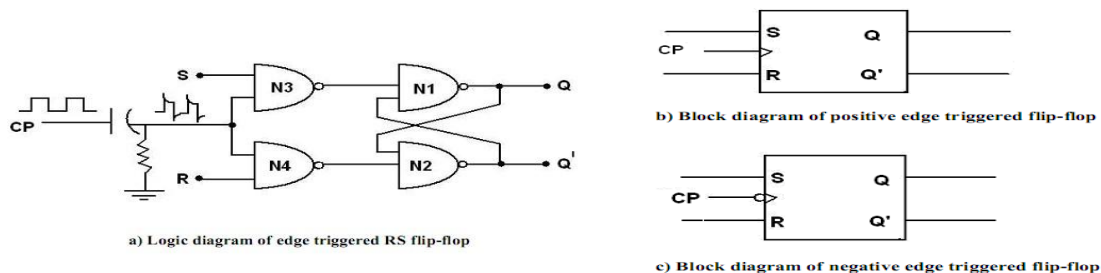
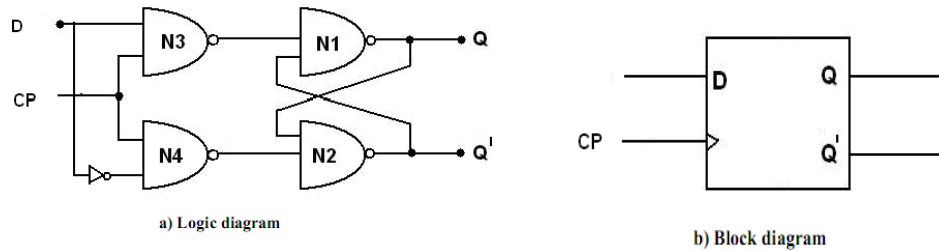


Figure: Edge triggered RS flip-flop

D flip-flop:

The D flip-flop is the modified form of R-S flip-flop. R-S flip-flop is converted to D flip-flop by adding an inverter between S and R and only one input D is taken instead of S and R. So one input is D and complement of D is given as another input. The logic diagram and the block diagram of D flip-flop with clocked input

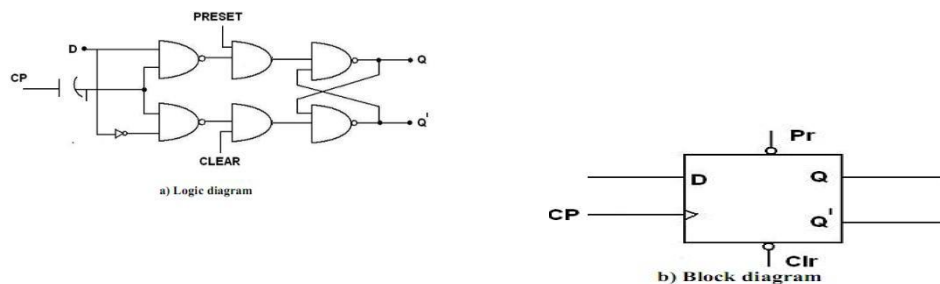


When the clock is low both the NAND gates (N1 and N2) are disabled and Q retains its last value. When clock is high both the gates are enabled and the input value at D is transferred to its output Q. D flip-flop is also called -Data flip-flop.

Truth table

CP	D	Q
0	x	Previous state
1	0	0
1	1	1

Edge Triggered D Flip-flop:



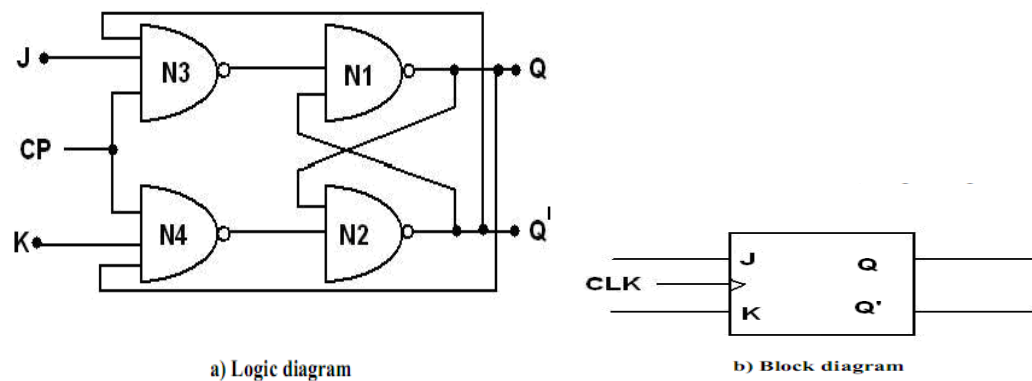
Truth table

PRESET	CLEAR	CP	D	Q
0	0	X	X	*(forbidden)
0	1	X	X	1
1	0	X	X	0
1	1	0	X	NC
1	1	1	X	NC
1	1	↓	X	NC
1	1	↑	0	0
1	1	↑	1	1

Figure: truth table, block diagram, logic diagram of edge triggered flip-flop

JK flip-flop (edge triggered JK flip-flop)

The race condition in RS flip-flop, when R=S=1 is eliminated in J-K flip-flop. There is a feedback from the output to the inputs. Figure 3.4 represents one way of building a JK flip-flop.



Truth table

J	K	$Q_{(t+1)}$	Comments
0	0	Q_t	No change
0	1	0	Reset / clear
1	0	1	Set
1	1	Q'_t	Complement/ toggle.

Figure: JK flip-flop

The J and K are called control inputs, because they determine what the flip-flop does when a positive clock edge arrives.

Operation:

1. When $J=0$, $K=0$ then both N3 and N4 will produce high output and the previous value of Q and Q' retained as it is.

2. When $J=0$, $K=1$, N3 will get an output as 1 and output of N4 depends on the value of Q. The final output is $Q=0$, $Q'=1$ i.e., reset state

3. When $J=1$, $K=0$ the output of N4 is 1 and N3 depends on the value of Q'. The final output is $Q=1$ and $Q'=0$ i.e., set state

4. When $J=1$, $K=1$ it is possible to set (or) reset the flip-flop depending on the current state of output. If $Q=1$, $Q'=0$ then N4 passes '0' to N2 which produces $Q'=1$, $Q=0$ which is reset state. When $J=1$, $K=1$, Q changes to the complement of the last state. The flip-flop is said to be in the toggle state.

The characteristic equation of the JK flip-flop is:

$$Q_{next} = J\bar{Q} + \bar{K}Q$$

JK flip-flop operation ^[28]									
<u>Characteristic table</u>				<u>Excitation table</u>					
J	K	Q _{next}	Comment	Q	Q _{next}	J	K	Comment	
0	0	Q	hold state	0	0	0	X	No change	
0	1	0	reset	0	1	1	X	Set	
1	0	1	set	1	0	X	1	Reset	
1	1	Q	toggle	1	1	X	0	No change	

T flip-flop:

If the T input is high, the T flip-flop changes state ("toggles") whenever the clock input is strobed. If the T input is low, the flip-flop holds the previous value. This behavior is described by the characteristic equation

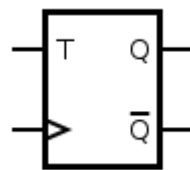


Figure : symbol for T flip flop

$$Q_{next} = T \oplus Q = T\bar{Q} + \bar{T}Q \text{ (expanding the XOR operator)}$$

When T is held high, the toggle flip-flop divides the clock frequency by two; that is, if clock frequency is 4 MHz, the output frequency obtained from the flip-flop will be 2 MHz. This "divide by" feature has application in various types of digital counters. A T flip-flop can also be built using a JK flip-flop (J & K pins are connected together and act as T) or D flip-flop (T input and P_{previous} is connected to the D input through an XOR gate).

T flip-flop operation ^[28]							
<u>Characteristic table</u>				<u>Excitation table</u>			
T	Q	Q_{next}	Comment	Q	Q_{next}	T	Comment
0	0	0	hold state (no clk)	0	0	0	No change
0	1	1	hold state (no clk)	1	1	0	No change
1	0	1	toggle	0	1	1	Complement
1	1	0	toggle	1	0	1	Complement

Flip flop operating characteristics:

The operation characteristics specify the performance, operating requirements, and operating limitations of the circuits. The operation characteristics mentioned here apply to all flip-flops regardless of the particular form of the circuit.

Propagation Delay Time: is the interval of time required after an input signal has been applied for the resulting output change to occur.

Set-up Time: is the minimum interval required for the logic levels to be maintained constantly on the inputs (J and K, or S and R, or D) prior to the triggering edge of the clock pulse in order for the levels to be reliably clocked into the flip-flop.

Hold Time: is the minimum interval required for the logic levels to remain on the inputs after the triggering edge of the clock pulse in order for the levels to be reliably clocked into the flip-flop.

Maximum Clock Frequency: is the highest rate that a flip-flop can be reliably triggered.

Power Dissipation: is the total power consumption of the device. It is equal to product of supply voltage (V_{cc}) and the current (I_{cc}).

$$P = V_{cc} \cdot I_{cc}$$

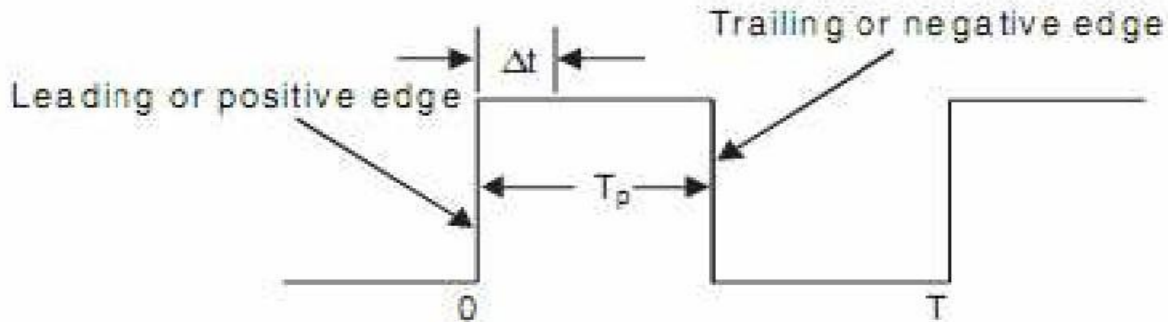
The power dissipation of a flip flop is usually in mW.

Pulse Widths: are the minimum pulse widths specified by the manufacturer for the Clock, SET and CLEAR inputs.

Clock transition times: for reliable triggering, the clock waveform transition times should be kept very short. If the clock signal takes too long to make the transitions from one level to other, the flip flop may either triggering erratically or not trigger at all.

Race around Condition

The inherent difficulty of an S-R flip-flop (i.e., $S = R = 1$) is eliminated by using the feedback connections from the outputs to the inputs of gate 1 and gate 2 as shown in Figure. Truth tables in figure were formed with the assumption that the inputs do not change during the clock pulse ($CLK = 1$). But the consideration is not true because of the feedback connections



- Consider, for example, that the inputs are $J = K = 1$ and $Q = 1$, and a pulse as shown in Figure is applied at the clock input.
- After a time interval t equal to the propagation delay through two NAND gates in series, the outputs will change to $Q = 0$. So now we have $J = K = 1$ and $Q = 0$.
- After another time interval of t the output will change back to $Q = 1$. Hence, we conclude that for the time duration of t_p of the clock pulse, the output will oscillate between 0 and 1. Hence, at the end of the clock pulse, the value of the output is not certain. This situation is referred to as a race-around condition.
- Generally, the propagation delay of TTL gates is of the order of nanoseconds. So if the clock pulse is of the order of microseconds, then the output will change thousands of times within the clock pulse.
- This race-around condition can be avoided if $t_p < t < T$. Due to the small propagation delay of the ICs it may be difficult to satisfy the above condition.
- A more practical way to avoid the problem is to use the master-slave (M-S) configuration as discussed below.

Applications of flip-flops:

Frequency Division: When a pulse waveform is applied to the clock input of a J-K flip-flop that is connected to toggle, the Q output is a square wave with half the frequency of the clock input. If more flip-flops are connected together as shown in the figure below, further division of the clock frequency can be achieved

Parallel data storage: a group of flip-flops is called register. To store data of N bits, N flip-flops are required. Since the data is available in parallel form. When a clock pulse is applied to all flip-flops simultaneously, these bits will transfer will be transferred to the Q outputs of the flip flops.

Serial data storage: to store data of N bits available in serial form, N number of D-flip-flops is connected in cascade. The clock signal is connected to all the flip-flops. The serial data is applied to the D input terminal of the first flip-flop.

Transfer of data: data stored in flip-flops may be transferred out in a serial fashion, i.e., bit-by-bit from the output of one flip-flops or may be transferred out in parallel form.

Excitation Tables:

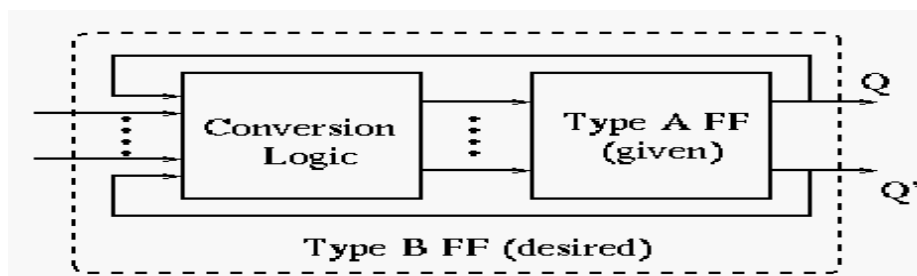
Previous State -> Present State	D
0 -> 0	0
0 -> 1	1
1 -> 0	0
1 -> 1	1

Previous State -> Present State	J	K
0 -> 0	0	X
0 -> 1	1	X
1 -> 0	X	1
1 -> 1	X	0

Previous State -> Present State	S	R
0 -> 0	0	X
0 -> 1	1	0
1 -> 0	0	1
1 -> 1	X	0

Previous State -> Present State	T
0 -> 0	0
0 -> 1	1
1 -> 0	1
1 -> 1	0

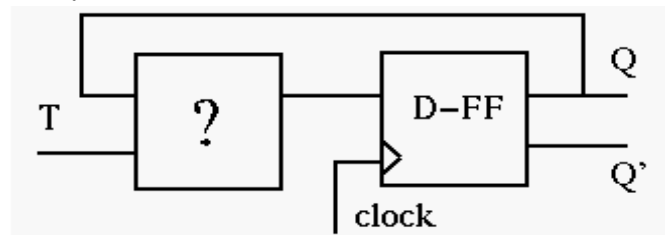
Conversions of flip-flops:



The key here is to use the excitation table, which shows the necessary triggering signal (S,R,J,K, D and T) for a desired flip-flop state transition :

Q_t	Q_{t+1}	S	R	J	K	D	T
0	0	0	x	0	x	0	0
0	1	1	0	1	x	1	1
1	0	0	1	x	1	0	1
1	1	x	0	x	0	1	0

Convert a D-FF to a T-FF:



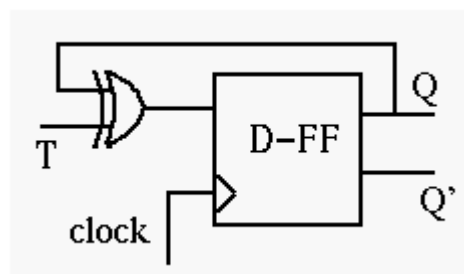
We need to design the circuit to generate the triggering signal D as a function of T and Q:
 . Consider the excitation table:

$$D = f(T, Q).$$

Q_t	Q_{t+1}	T	D
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	1

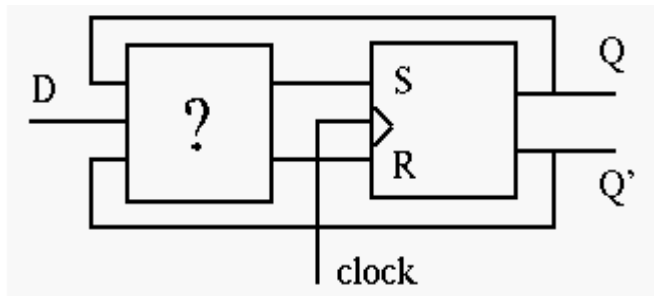
Treating as a function of and current FF state , we have

$$D = T'Q + TQ' = T \oplus Q$$



Convert a RS-FF to a D-FF:

We need to design the circuit to generate the triggering signals S and R as functions of and consider the excitation table:



Q_t	Q_{t+1}	D	S	R
0	0	0	0	x
0	1	1	1	0
1	0	0	0	1
1	1	1	x	0

The desired signal and can be obtained as functions of and current FF state from the Karnaugh maps:

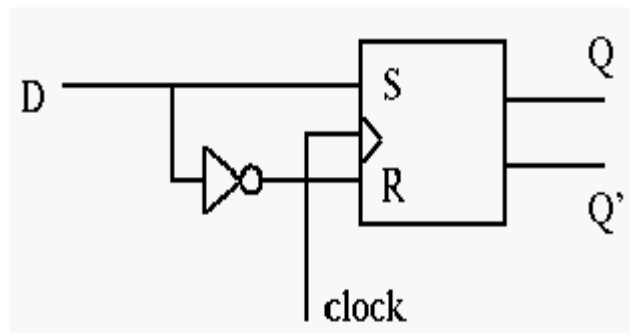
		Q	0	1
D	0	0	0	0
	1	1	X	

$$S=D$$

		Q	0	1
D	0	X	1	
	1	0	0	

$$R=D'$$

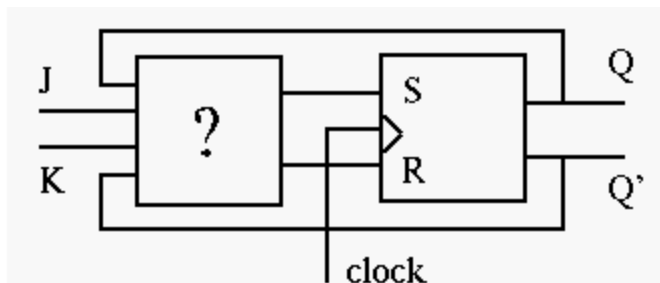
$$S = D, \quad R = D'$$



Convert a RS-FF to a JK-FF:

We need to design the circuit to generate the triggering signals S and R as functions of, J, K.

Consider the excitation table: The desired signal and as functions of, and current FF state can be obtained from the Karnaugh maps:



Q_t	Q_{t+1}	J	K	S	R
0	0	0	x	0	x
0	1	1	x	1	0
1	0	x	1	0	1
1	1	x	0	x	0

K-maps:

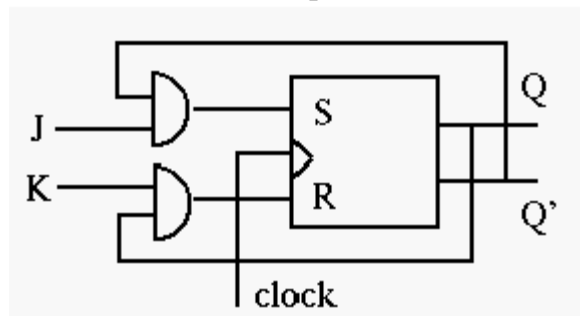
K \ QJ	00 01 11 10			
	0	1	X	X
0	0	1	X	X
1	0	1	0	0

$$S = Q'J$$

K \ QJ	00 01 11 10			
	0	0	0	0
0	X	0	0	0
1	X	0	1	1

$$R = QK$$

$$S = Q'J, \quad R = QK$$



The Master-Slave JK Flip-flop:

The Master-Slave Flip-Flop is basically two gated SR flip-flops connected together in a series configuration with the slave having an inverted clock pulse. The outputs from Q and Q from the "Slave" flip-flop are fed back to the inputs of the "Master" with the outputs of the "Master" flip-flop being connected to the two inputs of the "Slave" flip-flop. This feedback configuration from the slave's output to the master's input gives the characteristic toggle of the JK flip-flop as shown below.

The input signals J and K are connected to the gated "master" SR flip-flop which "locks" the input condition while the clock (Clk) input is "HIGH" at logic level "1". As the clock input of the "slave" flip-flop is the inverse (complement) of the "master" clock input, the "slave" SR flip-flop does not toggle. The outputs from the "master" flip-flop are only "seen" by the gated "slave" flip-flop when the clock input goes "LOW" to logic level "0". When the clock is "LOW", the outputs from the "master" flip-flop are latched and any additional changes to its inputs are ignored. The gated "slave" flip-flop now responds to the state of its inputs passed over by the "master" section. Then on the "Low-to-High" transition of the clock pulse the inputs of the "master" flip-flop are fed through to the gated inputs of the "slave" flip-flop and on the "High-to-Low" transition the same inputs are reflected on the output of the "slave" making this type of flip-flop edge or pulse-triggered. Then, the circuit accepts input data when the clock signal is "HIGH", and passes the data to the output on the falling-edge of the clock signal. In other words, the Master-Slave JK Flip-flop is a "Synchronous" device as it only passes data with the timing of the clock signal.

UNIT 4

Sequential circuit design and analysis

Sequential Circuit Design

- Steps in the design process for sequential circuits
 - State Diagrams and State Tables
 - Examples
 - Steps in Design of a Sequential Circuit
1. Specification – A description of the sequential circuit. Should include a detailing of the inputs, the outputs, and the operation. Possibly assumes that you have knowledge of digital system basics.
 2. Formulation: Generate a state diagram and/or a state table from the statement of the problem.
 3. State Assignment: From a state table assign binary codes to the states.
 4. Flip-flop Input Equation Generation: Select the type of flip-flop for the circuit and generate the needed input for the required state transitions
 5. Output Equation Generation: Derive output logic equations for generation of the output from the inputs and current state.
 6. Optimization: Optimize the input and output equations. Today, CAD systems are typically used for this in real systems.
 7. Technology Mapping: Generate a logic diagram of the circuit using ANDs, ORs, Inverters, and F/Fs.
 8. Verification: Use a HDL to verify the design.

Mealy and Moore

- Sequential machines are typically classified as either a Mealy machine or a Moore machine implementation.
- Moore machine: The outputs of the circuit depend only upon the current state of the circuit.
- Mealy machine: The outputs of the circuit depend upon both the current state of the circuit and the inputs.

An example to go through the steps

The specification: The circuit will have one input, X, and one output, Z. The output Z will be 0 except when the input sequence 1101 are the last 4 inputs received on X. In that case it will be a 1

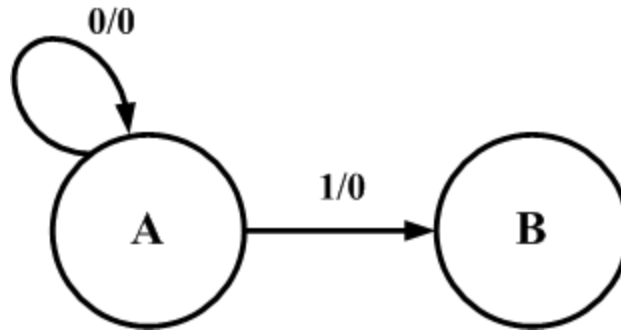
Generation of a state diagram

- Create states and meaning for them.

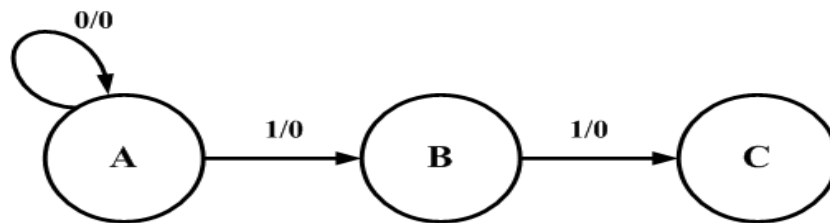
State A – the last input was a 0 and previous inputs unknown. Can also be the reset state.

State B – the last input was a 1 and the previous input was a 0. The start of a new sequence possibly.

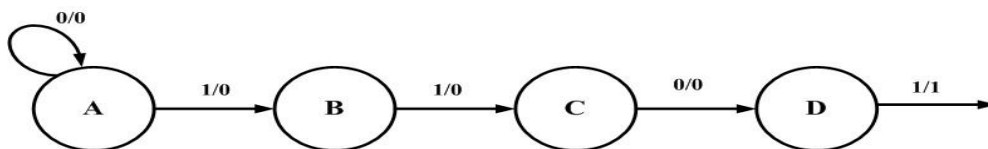
- Capture this in a state diagram



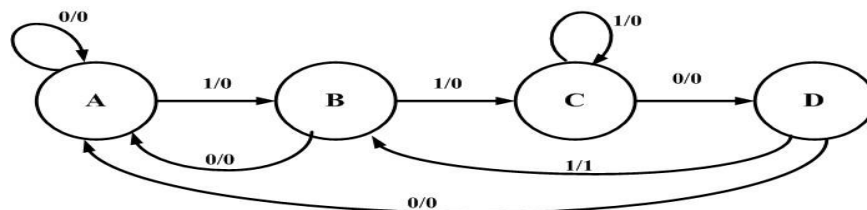
- ☐ Capture this in a state diagram
- ☐ Circles represent the states
- ☐ Lines and arcs represent the transition between states.
- ☐ The notation Input/output on the line or arc specifies the input that causes this transition and the output for this change of state.
- Add a state C – Have detected the input sequence 11 which is the start of the sequence



- ☐ Add a state D
- State D – have detected the 3rd input in the start of a sequence, a 0, now having 110. From State D, if the next input is a 1 the sequence has been detected and a 1 is output.



- ☐ The previous diagram was incomplete.
- ☐ In each state the next input could be a 0 or a 1. This must be included



- The state table
- This can be done directly from the state diagram

Present State	Next State		Output	
	X=0	X=1	X=0	X=1
A	A	B	0	0
B	A	C	0	0
C	D	C	0	0
D	A	B	0	1

- Now need to do a state assignment

Select a state assignment

- Will select a gray encoding
- For this state A will be encoded 00, state B 01, state C 11 and state D 10

Present State	Next State		Output	
	X=0	X=1	X=0	X=1
00	00	01	0	0
01	00	11	0	0
11	10	11	0	0
10	00	01	0	1

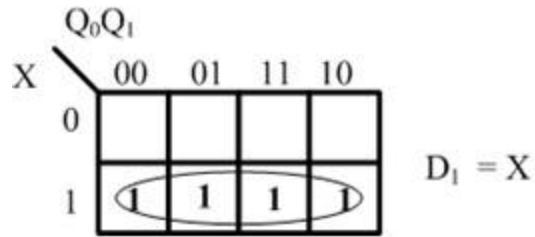
Flip-flop input equations

- Generate the equations for the flip-flop inputs
- Generate the D_0 equation

		Q_0Q_1			
		00	01	11	10
X	0			1	
	1		1	1	

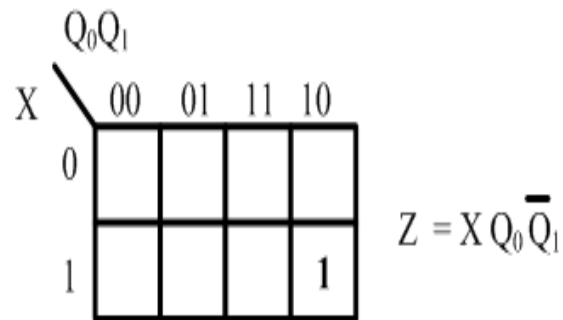
$$D_0 = Q_0 Q_1 + X Q_1$$

- Generate the D_1 equation



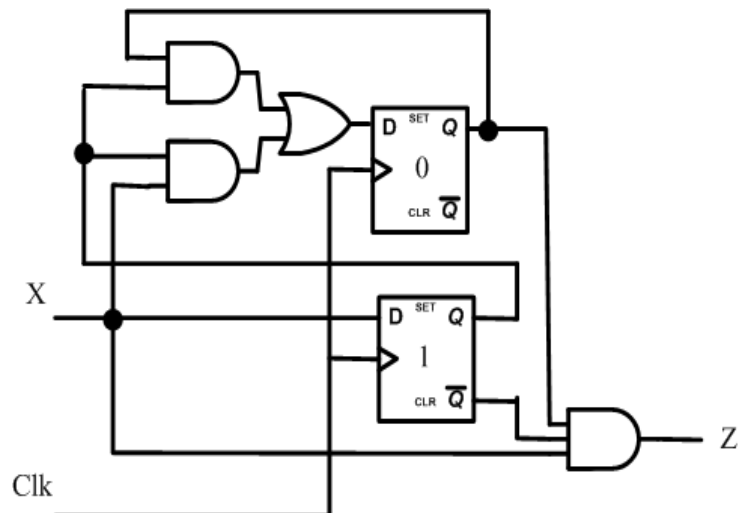
The output equation

- The next step is to generate the equation for the output Z and what is needed to generate it.
- Create a K-map from the truth table.



Now map to a circuit

- The circuit has 2 D type F/Fs



Shift registers:

In digital circuits, a **shift register** is a cascade of flip-flops sharing the same clock, in which the output of each flip-flop is connected to the "data" input of the next flip-flop in the chain, resulting in a circuit that shifts by one position the "bit array" stored in it, *shifting in* the data present at its input and *shifting out* the last bit in the array, at each transition of the clock input. More generally, a **shift register** may be multidimensional, such that its "data in" and stage outputs are themselves bit arrays: this is implemented simply by running several shift registers of the same bit-length in parallel.

Shift registers can have both parallel and serial inputs and outputs. These are often configured as **serial-in, parallel-out** (SIPO) or as **parallel-in, serial-out** (PISO). There are also types that have both serial and parallel input and types with serial and parallel output. There are also **bi-directional** shift registers which allow shifting in both directions: L→R or R→L. The serial input and last output of a shift register can also be connected to create a **circular shift register**

Shift registers are a type of logic circuits closely related to counters. They are basically for the storage and transfer of digital data.

Buffer register:

The buffer register is the simple set of registers. It simply stores the binary word. The buffer may be controlled buffer. Most of the buffer registers used D Flip-flops.

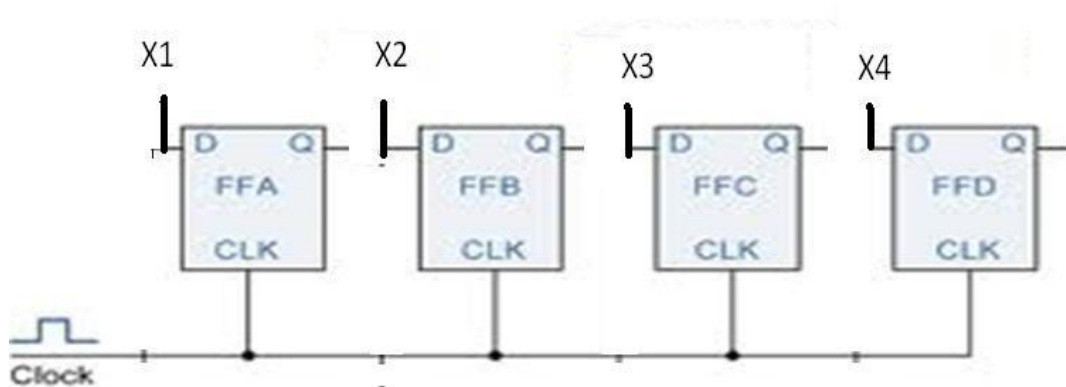


Figure: logic diagram of 4-bit buffer register

The figure shows a 4-bit buffer register. The binary word to be stored is applied to the data terminals. On the application of clock pulse, the output word becomes the same as the word applied at the terminals. i.e., the input word is loaded into the register by the application of clock pulse.

When the positive clock edge arrives, the stored word becomes:

$$Q_4Q_3Q_2Q_1 = X_4X_3X_2X_1$$
$$Q = X$$

Controlled buffer register:

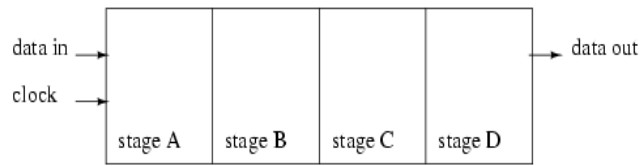
If *CLR* goes LOW, all the FFs are RESET and the output becomes, $Q = 0000$.

When *CLR* is HIGH, the register is ready for action. *LOAD* is the control input. When *LOAD* is HIGH, the data bits *X* can reach the D inputs of FF's.

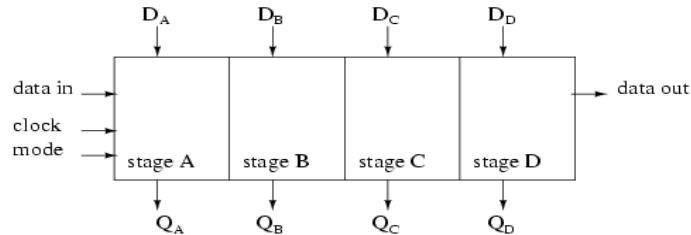
$$Q_4Q_3Q_2Q_1 = X_4X_3X_2X_1$$
$$Q = X$$

When load is low, the *X* bits cannot reach the FF's.

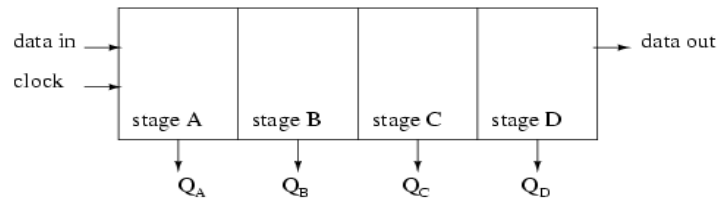
Data transmission in shift registers:



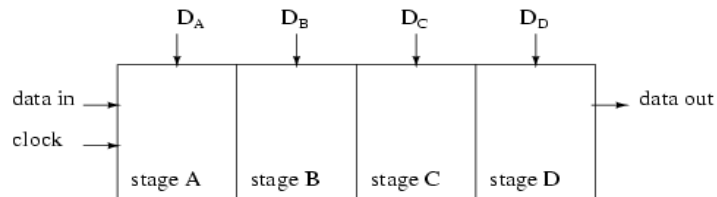
Serial-in, serial-out shift register with 4-stages



Parallel-in, parallel-out shift register with 4-stages



Serial-in, parallel-out shift register with 4-stages



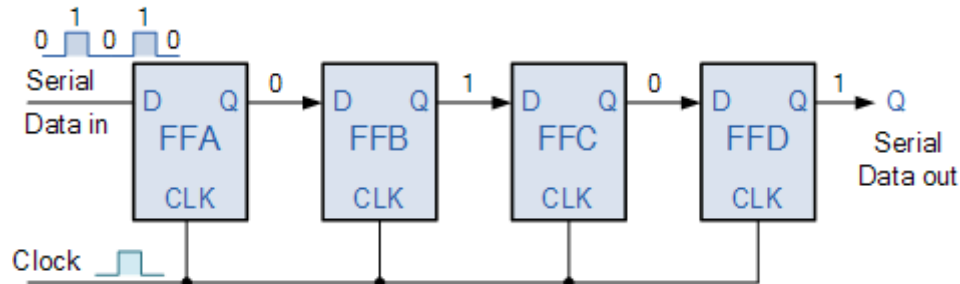
Parallel-in, serial-out shift register with 4-stages

A number of ff's connected together such that data may be shifted into and shifted out of them is called shift register. data may be shifted into or out of the register in serial form or in parallel form. There are four basic types of shift registers.

1. Serial in, serial out, shift right, shift registers
2. Serial in, serial out, shift left, shift registers
3. Parallel in, serial out shift registers
4. Parallel in, parallel out shift registers

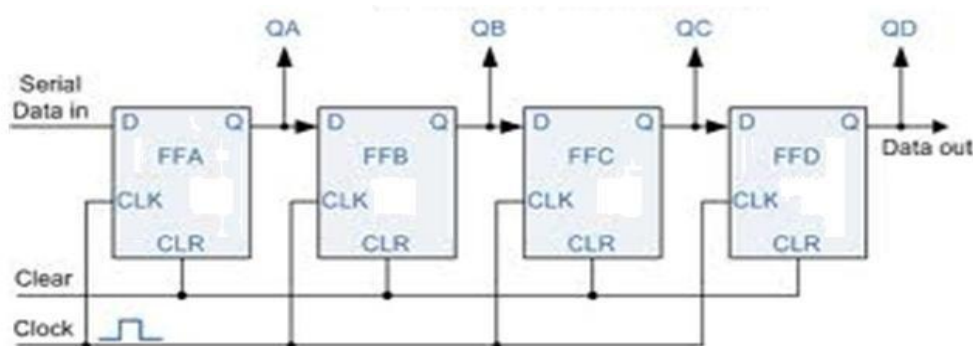
Serial IN, serial OUT, shift right, shift left register:

The logic diagram of 4-bit serial in serial out, right shift register with four stages. The register can store four bits of data. Serial data is applied at the input D of the first FF. the Q output of the first FF is connected to the D input of another FF. the data is outputted from the Q terminal of the last FF.



When serial data is transferred into a register, each new bit is clocked into the first FF at the positive going edge of each clock pulse. The bit that was previously stored by the first FF is transferred to the second FF. the bit that was stored by the Second FF is transferred to the third FF.

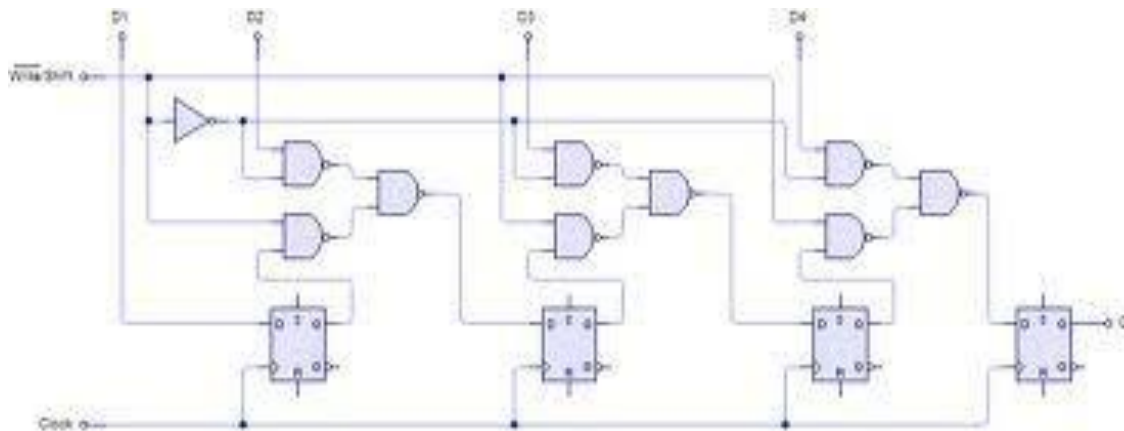
Serial-in, parallel-out, shift register:



In this type of register, the data bits are entered into the register serially, but the data stored in the register is shifted out in parallel form.

Once the data bits are stored, each bit appears on its respective output line and all bits are available simultaneously, rather than on a bit-by-bit basis with the serial output. The serial-in, parallel out, shift register can be used as serial-in, serial out, shift register if the output is taken from the Q terminal of the last FF.

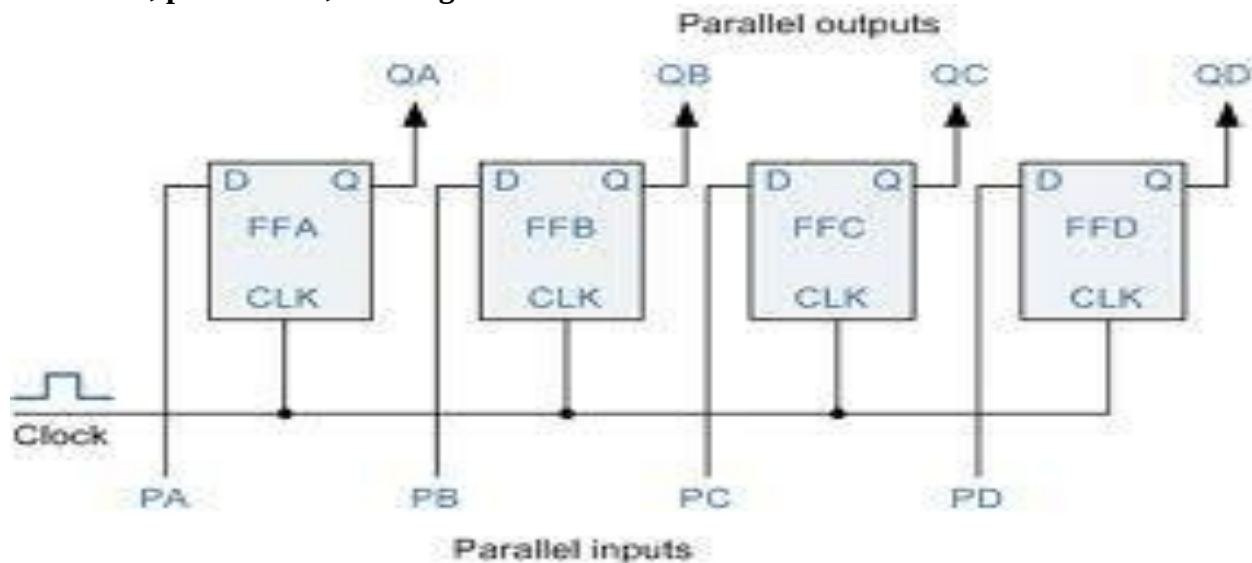
Parallel-in, serial-out, shift register:



For a parallel-in, serial out, shift register, the data bits are entered simultaneously into their respective stages on parallel lines, rather than on a bit-by-bit basis on one line as with serial data bits are transferred out of the register serially. On a bit-by-bit basis over a single line.

There are four data lines A,B,C,D through which the data is entered into the register in parallel form. The signal shift/ load allows the data to be entered in parallel form into the register and the data is shifted out serially from terminal Q4

Parallel-in, parallel-out, shift register



In a parallel-in, parallel-out shift register, the data is entered into the register in parallel form, and also the data is taken out of the register in parallel form. Data is applied to the D input terminals of the FF's. When a clock pulse is applied, at the positive going edge of the pulse, the D inputs are shifted into the Q outputs of the FFs. The register now stores the data. The stored data is available instantaneously for shifting out in parallel form.

Bidirectional shift register:

A bidirectional shift register is one which the data bits can be shifted from left to right or from right to left. A fig shows the logic diagram of a 4-bit serial-in, serial out, bidirectional shift register. Right/left is the mode signal, when right/left is a 1, the logic circuit works as a shift-register. the bidirectional operation is achieved by using the mode signal and two NAND gates and one OR gate for each stage.

A HIGH on the right/left control input enables the AND gates G1, G2, G3 and G4 and disables the AND gates G5, G6, G7 and G8, and the state of Q output of each FF is passed through the gate to the D input of the following FF. when a clock pulse occurs, the data bits are then effectively shifted one place to the right. A LOW on the right/left control inputs enables the AND gates G5, G6, G7 and G8 and disables the And gates G1, G2, G3 and G4 and the Q output of each FF is passed to the D input of the preceding FF. when a clock pulse occurs, the data bits are then effectively shifted one place to the left. Hence, the circuit works as a bidirectional shift register

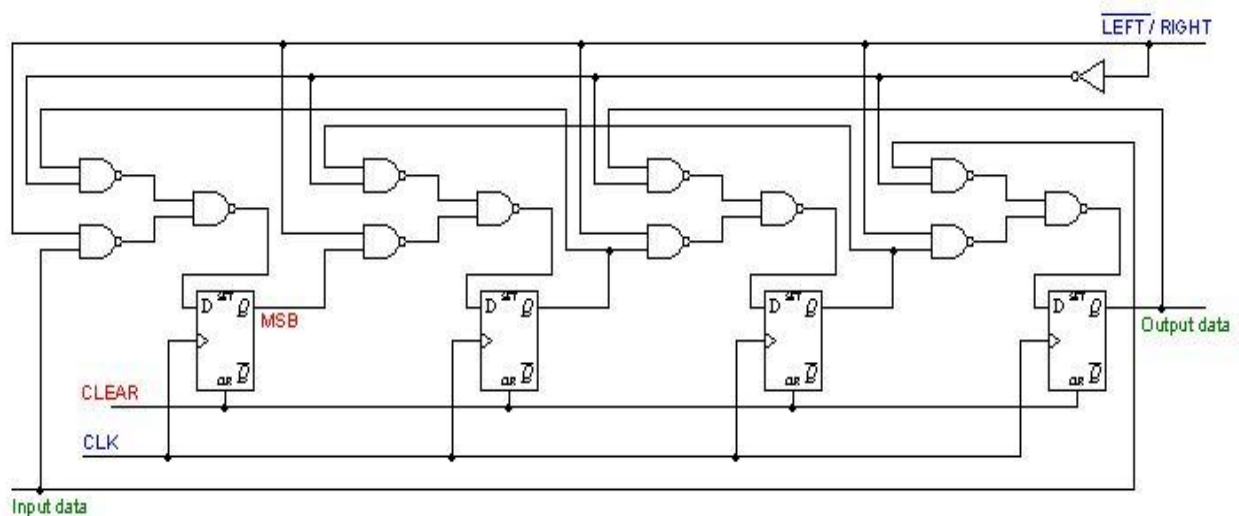


Figure: logic diagram of a 4-bit bidirectional shift register

Universal shift register:

A register is capable of shifting in one direction only is a unidirectional shift register. One that can shift both directions is a bidirectional shift register. If the register has both shifts and parallel load capabilities, it is referred to as a universal shift registers. Universal shift register is a bidirectional register, whose input can be either in serial form or in parallel form and whose output also can be in serial form or I parallel form.

The most general shift register has the following capabilities.

1. A clear control to clear the register to 0
2. A clock input to synchronize the operations
3. A shift-right control to enable the shift-right operation and serial input and output lines associated with the shift-right

4. A shift-left control to enable the shift-left operation and serial input and output lines associated with the shift-left
5. A parallel loads control to enable a parallel transfer and the n input lines associated with the parallel transfer
6. N parallel output lines
7. A control state that leaves the information in the register unchanged in the presence of the clock.

A universal shift register can be realized using multiplexers. The below fig shows the logic diagram of a 4-bit universal shift register that has all capabilities. It consists of 4 D flip-flops and four multiplexers. The four multiplexers have two common selection inputs s_1 and s_0 . Input 0 in each multiplexer is selected when $S_1S_0=00$, input 1 is selected when $S_1S_0=01$ and input 2 is selected when $S_1S_0=10$ and input 4 is selected when $S_1S_0=11$. The selection inputs control the mode of operation of the register according to the functions entries. When $S_1S_0=0$, the present value of the register is applied to the D inputs of flip-flops. The condition forms a path from the output of each flip-flop into the input of the same flip-flop. The next clock edge transfers into each flip-flop the binary value it held previously, and no change of state occurs. When $S_1S_0=01$, terminal 1 of the multiplexer inputs have a path to the D inputs of the flip-flop. This causes a shift-right operation, with serial input transferred into flip-flop A_4 . When $S_1S_0=10$, a shift left operation results with the other serial input going into flip-flop A_1 . Finally when $S_1S_0=11$, the binary information on the parallel input lines is transferred into the register simultaneously during the next clock cycle

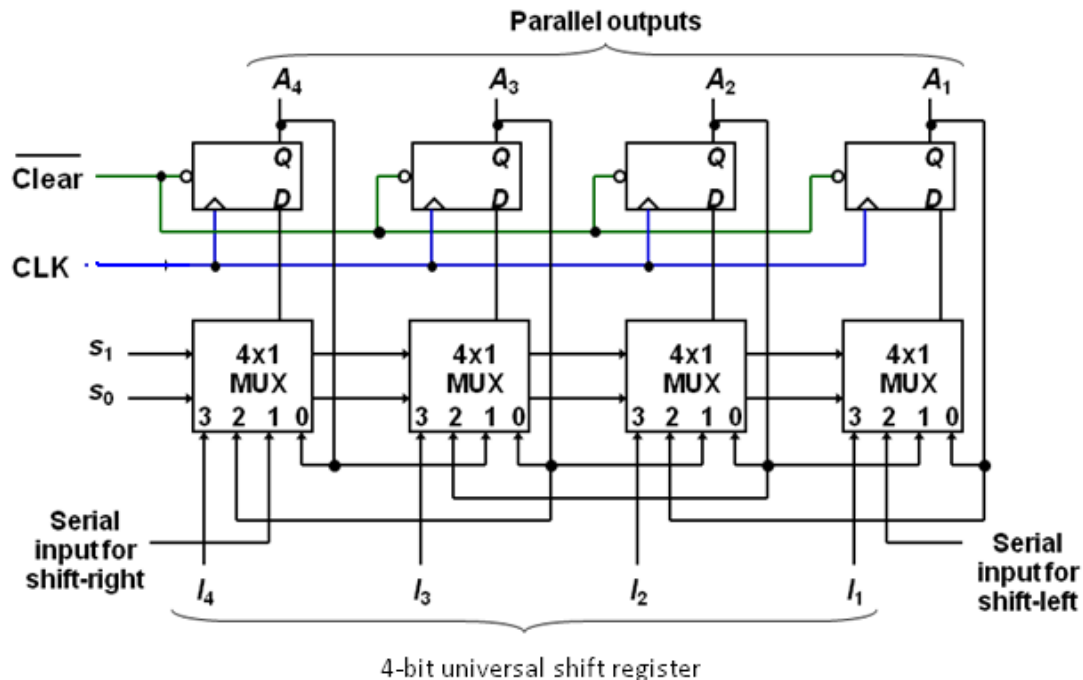


Figure: logic diagram 4-bit universal shift register

Function table for the register

mode control		
S0	S1	register operation
0	0	No change
0	1	Shift Right
1	0	Shift left
1	1	Parallel load

Counters:

Counter is a device which stores (and sometimes displays) the number of times particular event or process has occurred, often in relationship to a clock signal. A Digital counter is a set of flip flops whose state change in response to pulses applied at the input to the counter. Counters may be asynchronous counters or synchronous counters. Asynchronous counters are also called ripple counters

In electronics counters can be implemented quite easily using register-type circuits such as the flip-flops and a wide variety of classifications exist:

- Asynchronous (ripple) counter – changing state bits are used as clocks to subsequent state flip-flops
- Synchronous counter – all state bits change under control of a single clock
- Decade counter – counts through ten states per stage
- Up/down counter – counts both up and down, under command of a control input
- Ring counter – formed by a shift register with feedback connection in a ring
- Johnson counter – a *twisted* ring counter
- ☐ Cascaded counter
- ☐ Modulus counter.

Each is useful for different applications. Usually, counter circuits are digital in nature, and count in natural binary. Many types of counter circuits are available as digital building blocks, for example a number of chips in the 4000 series implement different counters.

Occasionally there are advantages to using a counting sequence other than the natural binary sequence such as the binary coded decimal counter, a linear feed-back shift register counter, or a gray-code counter.

Counters are useful for digital clocks and timers, and in oven timers, VCR clocks, etc.

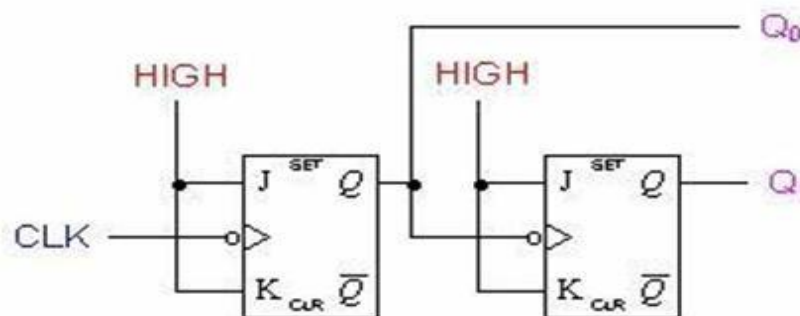
Asynchronous counters:

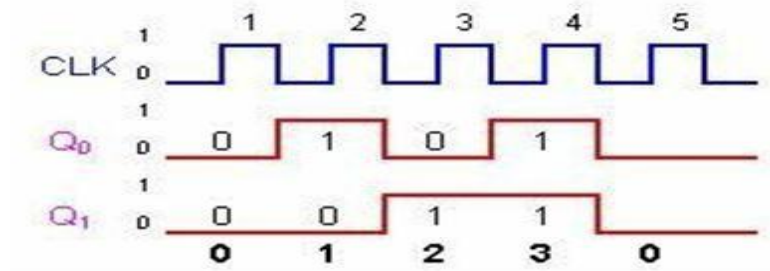
An asynchronous (ripple) counter is a single [JK-type flip-flop](#), with its J (data) input fed from its own inverted output. This circuit can store one bit, and hence can count from zero to one before it overflows (starts over from 0). This counter will increment once for every clock cycle and takes two clock cycles to overflow, so every cycle it will alternate between a transition from 0 to 1 and a transition from 1 to 0. Notice that this creates a new clock with a 50% [duty cycle](#) at exactly half the frequency of the input clock. If this output is then used as the clock signal for a similarly arranged D flip-flop (remembering to invert the output to the input), one will get another 1 bit counter that counts half as fast. Putting them together yields a two-bit counter:

Two-bit ripple up-counter using negative edge triggered flip flop:

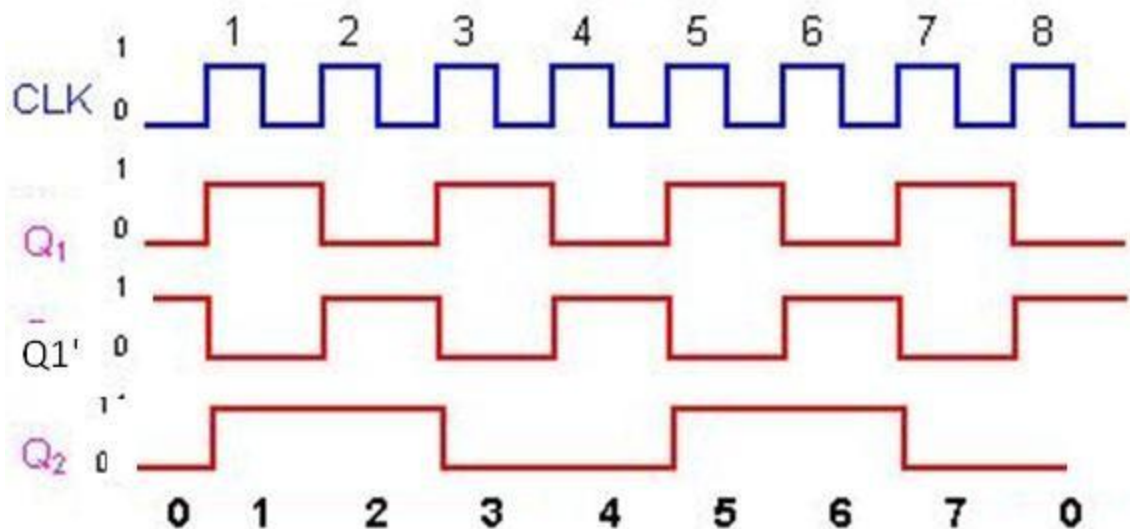
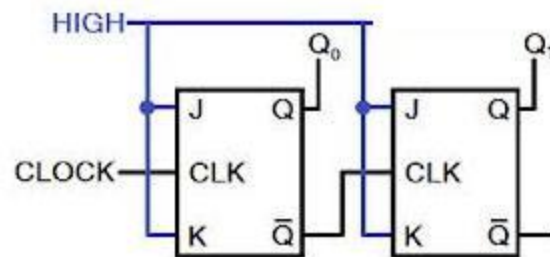
Two bit ripple counter used two flip-flops. There are four possible states from 2 – bit up-counting I.e. 00, 01, 10 and 11.

- The counter is initially assumed to be at a state 00 where the outputs of the two flip-flops are noted as Q_1Q_0 . Where Q_1 forms the MSB and Q_0 forms the LSB.
- For the negative edge of the first clock pulse, output of the first flip-flop FF_1 toggles its state. Thus Q_1 remains at 0 and Q_0 toggles to 1 and the counter state are now read as 01.
- During the next negative edge of the input clock pulse FF_1 toggles and $Q_0 = 0$. The output Q_0 being a clock signal for the second flip-flop FF_2 and the present transition acts as a negative edge for FF_2 thus toggles its state $Q_1 = 1$. The counter state is now read as 10.
- For the next negative edge of the input clock to FF_1 output Q_0 toggles to 1. But this transition from 0 to 1 being a positive edge for FF_2 output Q_1 remains at 1. The counter state is now read as 11.
- For the next negative edge of the input clock, Q_0 toggles to 0. This transition from 1 to 0 acts as a negative edge clock for FF_2 and its output Q_1 toggles to 0. Thus the starting state 00 is attained. Figure shown below





Two-bit ripple down-counter using negative edge triggered flip flop:



A 2-bit down-counter counts in the order 0,3,2,1,0,1.....,i.e. 00,11,10,01,00,11etc. the above fig. shows ripple down counter, using negative edge triggered J-K FFs and its timing diagram.

- For down counting, $Q1'$ of FF1 is connected to the clock of Ff2. Let initially all the FF1 toggles, so, $Q1$ goes from a 0 to a 1 and $Q1'$ goes from a 1 to a 0.

- The negative-going signal at Q_1' is applied to the clock input of FF2, toggles FF2 and, therefore, Q_2 goes from a 0 to a 1. so, after one clock pulse $Q_2=1$ and $Q_1=1$, I.e., the state of the counter is 11.
- At the negative-going edge of the second clock pulse, Q_1 changes from a 1 to a 0 and Q_1' from a 0 to a 1.
- This positive-going signal at Q_1' does not affect FF2 and, therefore, Q_2 remains at a 1. Hence, the state of the counter after second clock pulse is 10
- At the negative going edge of the third clock pulse, FF1 toggles. So Q_1 , goes from a 0 to a 1 and Q_1' from 1 to 0. This negative going signal at Q_1' toggles FF2 and, so, Q_2 changes from 1 to 0, hence, the state of the counter after the third clock pulse is 01.
- At the negative going edge of the fourth clock pulse, FF1 toggles. So Q_1 , goes from a 1 to a 0 and Q_1' from 0 to 1. . This positive going signal at Q_1' does not affect FF2 and, so, Q_2 remains at 0, hence, the state of the counter after the fourth clock pulse is 00.

Two-bit ripple up-down counter using negative edge triggered flip flop:

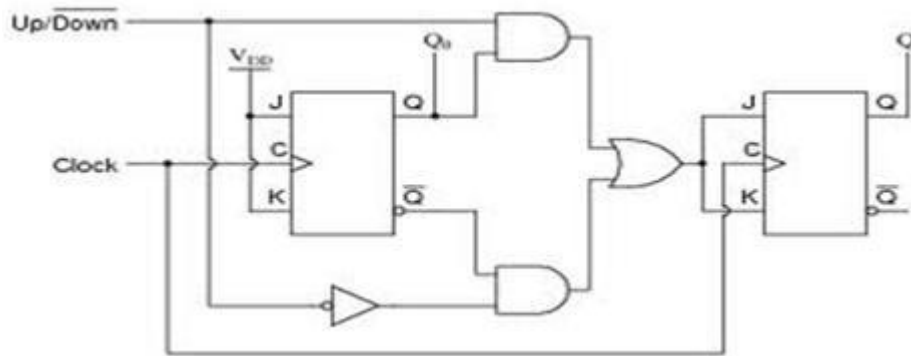


Figure: asynchronous 2-bit ripple up-down counter using negative edge triggered flip flop:

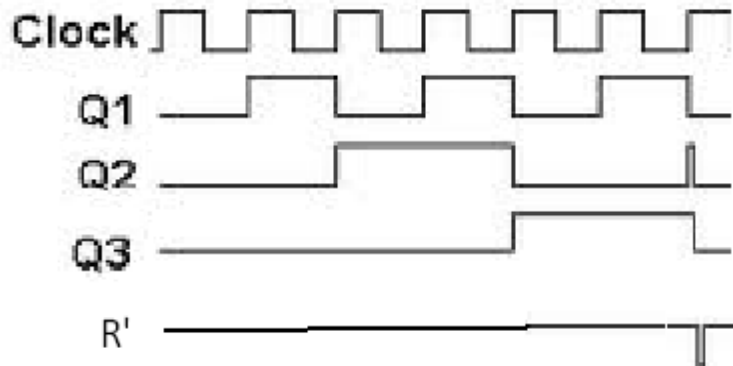
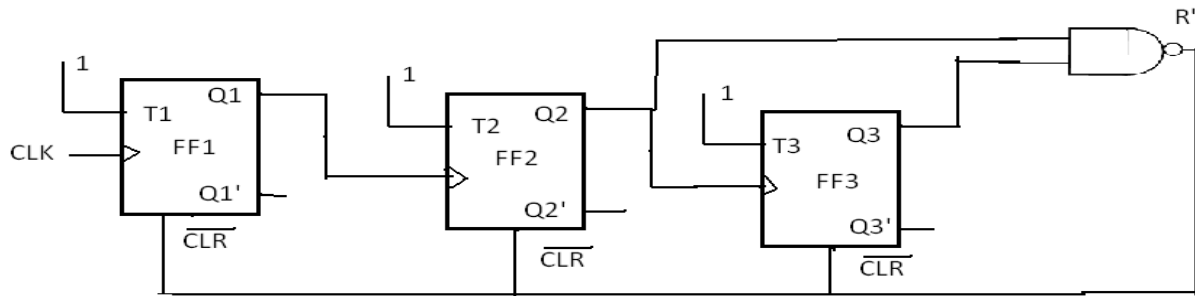
- As the name indicates an up-down counter is a counter which can count both in upward and downward directions. An up-down counter is also called a forward/backward counter or a bidirectional counter. So, a control signal or a mode signal M is required to choose the direction of count. When $M=1$ for up counting, Q_1 is transmitted to clock of FF2 and when $M=0$ for down counting, Q_1' is transmitted to clock of FF2. This is achieved by using two AND gates and one OR gates. The external clock signal is applied to FF1.
- Clock signal to FF2 = $(Q_1 \cdot \text{Up}) + (Q_1' \cdot \text{Down}) = Q_1 m + Q_1' M'$

Design of Asynchronous counters:

To design a asynchronous counter, first we write the sequence, then tabulate the values of reset signal R for various states of the counter and obtain the minimal expression for R and R' using K-Map or any other method. Provide a feedback such that R and R' resets all the FF's after the desired count

Design of a Mod-6 asynchronous counter using T FFs:

A mod-6 counter has six stable states 000, 001, 010, 011, 100, and 101. When the sixth clock pulse is applied, the counter temporarily goes to 110 state, but immediately resets to 000 because of the feedback provided. It is a divide-by-6 counter, in the sense that it divides the input clock frequency by 6. It requires three FFs, because the smallest value of n satisfying the condition $N \leq 2^n$ is $n=3$; three FFs can have 8 possible states, out of which only six are utilized and the remaining two states 110 and 111, are invalid. If initially the counter is in 000 state, then after the sixth clock pulse, it goes to 001, after the second clock pulse, it goes to 010, and so on.



After sixth clock pulse it goes to 000. For the design, write the truth table with present state outputs Q_3 , Q_2 and Q_1 as the variables, and reset R as the output and obtain an expression for R in terms of Q_3 , Q_2 , and Q_1 that decides the feedback into to be provided. From the truth table, $R = Q_3Q_2$. For active-low Reset, R' is used. The reset pulse is of very short duration, of the order of nanoseconds and it is equal to the propagation delay time of the NAND gate used. The expression for R can also be determined as follows.

$$R=0 \text{ for } 000 \text{ to } 101, R=1 \text{ for } 110, \text{ and } R=X \text{ for } 111$$

Therefore,

$$R = Q_3Q_2Q_1' + Q_3Q_2Q_1 = Q_3Q_2$$

The logic diagram and timing diagram of Mod-6 counter is shown in the above fig.

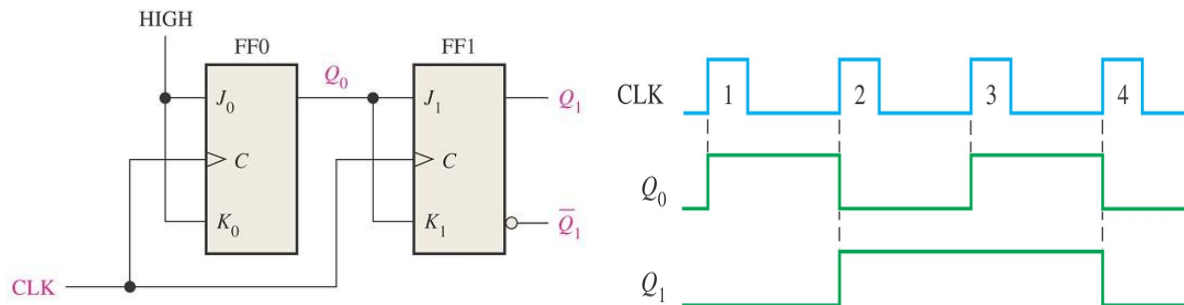
The truth table is as shown in below.

		Q2Q1			
		00	01	11	10
Q4Q3	00				
	01				
	11	X	X	X	X
	10		X	X	1

After pulses	Count			
	Q4	Q3	Q2	Q1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	0	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	0	1	0	1
10	0	0	0	0

Synchronous counters:

Asynchronous counters are serial counters. They are slow because each FF can change state only if all the preceding FFs have changed their state. If the clock frequency is very high, the asynchronous counter may skip some of the states. This problem is overcome in synchronous counters or parallel counters. Synchronous counters are counters in which all the flip-flops are triggered simultaneously by the clock pulses. Synchronous counters have a common clock pulse applied simultaneously to all flip-flops. □ A 2-Bit Synchronous Binary Counter



Design of synchronous counters:

For a systematic design of synchronous counters, the following procedure is used.

Step 1: State Diagram: draw the state diagram showing all the possible states. State diagram, which is also called an n th transition diagram, is a graphical means of depicting the sequence of states through which the counter progresses.

Step 2: number of flip-flops: based on the description of the problem, determine the required number n of the flip-flops. The smallest value of n is such that the number of states $N \leq 2^n$ and the desired counting sequence.

Step 3: choice of flip-flops excitation table: select the type of flip-flop to be used and write the excitation table. An excitation table is a table that lists the present state (ps), the next state (ns), and required excitations.

Step4: minimal expressions for excitations: obtain the minimal expressions for the excitations of the FF using K-maps drawn for the excitation of the flip-flops in terms of the present states and inputs.

Step5: logic diagram: draw a logic diagram based on the minimal expressions

Design of a synchronous 3-bit up-down counter using JK flip-flops:

Step1: determine the number of flip-flops required. A 3-bit counter requires three FFs. It has 8 states (000,001,010,011,101,110,111) and all the states are valid. Hence no don't cares. For selecting up and down modes, a control or mode signal M is required. When the mode signal M=1 and counts down when M=0. The clock signal is applied to all the FFs simultaneously.

Step2: draw the state diagrams: the state diagram of the 3-bit up-down counter is drawn as

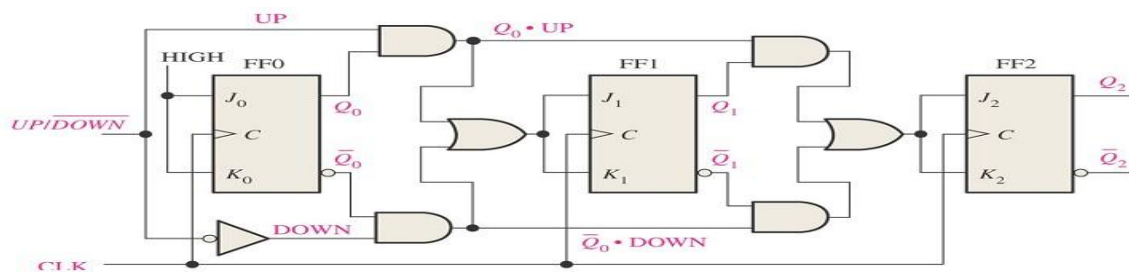
Step3: select the type of flip flop and draw the excitation table: JK flip-flops are selected and the excitation table of a 3-bit up-down counter using JK flip-flops is drawn as shown in fig.

PS			mode	NS			required excitations					
Q3	Q2	Q1	M	Q3	Q2	Q1	J3	K3	J2	K2	J1	K1
0	0	0	0	1	1	1	1	x	1	x	1	x
0	0	0	1	0	0	1	0	x	0	x	1	x
0	0	1	0	0	0	0	0	x	0	x	x	1
0	0	1	1	0	1	0	0	x	1	x	x	1
0	1	0	0	0	0	1	0	x	x	1	1	x
0	1	0	1	0	1	1	0	x	x	0	1	x
0	1	1	0	0	1	0	0	x	x	0	x	1
0	1	1	1	1	0	0	1	x	x	1	x	1
1	0	0	0	0	1	1	x	1	1	x	1	x
1	0	0	1	1	0	1	x	0	0	x	1	x
1	0	1	0	1	0	0	x	0	0	x	x	1
1	0	1	1	1	1	0	x	0	1	x	x	1
1	1	0	0	1	0	1	x	0	x	1	1	x
1	1	0	1	1	1	1	x	0	x	0	1	x
1	1	1	0	1	1	0	x	0	x	0	x	1
1	1	1	1	0	0	0	x	1	x	1	x	1

Step4: obtain the minimal expressions: From the excitation table we can conclude that J1=1 and K1=1, because all the entries for J1 and K1 are either X or 1. The K-maps for J3, K3, J2 and K2 based on the excitation table and the minimal expression obtained from them are shown in fig.

	00	01	11	10
Q3Q2 \ Q1M				
1	1			
		1		
X	X	X	X	X
X	X	X	X	X

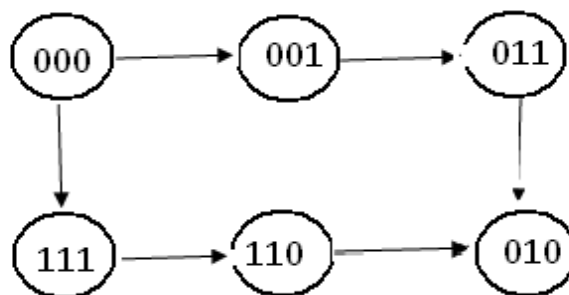
Step5: draw the logic diagram: a logic diagram using those minimal expressions can be drawn as shown in fig.



Design of a synchronous modulo-6 gray cod counter:

Step 1: the number of flip-flops: we know that the counting sequence for a modulo-6 gray code counter is 000, 001, 011, 010, 110, and 111. It requires $n=3$ FFs ($N \leq 2^n$, i.e., $6 \leq 2^3$). 3 FFs can have 8 states. So the remaining two states 101 and 100 are invalid. The entries for excitation corresponding to invalid states are don't cares.

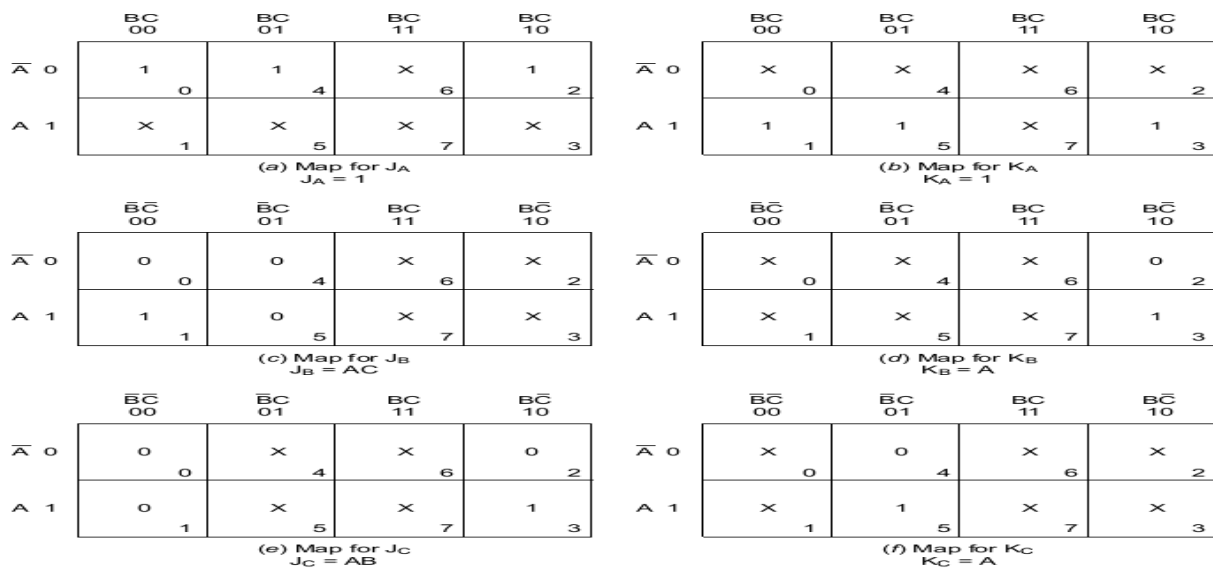
Step2: the state diagram: the state diagram of the mod-6 gray code converter is drawn as shown in fig.



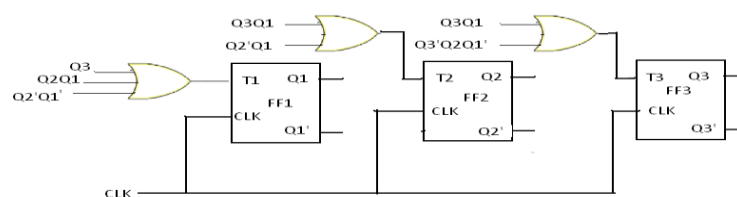
Step3: type of flip-flop and the excitation table: T flip-flops are selected and the excitation table of the mod-6 gray code counter using T-flip-flops is written as shown in fig.

PS			NS			required excitations		
Q3	Q2	Q1	Q3	Q2	Q1	T3	T2	T1
0	0	0	0	0	1	0	0	1
0	0	1	0	1	1	0	1	0
0	1	1	0	1	0	0	0	1
0	1	0	1	1	0	1	0	0
1	1	0	1	1	1	0	0	1
1	1	1	0	0	0	1	1	1

Step4: The minimal expressions: the K-maps for excitations of FFs T3,T2,and T1 in terms of outputs of FFs Q3,Q2, and Q1, their minimization and the minimal expressions for excitations obtained from them are shown if fig



Step5: the logic diagram: the logic diagram based on those minimal expressions is drawn as shown in fig.



Design of a synchronous BCD Up-Down counter using FFs:

Step1: the number of flip-flops: a BCD counter is a mod-10 counter has 10 states (0000 through 1001) and so it requires $n=4\text{FFs}(N \leq 2^n, \text{ i.e., } 10 \leq 2^4)$. 4 FFS can have 16 states. So out of 16 states, six states (1010 through 1111) are invalid. For selecting up and down mode, a control or mode signal M is required. , it counts up when $M=1$ and counts down when $M=0$. The clock signal is applied to all FFs.

Step2: the state diagram: The state diagram of the mod-10 up-down counter is drawn as shown in fig.

Step3: types of flip-flops and excitation table: T flip-flops are selected and the excitation table of the modulo-10 up down counter using T flip-flops is drawn as shown in fig.

The remaining minterms are don't cares ($\sum d(20,21,22,23,24,25,26,27,28,29,30,31)$) from the excitation table we can see that $T1=1$ and the expression for $T4, T3, T2$ are as follows.

$$T4 = \sum m(0,15,16,19) + d(20,21,22,23,24,25,26,27,28,29,30,31)$$

$$T3 = \sum m(7,15,16,8) + d(20,21,22,23,24,25,26,27,28,29,30,31)$$

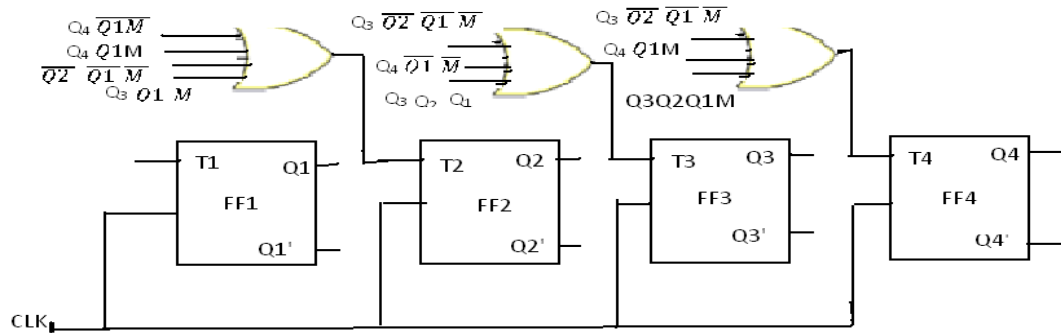
$$T2 = \sum m(3,4,7,8,11,12,15,16) + d(20,21,22,23,24,25,26,27,28,29,30,31)$$

PS				mode	NS				required excitations			
Q4	Q3	Q2	Q1		Q4	Q3	Q2	Q1				
0	0	0	0	0	1	0	0	1	1	0	0	1
0	0	0	0	1	0	0	0	1	0	0	0	1
0	0	0	1	0	0	0	0	0	0	0	0	1
0	0	0	1	1	0	0	1	0	0	0	1	1
0	0	1	0	0	0	0	0	1	0	0	1	1
0	0	1	0	1	0	0	1	1	0	0	0	1
0	0	1	1	0	0	0	1	0	0	0	0	1
0	0	1	1	1	0	1	0	0	0	1	1	1
0	1	0	0	0	0	0	1	1	0	1	1	1
0	1	0	0	1	0	1	0	1	0	0	0	1
0	1	0	1	0	0	1	0	0	0	0	0	1
0	1	0	1	1	0	1	1	0	0	0	1	1
0	1	1	0	0	0	1	0	1	0	0	1	1
0	1	1	0	1	0	1	1	1	0	0	0	1
0	1	1	1	0	0	1	1	0	0	0	0	1
0	1	1	1	1	1	0	0	0	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	1	1	0	0	1	0	0	0	1
1	0	0	1	0	1	0	0	0	0	0	0	1
1	0	0	1	1	0	0	0	0	1	0	0	1

Step4: The minimal expression: since there are 4 state variables and a mode signal, we require 5 variable kmaps. 20 conditions of $Q_4Q_3Q_2Q_1M$ are valid and the remaining 12 combinations are invalid. So the entries for excitations corresponding to those invalid combinations are don't cares. Minimizing K-maps for T2 we get

$$T_2 = Q_4Q_1'M + Q_4'Q_1M + Q_2Q_1'M' + Q_3Q_1'M'$$

Step5: the logic diagram: the logic diagram based on the above equation is shown in fig.



Shift register counters:

One of the applications of shift register is that they can be arranged to form several types of counters. The most widely used shift register counter is ring counter as well as the twisted ring counter.

Ring counter: this is the simplest shift register counter. The basic ring counter using D flip-flops is shown in fig. the realization of this counter using JK FFs. The Q output of each stage is connected to the D flip-flop connected back to the ring counter.

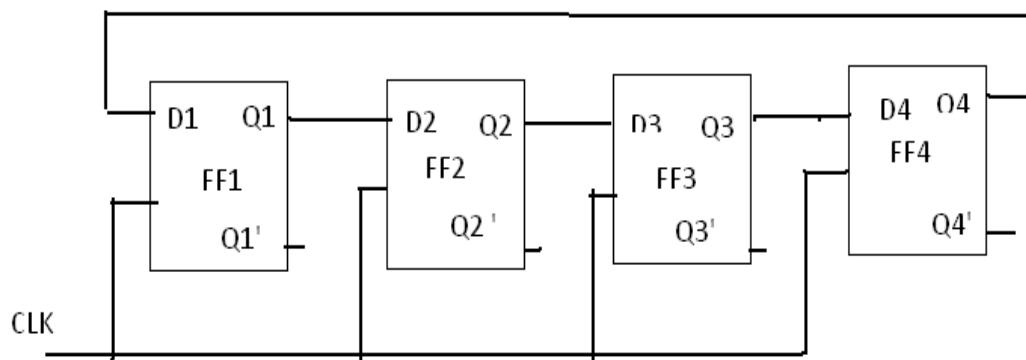


FIGURE: logic diagram of 4-bit ring counter using D flip-flops

Only a single 1 is in the register and is made to circulate around the register as long as clock pulses are applied. Initially the first FF is present to a 1. So, the initial state is 1000, i.e., $Q_1=1, Q_2=0, Q_3=0, Q_4=0$. After each clock pulse, the contents of the register are shifted to the right by one bit and Q_4 is shifted back to Q_1 . The sequence repeats after four clock pulses. The number

of distinct states in the ring counter, i.e., the mod of the ring counter is equal to number of FFs used in the counter. An n-bit ring counter can count only n bits, whereas n-bit ripple counter can count 2^n bits. So, the ring counter is uneconomical compared to a ripple counter but has advantage of requiring no decoder, since we can read the count by simply noting which FF is set. Since it is entirely a synchronous operation and requires no gates external FFs, it has the further advantage of being very fast.

Timing diagram:

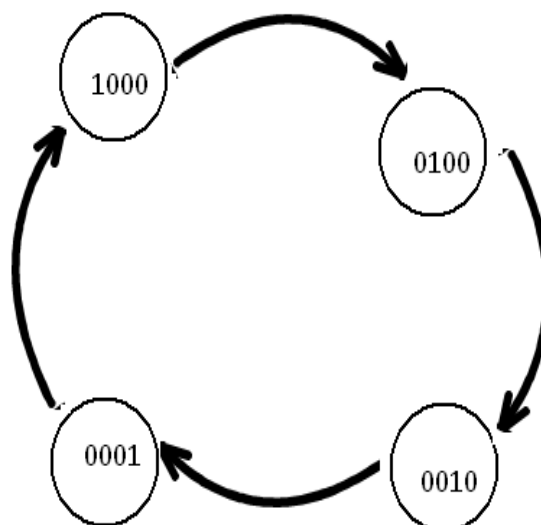
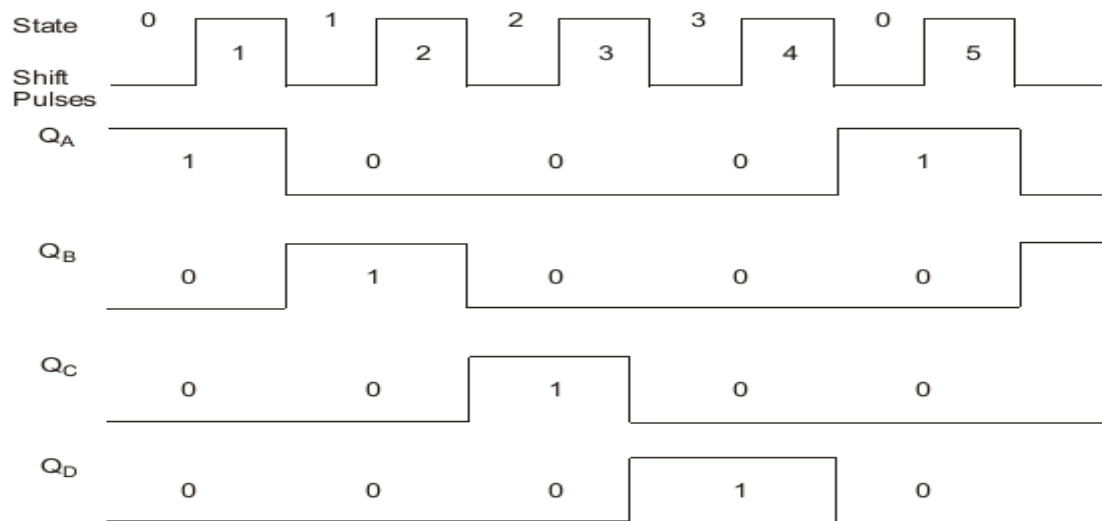


Figure: state diagram

Twisted Ring counter (Johnson counter):

This counter is obtained from a serial-in, serial-out shift register by providing feedback from the inverted output of the last FF to the D input of the first FF. the Q output of each is connected to the D input of the next stage, but the Q' output of the last stage is connected to the D input of the first stage, therefore, the name twisted ring counter. This feedback arrangement produces a unique sequence of states.

The logic diagram of a 4-bit Johnson counter using D FF is shown in fig. the realization of the same using J-K FFs is shown in fig.. The state diagram and the sequence table are shown in figure. The timing diagram of a Johnson counter is shown in figure.

Let initially all the FFs be reset, i.e., the state of the counter be 0000. After each clock pulse, the level of Q1 is shifted to Q2, the level of Q2 to Q3, Q3 to Q4 and the level of $Q4'$ to Q1 and the sequences given in fig.

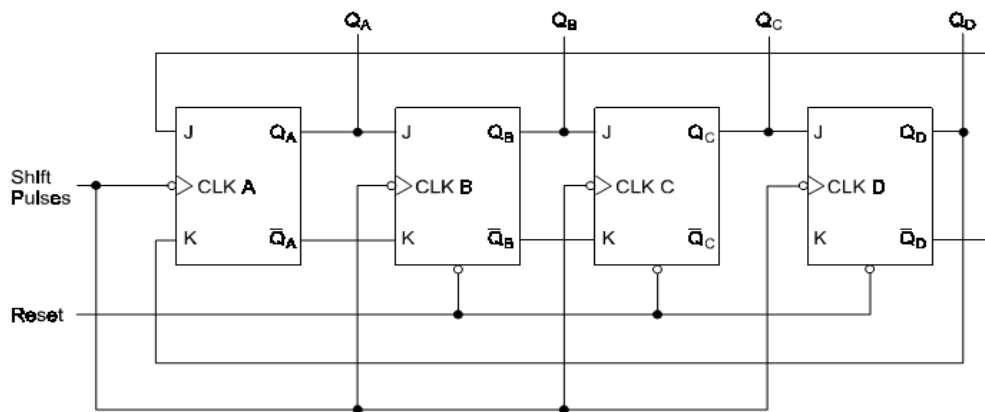


Figure: Johnson counter with JK flip-flops

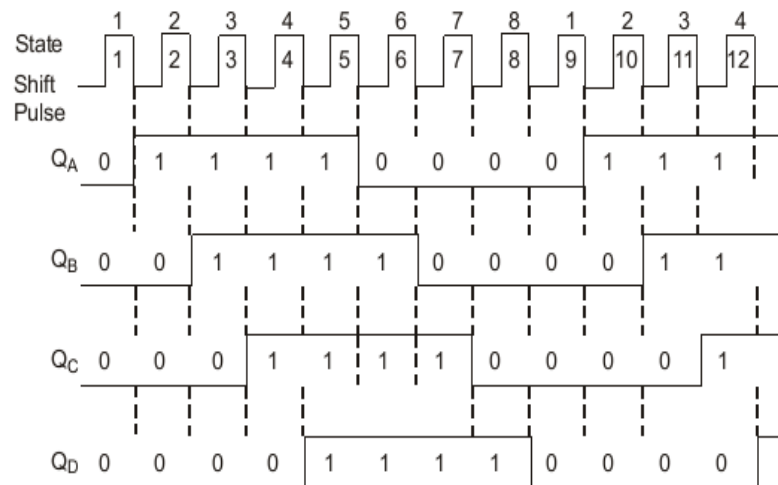
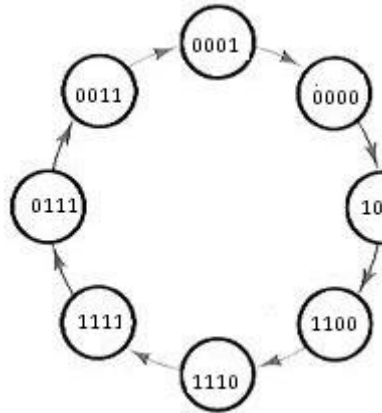


Figure: timing diagram

State diagram:



Q1	Q2	Q3	Q4	after clock pulse
0	0	0	0	0
1	0	0	0	1
1	1	0	0	2
1	1	1	0	3
1	1	1	1	4
0	1	1	1	5
0	0	1	1	6
0	0	0	1	7
0	0	0	0	8
1	0	0	0	9

Excitation table

Synthesis of sequential circuits:

The synchronous or clocked sequential circuits are represented by two models.

1. Moore circuit: in this model, the output depends only on the present state of the flip-flops
2. Mealy circuit: in this model, the output depends on both present state of the flip-flop. And the inputs.

Sequential circuits are also called finite state machines (FSMs). This name is due to the fact that the functional behavior of these circuits can be represented using a finite number of states.

State diagram: the state diagram or state graph is a pictorial representation of the relationships between the present state, the input, the next state, and the output of a sequential circuit. The state diagram is a pictorial representation of the behavior of a sequential circuit.

The state represented by a circle is also called the node or vertex and the transition between states is indicated by directed lines connecting circles. A directed line connecting a circle with itself indicates that the next state is the same as the present state. The binary number inside each circle identifies the state represented by the circle. The directed lines are labeled with two binary numbers separated by a symbol. The input value is applied during the present state is labeled after the symbol.

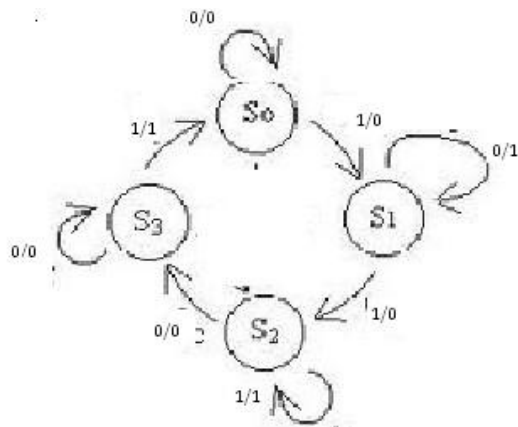


Fig :a) state diagram (meelay circuit)

NS,O/P		
INPUT X		
PS	X=0	X=1
a	a,0	b,0
b	b,1	c,0
c	d,0	c,1
d	d,0	a,1

fig: b) state table

In case of moore circuit ,the directed lines are labeled with only one binary number representing the input that causes the state transition. The output is indicated with in the circle below the present state, because the output depends only on the present state and not on the input.

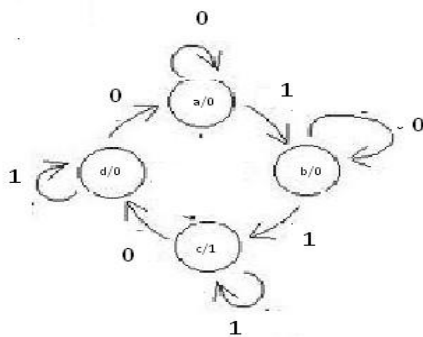


Fig: a) state diagram (moore circuit)

NS			
INPUT X			
PS	X=0	X=1	O/P
a	a	b	0
b	b	c	0
c	d	c	1
d	a	d	0

fig:b) state table

Serial binary adder:

Step1: word statement of the problem: the block diagram of a serial binary adder is shown in fig. it is a synchronous circuit with two input terminals designated X1 and X2 which carry the two binary numbers to be added and one output terminal Z which represents the sum. The inputs and outputs consist of fixed-length sequences 0s and 1s. the output of the serial Z_i at time t_i is a function of the inputs $X_1(t_i)$ and $X_2(t_i)$ at that time t_i and of carry which had been generated at t_{i-1} . The carry which represent the past history of the serial adder may be a 0 or 1. The circuit has two states. If one state indicates that carry from the previous addition is a 0, the other state indicates that the carry from the previous addition is a 1

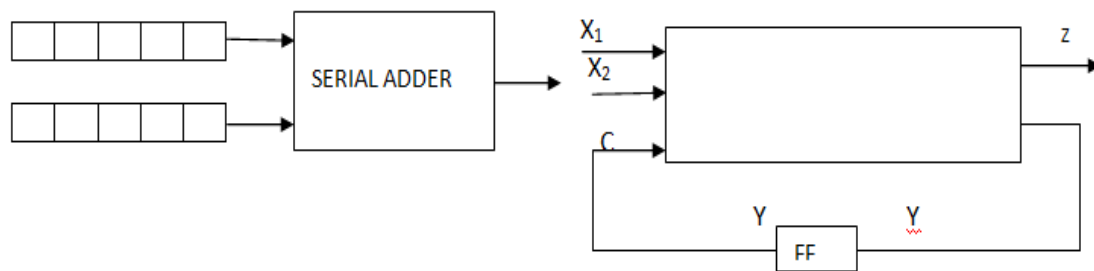
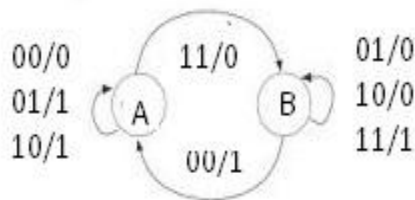


Figure: block diagram of serial binary adder

Step2 and 3: state diagram and state table: let a designate the state of the serial adder at t_i if a carry 0 was generated at t_{i-1} , and let b designate the state of the serial adder at t_i if carry 1 was generated at t_{i-1} . the state of the adder at that time when the present inputs are applied is referred to as the present state(PS) and the state to which the adder goes as a result of the new carry value is referred to as next state(NS).

The behavior of serial adder may be described by the state diagram and state table.



PS	NS ,O/P			
	X1 X2			
	0	0	1	1
	0	1	0	1
A	A,0	B,0	B,1	B,0
B	A,1	B,0	B,0	B,1

Figures: serial adder state diagram and state table

If the machine is in state B, i.e., carry from the previous addition is a 1, inputs $X_1=0$ and $X_2=1$ gives sum, 0 and carry 1. So the machine remains in state B and outputs a 0. Inputs $X_1=1$ and $X_2=0$ gives sum, 0 and carry 1. So the machine remains in state B and outputs a 0. Inputs $X_1=1$ and $X_2=1$ gives sum, 1 and carry 0. So the machine remains in state B and outputs a 1. Inputs $X_1=0$ and $X_2=0$ gives sum, 1 and carry 0. So the machine goes to state A and outputs a 1. The state table also gives the same information.

Setp4: reduced standard from state table: the machine is already in this form. So no need to do anything

Step5: state assignment and transition and output table:

The states, $A=0$ and $B=1$ have already been assigned. So, the transition and output table is as shown.

PS	NS				O/P			
	0	0	1	1	0	0	1	1
	0	1	0	1	0	1	0	1
0	0	0	0	1	0	1	1	1
1	0	1	1	1	1	0	0	1

STEP6: choose type of FF and excitation table: to write table, select the memory element the excitation table is as shown in fig.

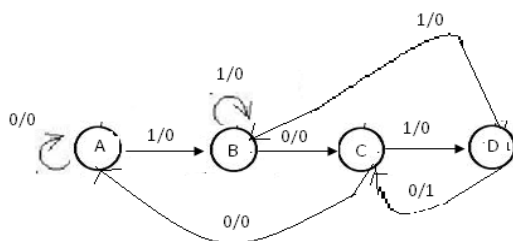
PS	I/P		NS	I/P-FF	O/P
y	x1	x2	Y	D	Z
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	1	1

Sequence detector:

Step1: word statement of the problem: a sequence detector is a sequential machine which produces an output 1 every time the desired sequence is detected and an output 0 at all other times

Suppose we want to design a sequence detector to detect the sequence 1010 and say that overlapping is permitted i.e., for example, if the input sequence is 01101010 the corresponding output sequence is 00000101.

Step2 and 3: state diagram and state table: the state diagram and the state table of the sequence detector. At the time t_1 , the machine is assumed to be in the initial state designed arbitrarily as A. while in this state, the machine can receive first bit input, either a 0 or a 1. If the input bit is 0, the machine does not start the detection process because the first bit in the desired sequence is a 1. If the input bit is a 1 the detection process starts.



PS	NS,Z	
	X=0	X=1
A	A,0	B,0
B	C,0	B,0
C	A,0	D,0
D	C,1	B,0

Figure: state diagram and state table of sequence detector

So, the machine goes to state B and outputs a 0. While in state B, the machinery may receive 0 or 1 bit. If the bit is 0, the machine goes to the next state, say state c, because the previous two bits are 10 which are a part of the valid sequence, and outputs 0.. if the bit is a 1, the two bits become 11 and this not a part of the valid sequence

Step4: reduced standard form state table: the machine is already in this form. So no need to do anything.

Step5: state assignment and transition and output table: there are four states therefore two states variables are required. Two state variables can have a maximum of four states, so, all states are utilized and thus there are no invalid states. Hence, there are no don't cares. Let a=00, B=01, C=10 and D=11 be the state assignment.

PS(y1y2)	NS(Y1Y2)				O/P(z)	
	X=0		X=1		X=0	X=1
A= 0 0	0	0	0	1	0	0
B=0 1	1	0	0	1	0	0
C=1 0	0	0	1	1	0	0
D=1 1	1	1	0	1	1	0

Step6: choose type of flip-flops and form the excitation table: select the D flip-flops as memory elements and draw the excitation table.

PS		I/P	NS		INPUTS -		O/P
y1	Y2		Y1	Y2	D1	D2	Z
0	0	0	0	0	0	0	0
0	0	1	0	1	0	1	0
0	1	0	1	0	1	0	0
0	1	1	0	1	0	1	0
1	0	0	0	0	0	0	0
1	0	1	1	1	1	1	0
1	1	0	1	0	1	0	1
1	1	1	0	1	0	1	0

Step7: K-maps and minimal functions: based on the contents of the excitation table , draw the k-map and simplify them to obtain the minimal expressions for D1 and D2 in terms of y1, y2 and x as shown in fig. The expression for z (z=y1,y2) can be obtained directly from table

Step8: implementation: the logic diagram based on these minimal expressions

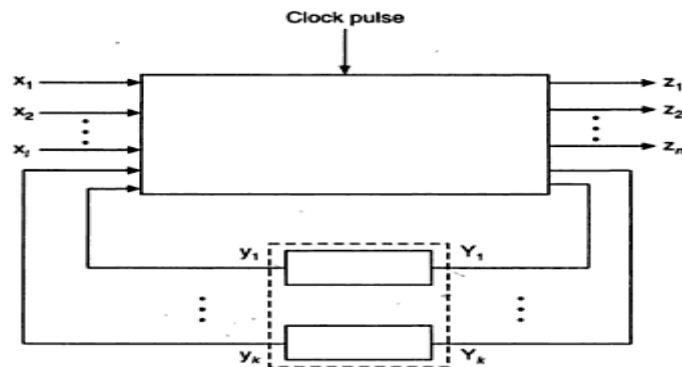
UNIT 5

Sequential circuits

Finite State Machine:

Finite state machine can be defined as a type of machine whose past histories can affect its future behavior in a finite number of ways. To clarify, consider for example of binary full adder. Its output depends on the present input and the carry generated from the previous input. It may have a large number of previous input histories but they can be divided into two types: (i) Input

The most general model of a sequential circuit has inputs, outputs and internal states. A sequential circuit is referred to as a finite state machine (FSM). A finite state machine is abstract model that describes the synchronous sequential machine. The fig. shows the block diagram of a finite state model. $X_1, X_2, \dots, X_i$ are inputs. $Z_1, Z_2, \dots, Z_m$ are outputs. $Y_1, Y_2, \dots, Y_k$ are state variables, and $Y_1, Y_2, \dots, Y_k$ represent the next state.



Capabilities and limitations of finite-state machine

Let a finite state machine have n states. Let a long sequence of input be given to the machine. The machine will progress starting from its beginning state to the next states according to the state transitions. However, after some time the input string may be longer than n , the number of states. As there are only n states in the machine, it must come to a state it was previously been in and from this phase if the input remains the same the machine will function in a periodically repeating fashion. From here a conclusion that _for a n state machine the output will become periodic after a number of clock pulses less than equal to n can be drawn. States are memory elements. As for a finite state machine the number of states is finite, so finite number of memory elements are required to design a finite state machine.

Limitations:

1. Periodic sequence and limitations of finite states: with n -state machines, we can generate periodic sequences of n states are smaller than n states. For example, in a 6-state machine, we can have a maximum periodic sequence as 0,1,2,3,4,5,0,1....
2. No infinite sequence: consider an infinite sequence such that the output is 1 when and only when the number of inputs received so far is equal to $P(P+1)/2$ for $P=1,2,3,\dots$, i.e., the desired input-output sequence has the following form:

Input: x
Output: 1 0 1 0 0 1 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 1

Such an infinite sequence cannot be produced by a finite state machine.

3. Limited memory: the finite state machine has a limited memory and due to limited memory it cannot produce certain outputs. Consider a binary multiplier circuit for multiplying two arbitrarily large binary numbers. The memory is not sufficient to store arbitrarily large partial products resulted during multiplication.

Finite state machines are two types. They differ in the way the output is generate they are:

1. Mealy type model: in this model, the output is a function of the present state and the present input.
2. Moore type model: in this model, the output is a function of the present state only.

Mathematical representation of synchronous sequential machine:

The relation between the present state $S(t)$, present input $X(t)$, and next state $s(t+1)$ can be given as

$$S(t+1) = f\{S(t), X(t)\}$$

The value of output $Z(t)$ can be given as

$$Z(t) = g\{S(t), X(t)\} \quad \text{for mealy model}$$

$$Z(t) = G\{S(t)\} \quad \text{for Moore model}$$

Because, in a mealy machine, the output depends on the present state and input, where as in a Moore machine, the output depends only on the present state.

Comparison between the Moore machine and mealy machine:

Moore machine	mealy machine
1. its output is a function of present state only $Z(t) = g\{S(t)\}$	1. its output is a function of present state as well as present input $Z(t) = g\{S(t), X(t)\}$
2. input changes do not affect the output	2. input changes may affect the output of the circuit
3. it requires more number of states for implementing same function	3. it requires less number of states for implementing same function

Mealy model:

When the output of the sequential circuit depends on the both the present state of the flip-flops and on the inputs, the sequential circuit is referred to as mealy circuit or mealy machine.

The fig. shows the logic diagram of the mealy model. Notice that the output depends up on the present state as well as the present inputs. We can easily realize that changes in the input during the clock pulse cannot affect the state of the flip-flop. They can affect the output of the circuit. If the input variations are not synchronized with a clock, he derived output will also not be synchronized with the clock and we get false output. The false outputs can be eliminated by allowing input to change only at the active transition of the clock.

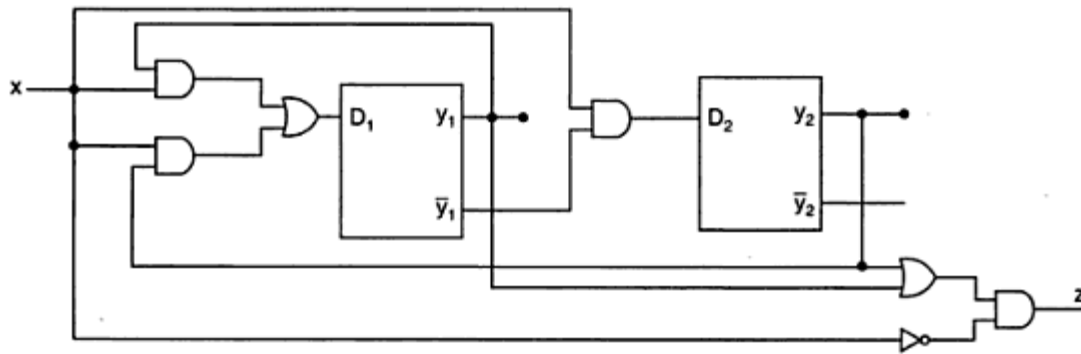


Fig: logic diagram of a mealy model

The behavior of a clocked sequential circuit can be described algebraically by means of state equations. A state equation specifies the next state as a function of the present state and inputs. The mealy model shown in fig. consists of two D flip-flops, an input x and an output z. since the D input of a flip-flop determines the value of the next state, the state equations for the model can be written as

$$Y_1(t+1) = y_1(t)x(t) + y_2(t)x(t)$$

$$Y_2(t+1) = y_1(t)x(t)$$

And the output equation is

$$Z(t) = \{ y_1(t) + y_2(t) \} X'(t)$$

Where $y(t+1)$ is the next state of the flip-flop one clock edge later, $x(t)$ is the present input, and $z(t)$ is the present output. If $y_1(t+1)$ are represented by $y_1(t)$ and $y_2(t)$, in more compact form, the equations are

$$Y_1(t+1) = y_1 = y_1x + y_2x$$

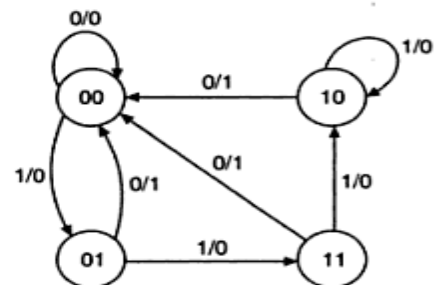
$$Y_2(t+1) = y_2 = y_1'x$$

$$Z = (y_1 + y_2)x'$$

The stable table of the mealy model based on the above state equations and output equation is shown in fig. the state diagram based on the state table is shown in fig.

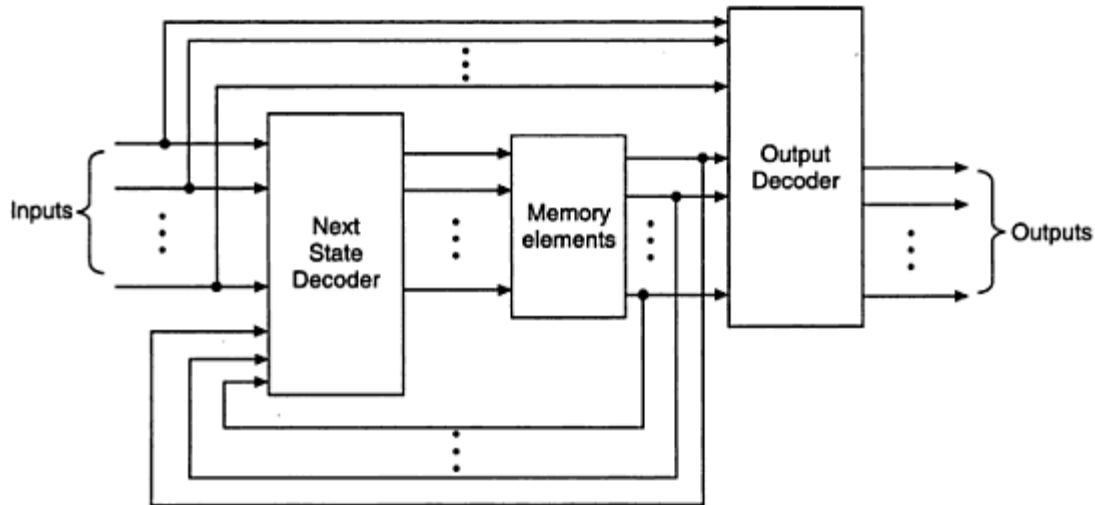
PS		NS				O/P	
		x = 0		x = 1		x = 0	x = 1
y_1	y_2	Y_1	Y_2	Y_1	Y_2	z	z
0	0	0	0	0	1	0	0
0	1	0	0	1	1	1	0
1	0	0	0	1	0	1	0
1	1	0	0	1	0	1	0

(a) State table



(b) State diagram

In general form, the mealy circuit can be represented with its block schematic as shown in below fig.



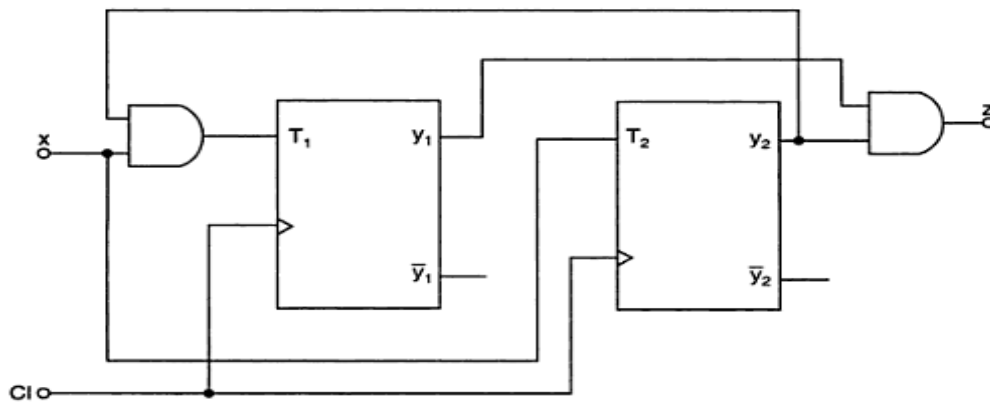
Moore model: when the output of the sequential circuit depends up only on the present state of the flip-flop, the sequential circuit is referred as to as the Moore circuit or the Moore machine.

Notice that the output depend only on the present state. It does not depend upon the input at all. The input is used only to determine the inputs of flip-flops. It is not used to determine the output. The circuit shown has two T flip-flops, one input x, and one output z. it can be described algebraically by two input equations an output equation.

$$T_1 = y_2 x$$

$$T_2 = x$$

$$Z = y_1 y_2$$



The characteristic equation of a T-flip-flop is

$$Q(t+1) = TQ' + T'Q$$

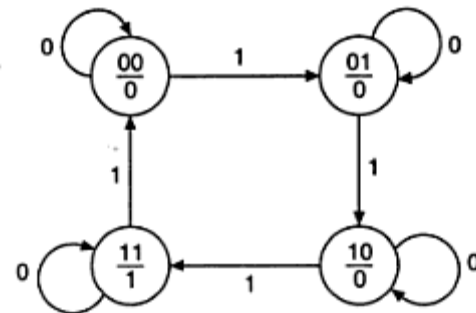
The values for the next state can be derived from the state equations by substituting T_1 and T_2 in the characteristic equation yielding

$$\begin{aligned} Y_1(t+1) &= Y_1 = (y_2 x) \oplus (y_2 x) y_1 + (y_2 x) y_1 x \\ &= y_1 y_2 + y_1 x + y_1 y_2 x \\ &= y_2 (t+1) = x \oplus y_2 = x y_2 + x y_2 \end{aligned}$$

The state table of the Moore model based on the above state equations and output equation is shown in fig.

PS		NS				O/P
		x = 0		x = 1		
y ₁	y ₂	Y ₁	Y ₂	Y ₁	Y ₂	z
0	0	0	0	0	1	0
0	1	0	1	1	0	0
1	0	1	0	1	1	0
1	1	1	1	0	0	1

(a) State table



(b) State diagram

In general form , the Moore circuit can be represented with its block schematic as shown in below fig.

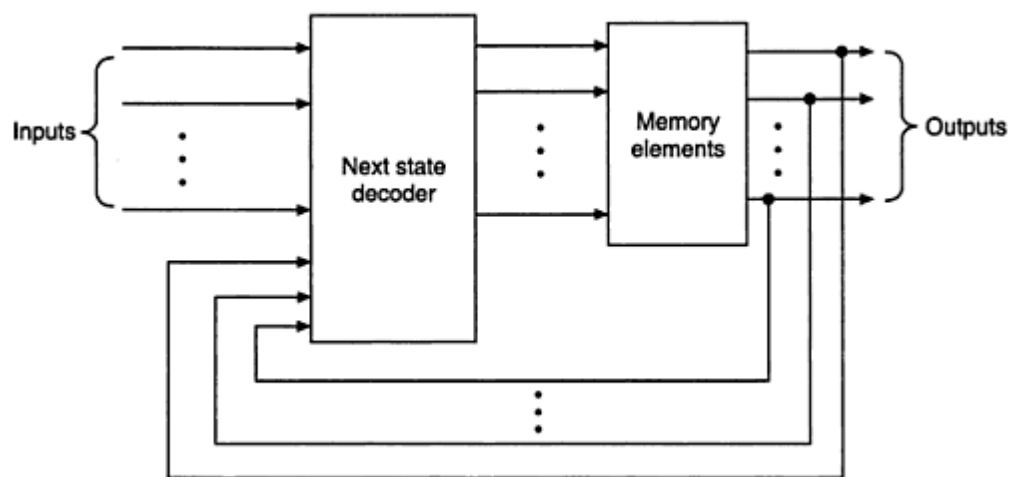


Figure: moore circuit model:

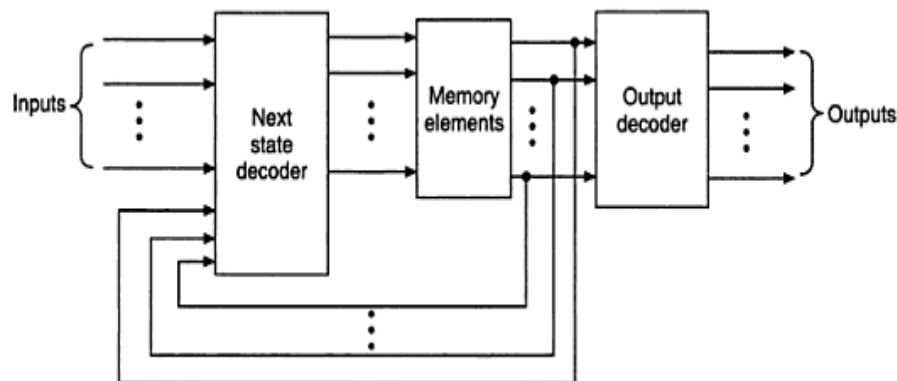
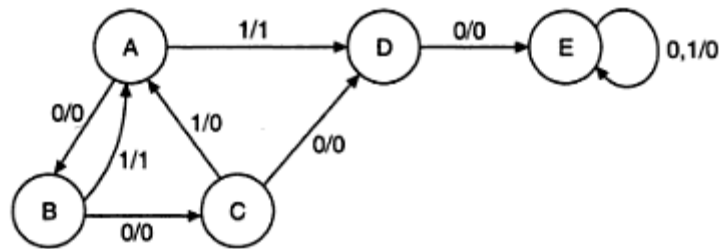


Figure: moore circuit model with an output decoder

Important definitions and theorems:

A). Finite state machine-definitions:

Consider the state diagram of a finite state machine shown in fig. it is five-state machine with one input variable and one output variable.



Successor: looking at the state diagram when present state is A and input is 1, the next state is D. this condition is specified as D is the successor of A. similarly we can say that A is the 1 successor of B, and C,D is the 11 successor of B and C, C is the 00 successor of A and D, D is the 000 successor of A,E, is the 10 successor of A or 0000 successor of A and so on.

Terminal state: looking at the state diagram , we observe that no such input sequence exists which can take the sequential machine out of state E and thus state E is said to be a terminal state.

Strongly-connected machine: in sequential machines many times certain subsets of states may not be reachable from other subsets of states. Even if the machine does not contain any terminal state. If for every pair of states s_i, s_j , of a sequential machine there exists an input sequence which takes the machine M from s_i to s_j , then the sequential machine is said to be strongly connected.

B). state equivalence and machine minimization:

In realizing the logic diagram from a stat table or state diagram many times we come across redundant states. Redundant states are states whose functions can be accomplished by other states. The elimination of redundant states reduces the total number of states of the machines which in turn results in reduction of the number of flip-flops and logic gates, reducing the cost of the final circuit.

Two states are said to be equivalent. When two states are equivalent, one of them can be removed without altering the input output relationship.

State equivalence theorem: it states that two states s_1 , and s_2 are equivalent if for every possible input sequence applied. The machine goes to the same next state and generates the same output. That is

$$\text{If } S_1(t+1) = s_2(t+1) \text{ and } z_1 = z_2, \text{ then } s_1 = s_2$$

C). distinguishable states and distinguishing sequences:

Two states s_a , and s_b of a sequential machine are distinguishable, if and only if there exists at least one finite input sequence which when applied to the sequential machine causes different outputs sequences depending on whether s_a or s_b is the initial state.

Consider states A and B in the state table, when input $X=0$, their outputs are 0 and 1 respectively and therefore, states A and B are called 1-distinguishable. Now consider states A and E . the output sequence is as follows.

$X=0 \quad A \rightarrow C, 0 \quad \text{and} \quad E \rightarrow D, 0 ; \text{ outputs are the same}$

X=0 A $\rightarrow$ C,0 and C $\rightarrow$ E, 0; outputs are the same

$C \rightarrow E, 0$ and $E \rightarrow D, 0$; outputs are the

Truth table for Distunigshable states:

PS	NS,Z	
	X=0	X=1
A	C,0	F,0
B	D,1	F,0
C	E,0	B,0
D	B,1	E,0
E	D,0	B,0
F	D,1	B,0

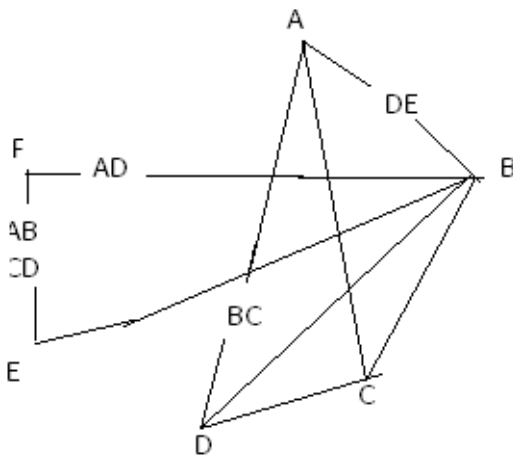
Merger graphs:

1. Each state in the state table is represented by a vertex in the merger graph. So it contains the same number of vertices as the state table contains states.
2. Each compatible state pair is indicated by an unbroken line drawn between the two state vertices
3. Every potentially compatible state pair with non-conflicting outputs but with different next states is connected by a broken line. The implied states are written in the line break between the two potentially compatible states.
4. If two states are incompatible no connecting line is drawn.

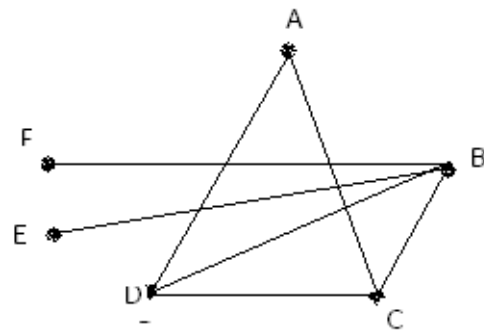
Consider a state table of an incompletely specified machine shown in fig. the corresponding merger graph shown in fig.

State table:

PS	NS,Z			
	I1	I2	I3	I4
A	...	E,1	B,1	...
B	...	D,1	...	F,1
C	F,1	...	...	...
D	...	...	C,1	...
E	C,0	...	A,0	F,1
F	D,0	A,1	B,0	...



a) Merger graph



b) simplified merger graph

States A and B have non-conflicting outputs, but the successor under input I_2 are compatible only if implied states D and E are compatible. So, draw a broken line from A to B with DE written in between. states A and C are compatible because the next states and output entries of states A and C are not conflicting. Therefore, a line is drawn between nodes A and C. states A and D have non-conflicting outputs but the successor under input I_3 are B and C. hence join A and D by a broken line with BC entered in between.

Two states are said to be incompatible if no line is drawn between them. If implied states are incompatible, they are crossed and the corresponding line is ignored. Like, implied states D and E are incompatible, so states A and B are also incompatible. Next, it is necessary to check whether the incompatibility of A and B does not invalidate any other broken line. Observe that states E and F also become incompatible because the implied pair AB is incompatible. The broken lines which remain in the graph after all the implied pairs have been verified to be compatible are regarded as complete lines.

After checking all possibilities of incompatibility, the merger graph gives the following seven compatible pairs.

(A, C) (A, D) (B, C) (B, D) (C, D) (B, E) (B, F)

These compatible pairs are further checked for further compatibility. For example, pairs (B,C)(B,D)(C,D) are compatible. So (B, C, D) is also compatible. Also pairs (A,c)(A,D)(C,D) are compatible. So (A,C,D) is also compatible. . In this way the entire set of compatibles of sequential machine can be generated from its compatible pairs.

To find the minimal set of compatibles for state reduction, it is useful to find what are called the maximal compatibles. A set of compatibles state pairs is said to be maximal, if it is not completely covered by any other set of compatible state pairs. The maximum compatible can be found by looking at the merger graph for polygons which are not contained within any higher order complete polygons. For example only triangles (A, C,D) and (B,C,D) are of higher order. The set of maximal compatibles for this sequential machine given as

(A, C, D) (B, C, D) (B, E) (B, F)

Example:

Draw the merger graph and obtain the set of maximal compatibles for the incompletely specified sequential machine whose state table is given in Table 7.24.

Table 7.24 Example 7.9: State table

PS	NS, Z	
	I ₁	I ₂
A	E, 0	B, 0
B	F, 0	A, 0
C	E, –	C, 0
D	F, 1	D, 0
E	C, 1	C, 0
F	D, –	B, 0

mark × in the corresponding cell. For example, states B and C are incompatible because their outputs are conflicting and hence the cell corresponding to them contains a cross mark ×. Similarly states B, E; D, E; E, F are incompatible. Hence put a × mark in the corresponding cells. On the other hand, states A and B are compatible and hence the cell corresponding to them contains the check mark ✓. Similarly, cells corresponding to states A, D; A, E; A, G; B, G; C, F; D, F ; D, G are also compatible. So a check mark is put in those cells also. The implied pairs or pairs corresponding to the state pair are written within the cell as shown in Table 7.26. For example, states A and C are compatible only when implied states E and F are compatible. Therefore, EF is written in the cell corresponding to states A and C. States C and E are compatible only when implied states A and B, and D and F are compatible. So AB and DF are written in the cell corresponding to states C and E. In a similar way, the entire merger table is written. Now it is necessary to check whether the implied pairs are compatible or not by observing the merger table. The implied states are incompatible if the corresponding cell contains a ×. For example, implied pair E, F is incompatible because cell EF contains a ×. Similarly, implied pairs EF, AF are incompatible because EF contains a ×. It is indicated by a ×.

PS	NS, Z			
	00	01	11	10
A	E, 0	–	–	–
B	–	F, 1	E, 1	A, 1
C	F, 0	–	A, 0	F, 1
D	–	–	A, 1	–
E	–	C, 0	B, 0	D, 1
F	C, 0	C, 1	–	–
G	E, 0	–	–	A, 1

Figure: state table

B	✓					
C	BC	×				
D	✓	AE	×			
E	✓	×	AB DF	×		
F	CE	CF	✓	✓	×	
G	✓	✓	EF AF	✓	AD	CE

State Minimization: Completely Specified Machines

- Two states, s_i and s_j of machine M are *distinguishable* if and only if there exists a finite input sequence which when applied to M causes different output sequences depending on whether M started in s_i or s_j .
- Such a sequence is called a *distinguishing sequence* for (s_i, s_j) .
- If there exists a distinguishing sequence of length k for (s_i, s_j) , they are said to be *k-distinguishable*.

EXAMPLE:

PS	NS, z	
	x=0	x=1
A	E, 0	D, 1
B	F, 0	D, 0
C	E, 0	B, 1
D	F, 0	B, 0
E	C, 0	F, 1
F	B, 0	C, 0

- states A and B are 1-distinguishable, since a 1 input applied to A yields an output 1, versus an output 0 from B.
- states A and E are 3-distinguishable, since input sequence 111 applied to A yields output 100, versus an output 101 from E.
- States s_i and s_j ($s_i \sim s_j$) are said to be equivalent iff no distinguishing sequence exists for (s_i, s_j).
- If $s_i \sim s_j$ and $s_j \sim s_k$, then $s_i \sim s_k$. So state equivalence is an equivalence relation (i.e. it is a reflexive, symmetric and transitive relation).
- An equivalence relation partitions the elements of a set into equivalence classes.
- Property: If $s_i \sim s_j$, their corresponding X-successors, for all inputs X, are also equivalent.
- Procedure: Group states of M so that two states are in the same group iff they are equivalent (forms a partition of the states).

Completely Specified Machines

PS	NS, z	
	x=0	x=1
A	E, 0	D, 1
B	F, 0	D, 0
C	E, 0	B, 1
D	F, 0	B, 0
E	C, 0	F, 1
F	B, 0	C, 0

P_i : partition using distinguishing sequences of length i .

Partition:

Distinguishing Sequence:

$P_0 = (A\ B\ C\ D\ E\ F)$

$P_1 = (A\ C\ E)(B\ D\ F)$

$x = 1$

$P_2 = (A\ C\ E)(B\ D)(F)$

$x = 1; x = 1$

$P_3 = (A\ C)(E)(B\ D)(F)$

$x = 1; x = 1; x = 1$

$P_4 = (A\ C)(E)(B\ D)(F)$

Algorithm terminates when $P_k = P_{k+1}$

Outline of state minimization procedure:

- All states equivalent to each other form an equivalence class. These may be combined into one state in the reduced (quotient) machine.
- Start an initial partition of a single block. Iteratively refine this partition by separating the 1-distinguishable states, 2-distinguishable states and so on.
- To obtain P_{k+1} , for each block B_i of P_k , create one block of states that not 1-distinguishable within B_i , and create different blocks states that are 1-distinguishable within B_i .

Theorem: The equivalence partition is unique.

Theorem: If two states, s_i and s_j , of machine M are distinguishable, then they are $(n-1)$ -distinguishable, where n is the number of states in M .

Definition: Two machines, M_1 and M_2 , are *equivalent* ($M_1 \sim M_2$) if, for every state in M_1 there is a corresponding equivalent state in M_2 and vice versa.

Theorem. For every machine M there is a minimum machine $M_{red} \sim M$. M_{red} is unique up to isomorphism.

Completely Specified Machines

- Reduced machine obtained from previous example:

$$P_4 = (A\ C)(E)(B\ D)(F)$$

$$= \alpha\ \beta\ \gamma\ \delta$$

PS	NS, z	
	x=0	x=1
A	E, 0	D, 1
B	F, 0	D, 0
C	E, 0	B, 1
D	F, 0	B, 0
E	C, 0	F, 1
F	B, 0	C, 0

PS	NS, z	
	x=0	x=1
α	β , 0	γ , 1
β	α , 0	δ , 1
γ	δ , 0	γ , 0
δ	γ , 0	α , 0

State Minimization of CSMs: Complexity

Algorithm DFA $\sim$ DFA_{min}

Input: A finite automaton $M = (Q, \Sigma, \delta, q_0, F)$ with no unreachable states.

Output: A minimum finite automaton $M' = (Q', \Sigma, \delta', q'_0, F')$.

Method:

1. $t := 2$; $Q_0 := \{ \text{undefined} \}$; $Q_1 := F$; $Q_2 := Q \setminus F$.
 2. while there is $0 < i \leq t$, $a \in \Sigma$ with $\delta(Q_i, a) \subseteq Q_j$, for all $j \leq t$
do (a) Choose such an i , a , and $j \leq t$ with $\delta(Q_i, a) \cap Q_j \neq \emptyset$.
 (b) $Q_{t+1} := \{ q \in Q_i \mid \delta(q, a) \in Q_j \}$;
 $Q_i := Q_i \setminus Q_{t+1}$;
 $t := t + 1$.
- end.
3. (* Denote $[q]$ the equivalence class of state q , and $\{Q_i\}$ the set of all equivalence classes. *)
 $Q' := \{Q_1, Q_2, \dots, Q_t\}$.
 $q'_0 := [q_0]$.
 $F' := \{ [q] \in Q' \mid q \in F \}$.
 $\delta'([q], a) := [\delta(q, a)]$ for all $q \in Q$, $a \in \Sigma$.

Standard implementation: $O(kn^2)$, where $n = |Q|$ and $k = |\Sigma|$

Modification of the body of the while loop:

1. Choose such an i , $a \in \Sigma$, and choose $j_1, j_2 \leq t$ with $j_1 \neq j_2$, $\delta(Q_i, a) \cap Q_{j_1} \neq \emptyset$, and $\delta(Q_i, a) \cap Q_{j_2} \neq \emptyset$.
2. If $|\{q \in Q_i \mid \delta(q, a) \in Q_{j_1}\}| \leq |\{q \in Q_i \mid \delta(q, a) \in Q_{j_2}\}|$

then $Q_{t+1} := \{q \in Q_i \mid \delta(q,a) \in Q_{j1}\}$
 else $Q_{t+1} := \{q \in Q_i \mid \delta(q,a) \in Q_{j2}\}$ fl;
 $Q_i := Q_i \setminus Q_{t+1};$
 $t := t + 1.$
 (i.e. put smallest set in $t + 1$)

Note: $|Q_{t+1}| \leq 1/2|Q_i|$. Therefore, for all $q \in Q$, the name of the class which contains a given state q changes at most $\log(n)$ times.

Goal: Develop an implementation such that all computations can be assigned to transitions containing a state for which the name of the corresponding class is changed.

Suitable data structures achieve an $O(kn \log n)$ implementation.

State Minimization:

Incompletely Specified Machines

Statement of the problem: given an incompletely specified machine M , find a machine M' such that:

- on any input sequence, M' produces the same outputs as M , whenever M is specified.
- there does not exist a machine M'' with fewer states than M' which has the same property

Machine M :

PS	NS, z	
	x=0	x=1
s1	s3, 0	s2, 0
s2	s2, -	s3, 0
s3	s3, 1	s2, 0

Attempt to reduce this case to usual state minimization of completely specified machines.

- Brute Force Method: Force the don't cares to all their possible values and choose the smallest of the completely specified machines so obtained.
- In this example, it means to state minimize two completely specified machines obtained from M , by setting the don't care to either 0 and 1.

Suppose that the - is set to be a 0.

PS	NS, z	
	x=0	x=1
s1	s3, 0	s2, 0
s2	s2, 0	s3, 0
s3	s3, 1	s2, 0

- States s1 and s2 are equivalent if s3 and s2 are equivalent, but s3 and s2 assert different outputs under input 0, so s1 and s2 are not equivalent.
- States s1 and s3 are not equivalent either.

- So this completely specified machine cannot be reduced further (3 states is the minimum).

Suppose that the - is set to be a 1.

PS	NS, z	
	x=0	x=1
s1	s3, 0	s2, 0
s2	s2, 1	s3, 0
s3	s3, 1	s2, 0

- States s1 is incompatible with both s2 and s3.
- States s3 and s2 are equivalent.
- So number of states is reduced from 3 to 2.

Machine M''_{red} :

PS	NS, z	
	x=0	x=1
A	A, 1	A, 0
B	B, 0	A, 0

Can this always be done?

Machine M :

PS	NS, z	
	x=0	x=1
s1	s3, 0	s2, 0
s2	s2, -	s1, 0
s3	s1, 1	s2, 0

Machine M_2 :

PS	NS, z	
	x=0	x=1
s1	s3, 0	s2, 0
s2	s2, 0	s1, 0
s3	s1, 1	s2, 0

Machine M_3 :

PS	NS, z	
	x=0	x=1
s1	s3, 0	s2, 0
s2	s2, 1	s1, 0
s3	s1, 1	s2, 0

Machine M_2 and M_3 are formed by filling in the unspecified entry in M with 0 and 1, respectively.

Both machines M_2 and M_3 cannot be reduced.

Conclusion?: M cannot be minimized further!

But is it a correct conclusion?

Note: that we want to merge two states when, for any input sequence, they generate the same output sequence, but only where both outputs are specified.

Definition: A set of states is compatible if they agree on the outputs where they are all specified.

Machine M'' :

PS	NS, z	
	x=0	x=1
s1	s3, 0	s2, 0
s2	s2, -	s1, 0
s3	s1, 1	s2, 0

In this case we have two compatible sets: $A = (s1, s2)$ and $B = (s3, s2)$. A reduced machine M_{red} can be built as follows.

Machine M_{red}

PS	NS, z	
	x=0	x=1
A	A, 1	A, 0
B	B, 0	A, 0

PS	I1	NS, z		I4
		I2	I3	
s1	s3, 0	s1, -	-	-
s2	s6, -	s2, 0	s1, -	-
s3	-, 1	-, -	s4, 0	-
s4	s1, 0	-, -	-	s5, 1
s5	-, -	s5, -	s2, 1	s1, 1
s6	-, -	s2, 1	s6, -	s4, 1

A set of compatibles that cover all states is: $(s3s6)$, $(s4s6)$, $(s1s6)$, $(s4s5)$, $(s2s5)$.

But $(s3s6)$ requires $(s4s6)$,

$(s4s6)$ requires $(s4s5)$, $(s4s5)$ requires $(s1s5)$,

$(s1s6)$ requires $(s1s2)$, $(s1s2)$ requires $(s3s6)$,

$(s2s5)$ requires $(s1s2)$.

So, this selection of compatibles requires too many other compatibles...

PS	NS, z			
	I1	I2	I3	I4
s1	s3,0	s1,-	-	-
s2	s6,-	s2,0	s1,-	-
s3	-,1	-, -	s4,0	-
s4	s1,0	-, -	-	s5,1
s5	-, -	s5,-	s2,1	s1,1
s6	-, -	s2,1	s6,-	s4,1

- Another set of compatibles that covers all states is $(s1s2s5)$, $(s3s6)$, $(s4s5)$.
- But $(s1s2s5)$ requires $(s3s6)$ $(s3s6)$ requires $(s4s6)$
- $(s4s6)$ requires $(s4s5)$ $(s4s5)$ requires $(s1s5)$.
- So must select also $(s4s6)$ and $(s1s5)$.
- Selection of minimum set is a bipartite covering problem

When a next state is unspecified, the future behavior of the machine is unpredictable. This suggests the definition of admissible input sequence.

Definition. An input sequence is *admissible*, for a starting state of a machine if no unspecified next state is encountered, except possibly at the final step.

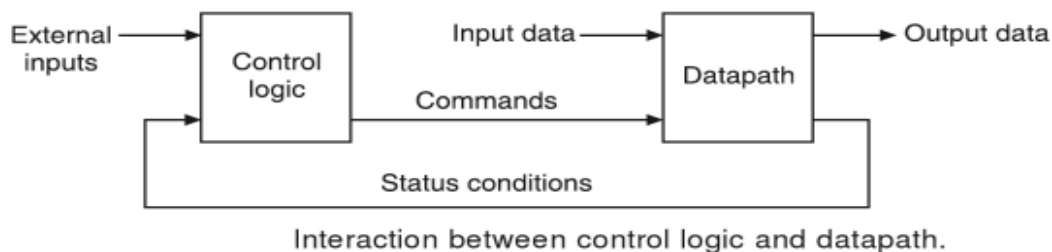
Definition. State s_i of machine M_1 is said to *cover*, or *contain*, state s_j of M_2 provided

1. every input sequence admissible to s_j is also admissible to s_i , and
2. its application to both M_1 and M_2 (initially is s_i and s_j , respectively) results in identical output sequences whenever the outputs of M_2 are specified.

Definition. Machine M_1 is said to cover machine M_2 if for every state s_j in M_2 , there is a corresponding state s_i in M_1 such that s_i covers s_j .

Algorithmic State Machines:

- The binary information stored in the digital system can be classified as either data or control information.
- The data information is manipulated by performing arithmetic, logic, shift and other data processing tasks.
- The control information provides the command signals that controls the various operations on the data in order to accomplish the desired data processing task.
- Design a digital system we have to design two subsystems data path subsystem and control subsystem.



ASM CHART:

- A special flow chart that has been developed specifically to define digital hardware algorithms is called ASM chart.
- A hardware algorithm is a step by step procedure to implement the desire task.

Difference b/n conventional flow chart and ASM chart:

- conventional flow chart describes the sequence of procedural steps and decision paths for an algorithm without concern for their time relationship
- An ASM chart describes the sequence of events as well as the timing relationship b/n the states of sequential controller and the events that occur while going from one state to the next

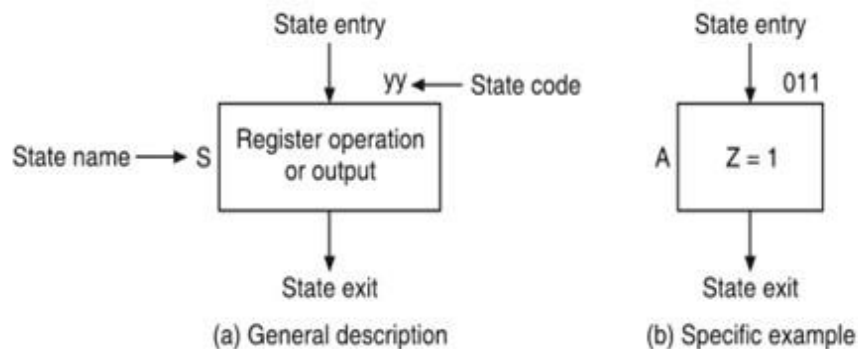
1. State box: A state of a clocked sequential circuit is represented by a rectangle called *state box*. It is equivalent to a node in the state diagram or a row in the state table. The name of the state is written to the left of the box. The binary code assigned to the state is indicated outside on the top right-side of the box. A list of unconditional outputs if any associated with the state are written within the box.

2. Decision box: The decision box or condition box is represented by a diamond-shaped symbol with one input and two or more output paths. The output branches are true and false branches. The decision box describes the effect of an input on the control subsystem. A Boolean variable or input or expression written inside the diamond indicates a condition which is evaluated to determine which branch to take.

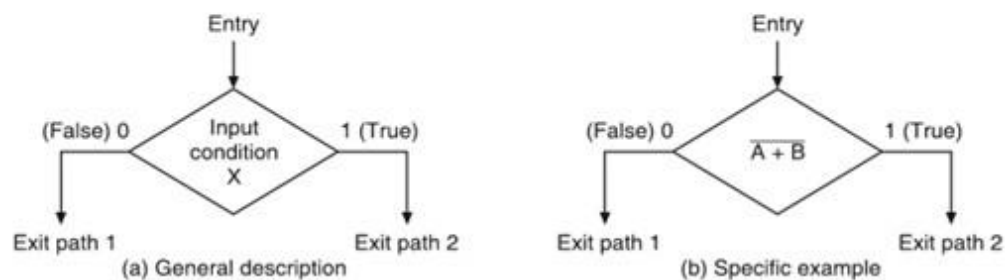
ASM consists of

1. State box
2. Decision box
3. Conditional box

State box

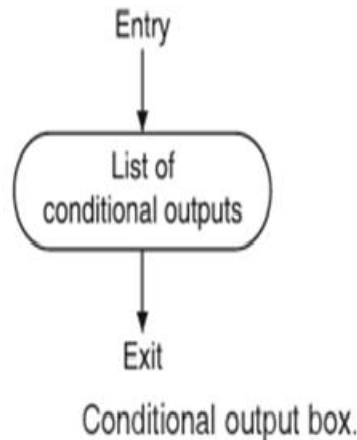


Decision box



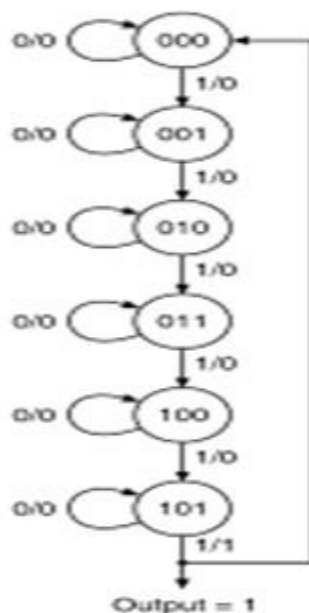
Decision box.

3. Conditional output box: The conditional output box is represented by a rectangle with rounded corners or by an oval with one input line and one output line. The outputs that depend on both the state of the system and the inputs are indicated inside the box.

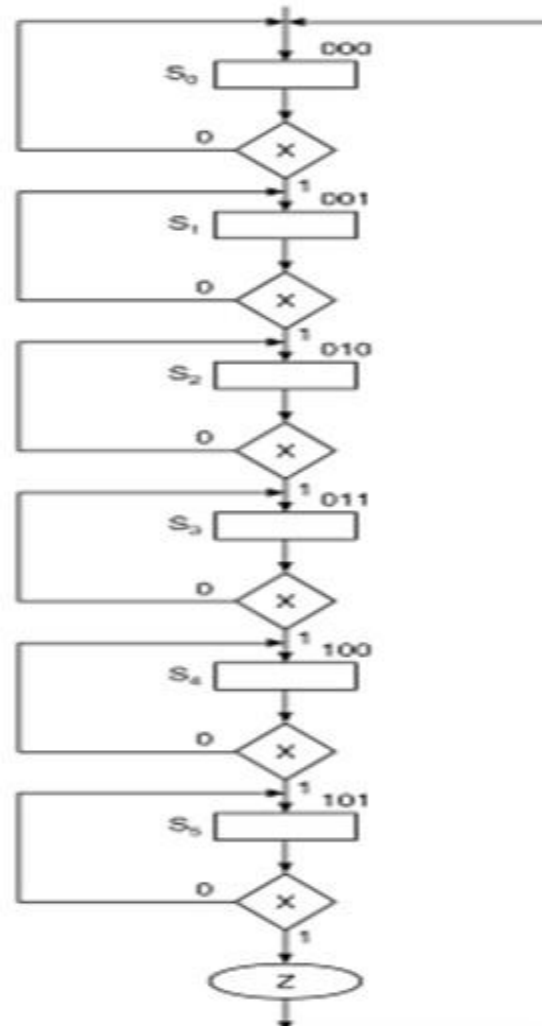


SALIENT FEATURES OF ASM CHARTS

1. An ASM chart describes the sequence of events as well as the timing relationship between the states of a sequential controller and the events that occur while going from one state to the next.
2. An ASM chart contains one or more interconnected ASM blocks.
3. Each ASM block contains exactly one state box together with the decision boxes and conditional output boxes associated with that state.
4. Every block in an ASM chart specifies the operations that are to be performed during one common clock pulse.
5. An ASM block has exactly one entrance path and one or more exit paths represented by the structure of the decision boxes.
6. A path through an ASM block from entrance to exit is referred to as a link path.
7. The operations specified within the state and conditional output boxes in the block are performed in the datapath subsystem.
8. Internal feedback within an ASM block is not permitted. Even so, following a decision box or conditional output boxes, the machine may reenter the same state.
9. Each block in the ASM chart describes the state of the system during one clock pulse interval. When a digital system enters the state associated with a given ASM block, the outputs indicated within the state box become true. The conditions associated with the decision boxes are evaluated to determine which path or paths to be followed to enter the next ASM block.



(a) State diagram



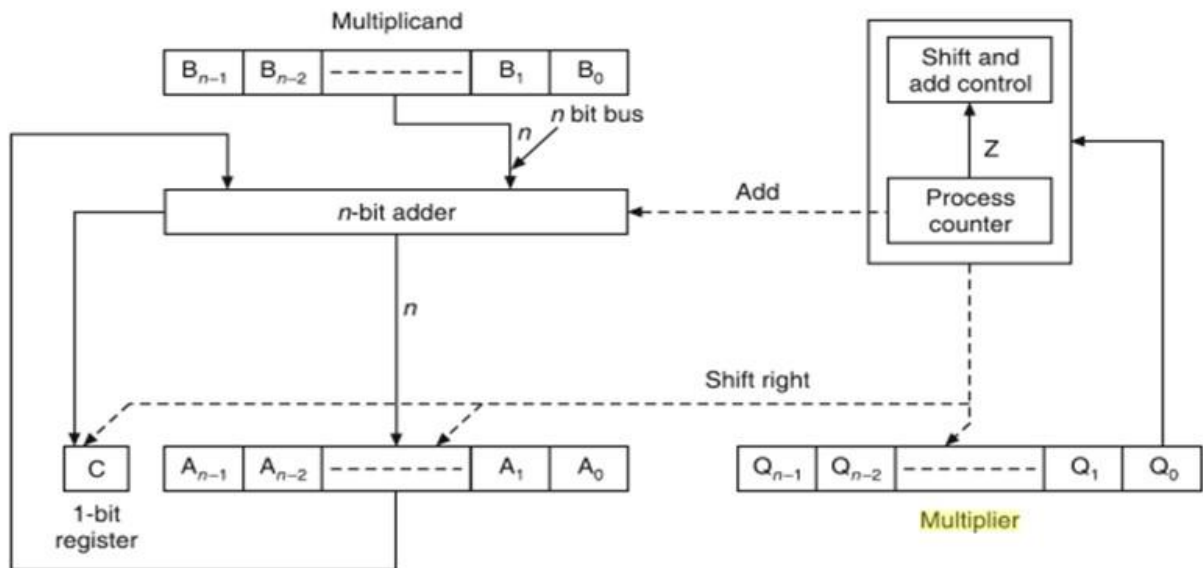
(b) ASM chart

State diagram and ASM chart for mod-6 counter.

BINARY MULTIPLIER

1 1 0 1	←	13_{10} ... Multiplicand
1 0 1 0	←	10_{10} ... Multiplier
0 0 0 0	←	Partial product 1
1 1 0 1	←	Partial product 2
0 0 0 0	←	Partial product 3
1 1 0 1	←	Partial product 4
1 0 0 0 0 0 1 0	←	130_{10} ... Product

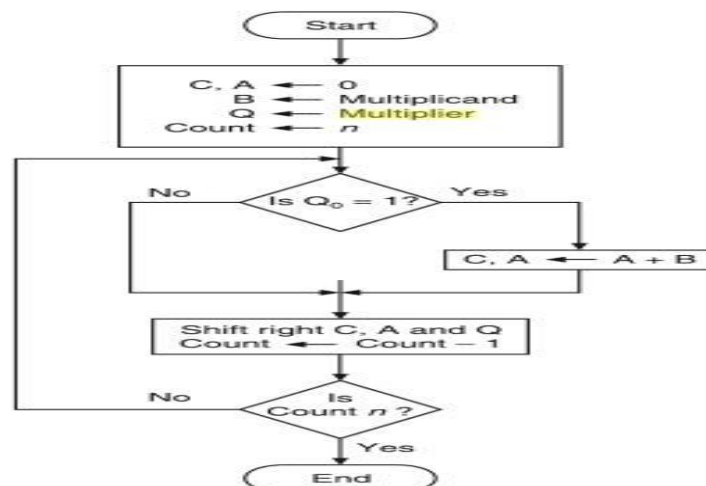
Data path subsystem for binary multiplier



Datapath subsystem for binary multiplier.

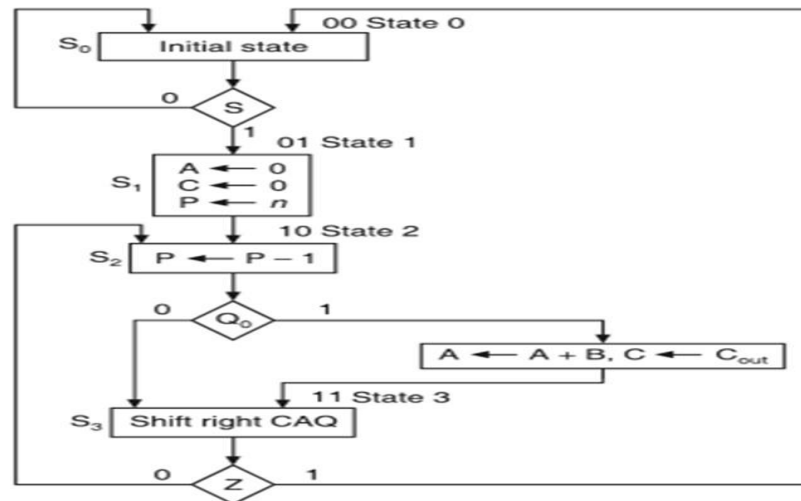
Multiplication Operation Steps

1. Bit 0 of multiplier operand (Q_0 of Q register) is checked.
2. If bit 0 (Q_0) is one then multiplicand and partial product are added and all bits of C, A and Q registers are shifted to the right one bit, so that the C bit goes into A_{n-1} , A_0 goes into Q_{n-1} , and Q_0 is lost. If bit 0 (Q_0) is 0, then no addition is performed, only shift operation is carried out.
3. Steps 1 and 2 are repeated n times to get the desired result in the A and Q registers.



B	C	A	Q	Components	Count P
1 1 0 1	0	0 0 0 0	1 0 1 0	$B \leftarrow \text{Multiplicand}$ $Q \leftarrow \text{Multiplier}$ $A \leftarrow 0, C \leftarrow 0, P \leftarrow n$	100 (4)
1 1 0 1	0	0 0 0 0	1 0 1 0	$P \leftarrow P - 1$ $Q_0 = 0$	011 (3)
	0	0 0 0 0	0 1 0 1	C A Q shifted right	
1 1 0 1	0	1 1 0 1	0 1 0 1	$P \leftarrow P - 1$ $Q_0 = 1, A \leftarrow A + B$	010 (2)
	0	0 1 1 0	1 0 1 0	C A Q shifted right	
1 1 0 1	0	0 1 1 0	1 0 1 0	$P \leftarrow P - 1$ $Q_0 = 0,$	001 (1)
	0	0 0 1 1	0 1 0 1	C A Q shifted right	
1 1 0 1	1	0 0 0 0	0 1 0 1	$P \leftarrow P - 1$ $Q_0 = 1, A \leftarrow A + B$	000 (0)
	0	1 0 0 0	0 0 1 0	C A Q shifted right	

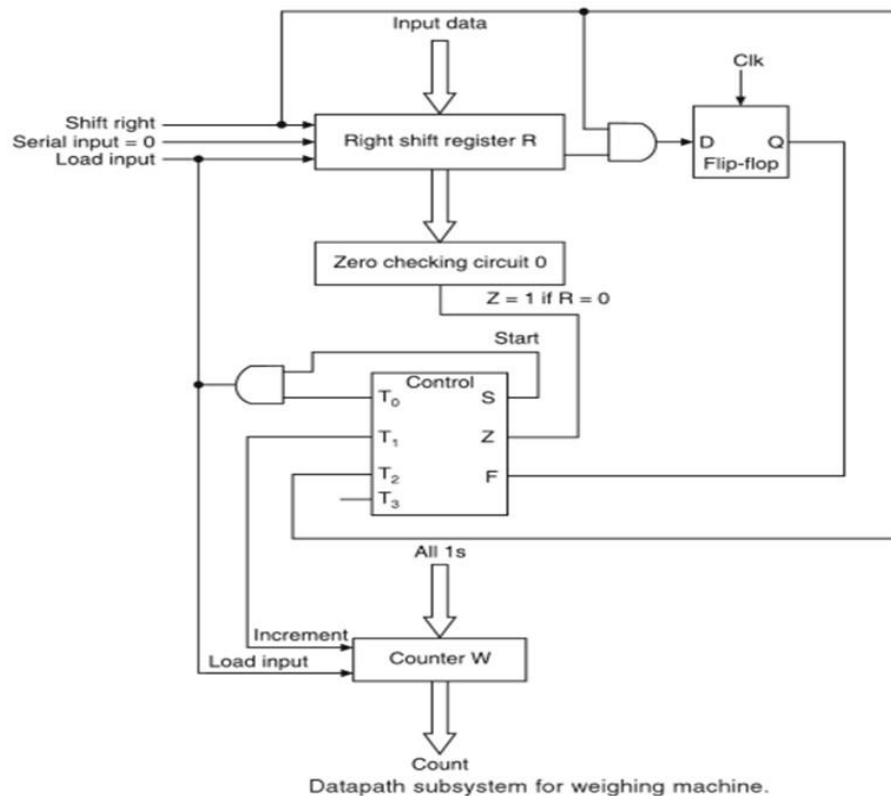
Flow chart for multiplication in a computer.



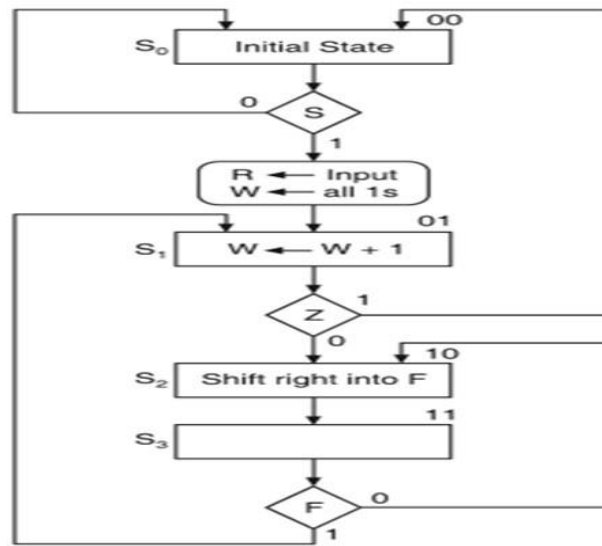
ASM chart for a binary multiplier.

ASM FOR WEIGHING MACHINE

In the algorithm for tabular minimization of Boolean expressions, we have to arrange the minterms in the ascending order of their weights. This is only one of the many situations when we have to examine the 1s of a given binary word. The weight of a binary number is defined as the number of 1s present in its binary representation.



Datapath subsystem for weighing machine.



ASM chart for weighing machine.

State S_0 : Initially the weighing machine is in state S_0 . The weighing process starts when start (S) signal becomes 1. While in state S_0 , if S is 1, the clock pulse causes three jobs to be done simultaneously:

1. Binary number is loaded into register R.
2. W register is set to all 1s.
3. The machine is transferred to state S_1 .

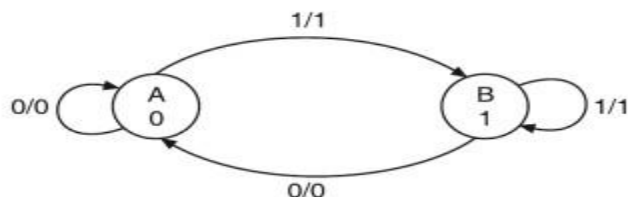
State S_1 : While in state S_1 , the clock pulse causes two jobs to be done simultaneously:

1. Counter W is incremented by 1 (in the first round, all 1s become all 0s).
2. If Z is 0, the machine goes to the state S_2 ; if Z is 1, the machine goes to state S_0 .

State S_2 : In this state, register R is shifted right by 1 bit so that LSB goes into F and MSB is loaded with 0.

State S_3 : In this state, the value of F is checked. If it is 0, the machine is transferred to the state S_2 , otherwise the machine is transferred to state S_1 . Thus, when $F = 1$, W is incremented.

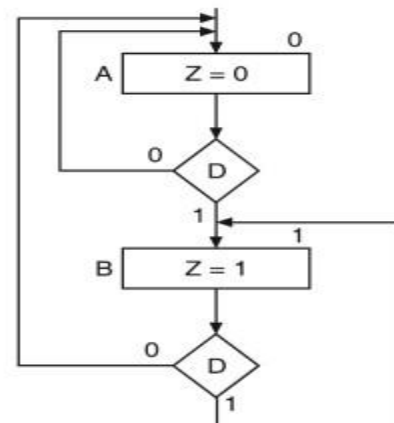
All the operations occur in coincidence with the clock pulse while in the corresponding state. Also notice that the register R should eventually contain all 0s when the last 1 is shifted into it.



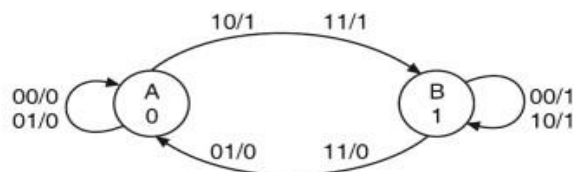
(a) State diagram

PS	NS, O/P Input D	
	D = 0	D = 1
A	A, 0	B, 1
B	A, 0	B, 1

(b) State table



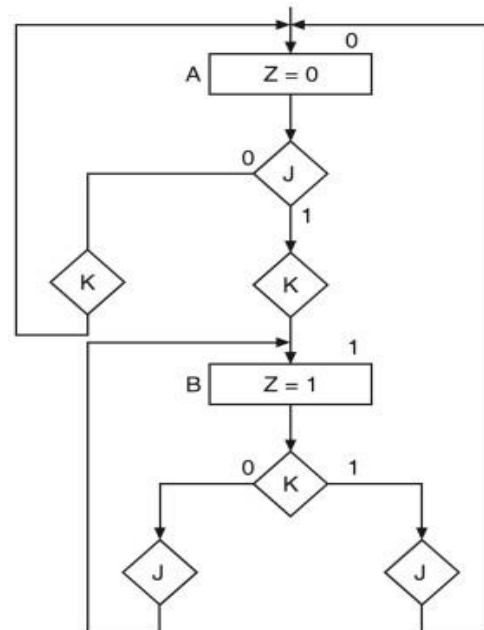
(c) ASM chart



(a) State diagram

PS	NS, O/P Input J-K			
	00	01	10	11
A	A, 0	A, 0	B, 1	B, 1
B	B, 1	A, 0	B, 1	A, 0

(b) State table



(c) ASM chart